Workers for LabVIEW

v5.0

SCARFE CONTROLS

Training Course Manual

Table of Contents

Course Exercises

Introduction

Welcome to the official Workers for LabVIEW Training Course! You might be wondering why another framework is needed for creating modular, asynchronous, multi-process applications in LabVIEW when several already exist. Workers for LabVIEW was developed to address specific challenges that existing frameworks do not fully solve.

Workers for LabVIEW sets itself apart by bridging the gap between the simplicity of easy-to-read, traditional LabVIEW code and the flexibility of advanced LabVIEW Object-Oriented Programming (LVOOP) frameworks. It allows you to develop applications using the familiar LabVIEW Queued Message Handler (QMH) template style of development - a design pattern taught in the NI LabVIEW Core 3 course and a cornerstone of numerous multi-process applications that have been created by the LabVIEW community over the last several decades.

If adopting an object-oriented framework seems daunting to you, rest assured that LVOOP is utilized minimally within the framework - only where it adds significant value. This approach ensures ease of use while simultaneously providing the benefits of object-oriented design. The goal of Workers for LabVIEW is to provide novice LabVIEW developers with the ability to create large, scalable multi-process applications while gradually introducing LVOOP features as they gain experience.

For advanced developers, Workers for LabVIEW facilitates the creation of complex applications in an easy-to-read format, making code more maintainable and accessible to a wider range of developers over time. This readability enhances collaboration and ensures that your codebase can be efficiently expanded or modified by others in the future.

This training course is divided into two parts. **Part 1** focuses on teaching you how to use the framework in its classic LabVIEW QMH template form, with minimal discussion of LVOOP. Any developer can use the knowledge gained in this part to develop modular and scalable applications. **Part 2** introduces the LVOOP features of the framework, showing you how to further enhance your applications with simple object-oriented techniques, assisted by use of the framework's scripted development tools.

As hardware capabilities have exponentially increased since LabVIEW's introduction, the potential to develop larger and more advanced applications has grown significantly. Frameworks like Workers for LabVIEW are essential for utilizing this potential, ensuring that LabVIEW remains a relevant and powerful programming language for the future. The goal of Workers for LabVIEW is to be both easy to use and rich in advanced LVOOP features, bridging the gap between simplicity and advanced flexibility.

By the end of this course, you will not only understand how to effectively use the framework but also be equipped to build robust, scalable, and maintainable applications that leverage the full power of modern software development practices. We are excited to embark on this journey with you and look forward to seeing the innovative solutions you will create using Workers for LabVIEW.

Happy programming. 😊

Course Pre-requisites

We recommend that you have a basic understanding of LabVIEW development, including the material covered in the NI LabVIEW Core 1, Core 2, and Core 3 courses. Even if you haven't taken the official NI LabVIEW training courses, you should have knowledge and experience in developing applications using the LabVIEW Queued Message Handler (QMH) design pattern. The Workers framework is a natural extension of this development style, and its basic concepts and design are directly based on the LabVIEW QMH template. Therefore, we assume you are familiar with this design pattern and the terminology associated with its components.

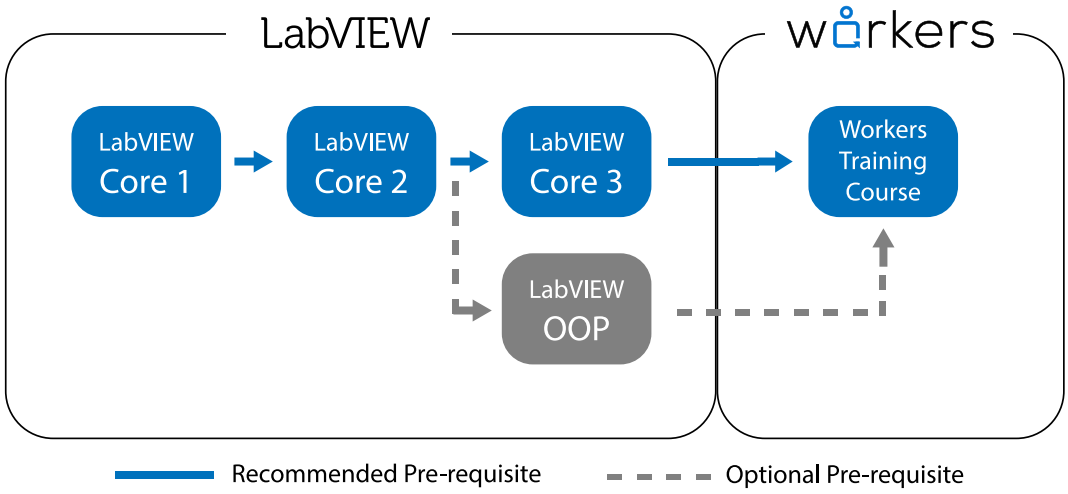

Figure 1 - Recommended Course Pre-requisite training

Although Workers for LabVIEW is built using LabVIEW Object-Oriented Programming (LVOOP), prior knowledge of LVOOP is not required to begin using the framework comfortably. However, understanding LVOOP concepts will help you maximize the framework's capabilities. This training course is divided into two parts:

- **Part 1** requires no prior knowledge of LVOOP and focuses on using the framework in its classic LabVIEW QMH form.
- **Part 2** introduces LVOOP principles and discusses how you can leverage them to enhance your application development with the framework.

Course Description

This course consists of a conceptual manual accompanied by corresponding exercises. Each exercise demonstrates the practical application of the concepts discussed. The course is divided into two main parts:

Part 1: Basics and Fundamentals

This part introduces the Workers framework, explaining the four design pillars upon which the framework is predicated, followed by an explanation of what a Worker is and its various components. You will learn how to combine Workers to form a multi-process asynchronous LabVIEW application and how to create and use local and public APIs to send messages between Workers within an application. This section also introduces the framework's core development tools including the Workers Debug Server application.

The use of LabVIEW Object-Oriented Programming (LVOOP) will not be discussed in detail in this section. By the end of Part 1, you will be able to comfortably develop multi-process, Queued Message Handler (QMH)-based LabVIEW applications in a scalable and modular way. Many developers find that this foundational knowledge is sufficient to begin developing with the framework and can apply this knowledge to create real-world projects.

Part 2: The LVOOP features

Building upon the foundations learnt in Part 1, this part introduces the LVOOP features of the framework and shows you how to use them to further enhance your applications. We will discuss concepts such as class inheritance, overriding framework API VIs, creating and using Worker base classes, and extending and overriding base class Message Handling Loop (MHL) cases. You will also learn how to create public API abstractions for your Workers using the framework's development tools. This is the foundation for creating Hardware Abstraction Layers (HALs), a highly useful feature of the framework.

Note: Prior knowledge of LVOOP is not required for this part of the course, but familiarity with object-oriented programming concepts will enhance your understanding. We will introduce basic LVOOP concepts as needed throughout this part. Gaining additional LVOOP knowledge independently afterward will further enhance your development capabilities with the framework. You can always refer back to this course material to refresh the concepts taught.

Course Goals

After completing this course, you will:

1. Have a solid understanding of the Workers framework, including the four pillars its design is predicated on, to create modular, scalable, and maintainable applications using the LabVIEW Queued Message Handler (QMH) design pattern.
2. Learn how to combine multiple Workers to form a hierarchy of modular processes, to develop asynchronous, multi-process LabVIEW applications and know how to communicate between them using asynchronous priority messages.
3. Know how to create Local and Public Worker APIs to ensure modular, decoupled design, enabling reusable Worker modules that can be shared across projects or with other developers.
4. Understand how to use the framework's LabVIEW Object-Oriented Programming (LVOOP) features such as inheritance, abstraction, and dynamic dispatch to reduce code duplication, maintain clean architectures, and develop API abstractions such as Hardware Abstraction Layers (HALs).
5. Use the Workers Debug Server to monitor, debug, and troubleshoot applications in real time, by understanding how to use the tool's Application Manager and Message Logs.
6. Know how to use the framework's scripted tools, allowing you to create Workers, visualize Worker call-chain hierarchies, and use the framework's QuickDrop shortcuts.
7. Learn best practices for designing scalable and maintainable architectures, employing hierarchical structures, loose coupling, and plug-in capabilities to enhance application design flexibility and reliability.
8. Gain practical experience through guided exercises and learn how to create Worker APIs that can be called externally from other LabVIEW frameworks or application's such as NI TestStand.

What you need to Get Started

This guide assumes that you have already installed the following software:

- LabVIEW 2017 or later
- Workers SDK v5.0

The Workers SDK v5.0 can be downloaded through the VI Package Manager.

VIPM download link here: https://www.vipm.io/package/sc_workers/

Part 1

Basics and Fundamentals

In this section, you will be introduced to the Workers framework and the rationale behind its design. The fundamental components of a Worker and its various parts will be discussed. You'll learn how to combine Workers to create multi-process, asynchronous LabVIEW applications and how to create local and public APIs for inter-Worker communication.

This section also introduces the essential Workers development tools, including the Workers Debug Server application, which is used for debugging your Workers applications over a local network.

Although the LabVIEW Object-Oriented Programming (LVOOP) features of the framework will not be discussed here, they will be briefly mentioned. By the end of this section, you will be able to develop multi-process, QMH-style LabVIEW applications in an efficient and scalable way.

The four pillars of the framework

Before diving into the specifics, it is essential to understand the four core pillars upon which the Workers framework is built. These pillars ensure that the framework is adaptable, robust, easy to use, and maintainable over an application's lifetime.

Pillar 1 : LVOOP Modularity uses LabVIEW Object-Oriented Programming to create flexible modules that are easy to maintain, abstract and extend.

Pillar 2 : Hierarchy Architecture helps to ensure that the modular components of an application are well-structured and scalable, avoiding the growth of disorganized “spaghetti” architecture.

Pillar 3 : Plug-in Architecture allows modules to be loaded, unloaded or exchanged on demand at runtime without disrupting the core application.

Pillar 4 : The **LabVIEW QMH template** style of development provides users with an easy and well-known development style, making the framework accessible to a wide range of developers.

Together, these pillars form a synergistic approach that ensures the framework can handle the requirements of modern software development. In this section, we will dive deeper into each of these pillars, exploring how they contribute to the overall strength and flexibility of the Workers framework.

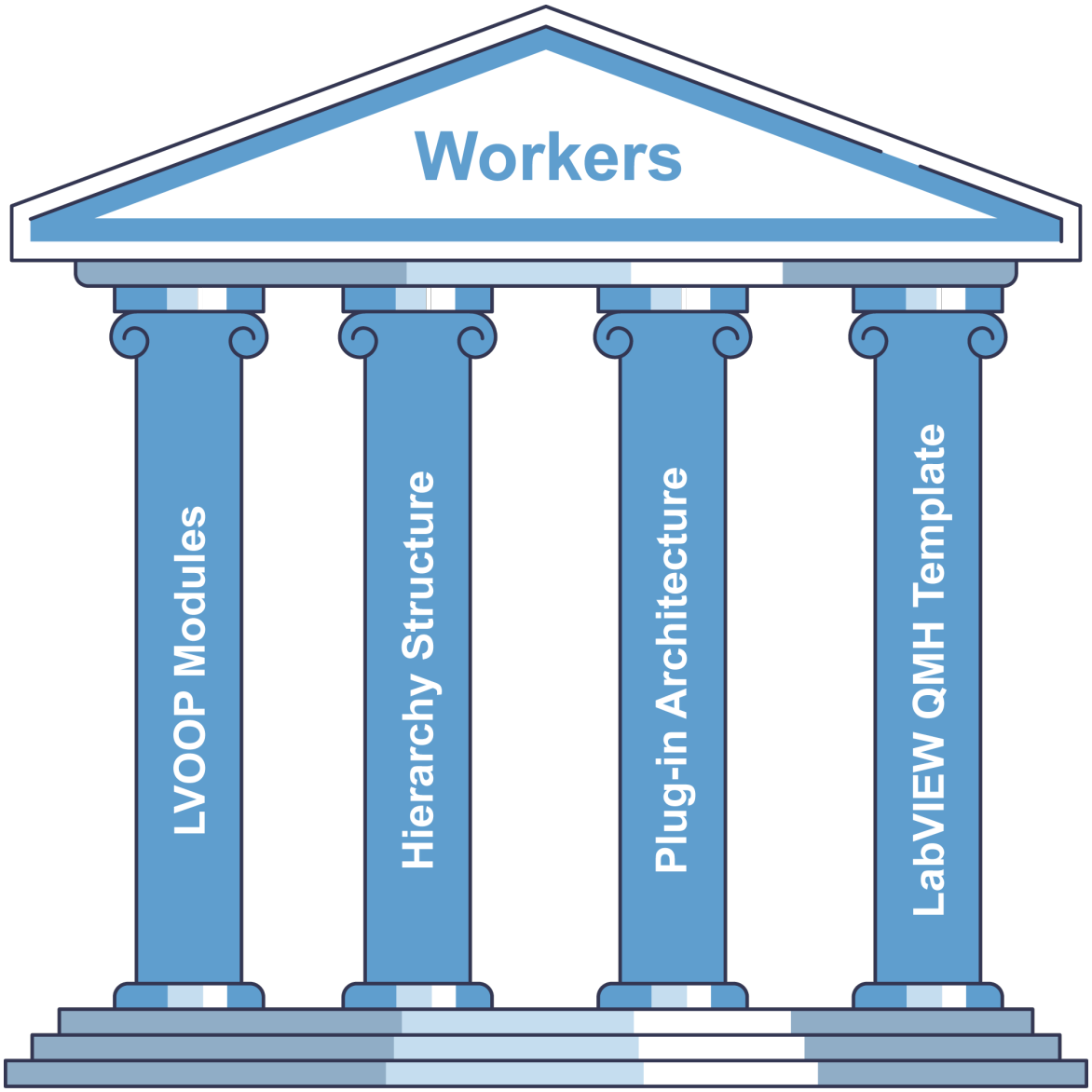

Figure 2 – The Workers framework is designed around these four software development pillars

LVOOP Modules

Modularity

Modularity is a core principle in software engineering that involves dividing complex systems into smaller, self-contained components known as modules. Each module performs a specific task and interacts with other modules through well-defined interfaces. This approach simplifies development by organizing systems into manageable parts, allowing developers to focus on individual elements without being overwhelmed by the system's overall complexity.

Key Advantages of Modularity:

- **Encapsulation:** Modules hide their internal details from the rest of the system. By using clear interfaces like public APIs, developers can update or modify a module without

affecting other parts of the system. This promotes flexibility and maintainability, reducing the risk of introducing bugs elsewhere.

- **Maintainability:** Each module can be tested, debugged, and updated independently. Modular testing reduces the likelihood of widespread issues, making it easier to identify and fix bugs efficiently.
- **Interchangeability:** When modules adhere to agreed-upon interfaces such as a standardized Public API, they can be replaced or upgraded without impacting the rest of the system. This allows for easier system updates over time.
- **Reusability:** Modules can be reused across different projects, saving development time and effort. This reusability, combined with the ability to scale by adding new modules, ensures that systems can evolve to meet changing demands.
- **Scalability:** New modules can be added to enhance functionality without disrupting the existing structure. This is particularly important in large systems.
- **Flexibility:** Developers can update or replace individual modules without overhauling the entire system. As new requirements arise, systems can evolve without extensive rewrites.
- **Enhanced Testing and Quality Assurance:** Individual modules can be tested independently before integration, ensuring that each part functions correctly in isolation.
- **Collaboration:** Different teams can work on separate modules simultaneously, accelerating development. By assigning specialized teams to focus on distinct components, modular systems allow for more efficient and specialized development efforts.

In summary, modularity simplifies the development, maintenance, and evolution of complex systems by dividing them into smaller, manageable components. By enhancing flexibility, scalability, and collaboration, modularity stands as a fundamental pillar in the framework, making Workers applications easier to manage, update, and extend over time.

The advantages of LVOOP Modules

While modularity alone offers numerous benefits, these can be further enhanced by creating modules using LabVIEW Object-Oriented Programming (LVOOP).

In the Workers framework, each module is implemented as a LabVIEW Class. Using LVOOP provides each module with additional features over using a standard LabVIEW library to modularize units of specific, tightly coupled code within the framework. LVOOP facilitates inheritance, encapsulation, and polymorphism, which enhance code reuse and maintainability.

The advantages provided by LVOOP to the framework's modules will be discussed in greater detail in Part 2 of the course, starting from page 87.

Hierarchy Architecture

During the development of the framework, an important decision had to be made: Should each Worker's queue be globally accessible to all other modules, or should it be accessible only to its Caller and sub-workers? This section explains why a hierarchical architecture—where each Worker's queue is accessible only within its immediate hierarchy—was chosen.

Minimizing Spaghetti Architecture

In software design, spaghetti architecture describes a system of modules lacking clear organization, often tangled with dependencies and interactions. And just as you can create spaghetti code in LabVIEW, a modular LabVIEW application, when zoomed out to the architectural level, can also resemble spaghetti architecture.

The primary causes of spaghetti architecture include poor initial planning, tight coupling between modules, and the accumulation of technical debt, resulting from quick fixes and ad hoc changes while developing. Tight coupling occurs when modules have global access to each other's data or message queues, so changes to one module can adversely affect many others, entangling system functionality. Maintaining such systems over time becomes difficult because these tangled dependencies make isolating and fixing issues challenging.

To minimize spaghetti architecture, a framework that adopts a structured modular hierarchy of responsibilities, and minimizes inter-module dependencies, results in applications that are easier to maintain, scale, and debug, enhancing long-term maintainability and reliability.

A Structured Hierarchy of Asynchronous Processes

Good architecture is important in any field of engineering, and the development of larger systems require an organized architectural framework. Workers for LabVIEW employs hierarchical architecture (see Figure 3) to provide structure and organization to systems made out of multiple asynchronous processes. This approach ensures that each process can contribute to the overall system without becoming cross-dependent with one another.

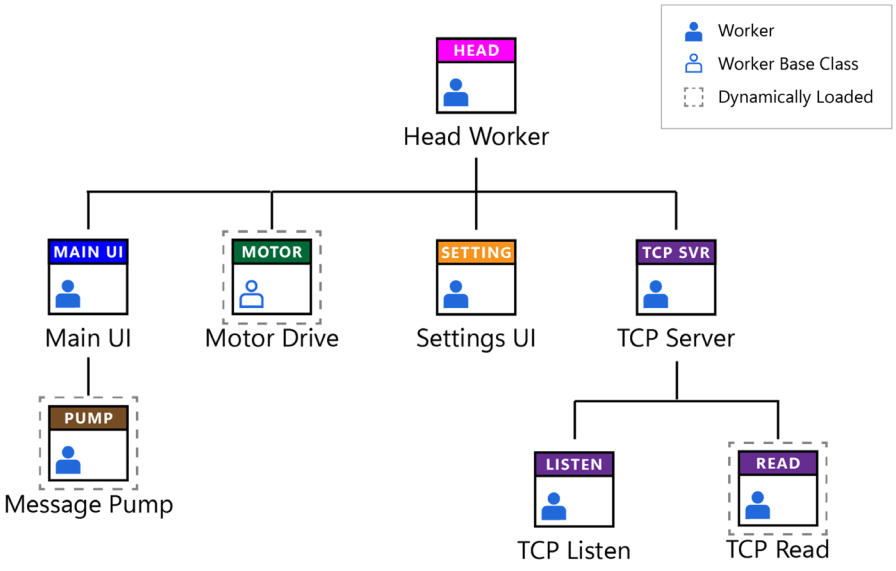

Figure 3 - Example of a Workers application Worker call-chain, with Workers arranged in a hierarchy

Supports Modularity and Scalability

Modularity and scalability are natural strengths of hierarchical architecture. Each process functions as a self-contained module with well-defined interfaces, making development, testing, and maintenance more manageable. Modules can be updated or extended without affecting other parts of the system, reducing risks and ensuring that the system remains adaptable as requirements evolve.

When scalability is needed, new modules or branches can be easily integrated into the hierarchy. This allows for growth without requiring major architectural re-designs. For example, adding a new feature might involve introducing a new child process under an existing parent, leaving the rest of the system untouched.

Clearer Communication and Coordination

Communication and coordination in hierarchical architecture are handled through local messaging, where parent processes manage child processes in a clear parent-child relationship. This approach ensures that all processes also have a clear hierarchy of responsibilities and can initialize and shut down in a synchronized and structured way.

By using decoupled Public APIs, communication paths are restricted to parent-child processes, preventing unnecessary dependencies. This structure also clarifies the application's message routing pathways and reduces the likelihood of coupling modules together that exist across branches in the modular hierarchy.

Isolated Fault Tolerance and Error Handling

Fault tolerance and error handling are significantly improved by this architecture. Hierarchical architecture isolates failures within specific branches, allowing the system to contain errors and prevent them from affecting unrelated processes. This isolation leads to graceful degradation, where non-critical processes can be terminated or restarted without impacting the rest of the application.

For instance, if a child process responsible for a specific hardware device fails, it can be restarted by its parent without disrupting the main application. The ability to handle failures modularly ensures that systems remain robust and resilient.

Conclusion

Hierarchical architecture is a proven solution for building maintainable, scalable, and fault-tolerant systems. By organizing processes in a structured way, this approach avoids the complexities of spaghetti architecture and ensures that systems remain flexible, stable, and capable of future growth.

The Workers framework utilizes hierarchical architecture to help developers construct robust applications. By adopting this approach, you can create systems that are easier to manage and are prepared to adapt to future extensions.

Advantages of Plug-in Architecture

A plug-in architecture is a software design pattern that enhances modularity in software architectures by allowing modules to integrate or be plugged-in to existing applications without altering the core codebase.

Central to plug-in architecture is the principle of decoupled design, which minimizes dependencies between modules in a system. This section discusses the multiple ways in which plug-in architecture and decoupled design enhances system flexibility, maintainability, and testability.

Flexibility and Adaptability

The primary advantage of plug-in architecture is flexibility. Users can easily add or remove plug-ins to customize the software based on their specific needs without requiring changes be made to the core system. Workers for LabVIEW allows you to integrate new modules into an application's modular hierarchy through the use of scripted tools, as well as at runtime through the use of dynamic loading through pre-defined abstractable interfaces (i.e. abstractable Public APIs). By interacting with modules through predefined interfaces, each plug-in operates independently, reducing the likelihood that changes to one module will affect others.

Maintainability and Reusability

Plug-in architecture enhances maintainability by isolating functionality within individual plug-ins. Bug fixes, updates, or enhancements can be applied to specific modules without disrupting the rest of the system. This separation reduces the risk of introducing errors and streamlines the development process. Because plug-ins are designed with high cohesion—focusing on specific tasks—they can be reused across multiple projects, promoting consistency and reducing development time.

Enhanced Collaboration and Parallel Development

Collaboration is enhanced through decoupled design, as separate teams can work on different plug-ins independently. This parallel development process speeds up overall production, as specialized expertise can be applied without impacting the core system or other plug-ins. The decoupled nature of the modules allows teams to develop, test, and maintain their plug-ins in isolation, minimizing dependencies and potential conflicts.

Separation of Concerns

Plug-in architecture encourages a clear separation of concerns. The core system provides basic functionality and defines interfaces for plug-ins, while plug-ins manage specific, often specialized, tasks. This division allows the core to remain lightweight and stable, while plug-ins handle extended features. Interfaces and abstractions play a critical role here, facilitating communication between modules while keeping the underlying implementation details hidden. This abstraction enables developers to update or replace individual plug-ins without affecting the rest of the system, making the architecture highly adaptable.

Improved Testability and Reliability

Decoupled design enhances testability. Plug-ins can be tested independently before being integrated into the larger system, ensuring that each component functions as intended. This modular and decoupled approach to testing improves reliability and stability across the system, reducing the risk of critical failures. Isolating issues within specific plug-ins makes debugging and updates easier to manage, further contributing to system robustness.

Dynamic Loading of Plug-ins at Runtime

Plug-in architecture supports dynamic loading, allowing modules to be loaded, exchanged, or unloaded at runtime without needing to restart the entire system. On a single abstractable interface to the core system, developers can swap modules in and out without changing the application's core code, providing users with significant flexibility, allowing the system to adapt to changes quickly and on demand.

Enabling High Cohesion and Loose Coupling

A key principle in plug-in architecture is designing modules with high cohesion and loose coupling. High cohesion ensures that each plug-in focuses on a specific task, simplifying module design and making the system easier to understand, maintain, and expand. Loose coupling minimizes dependencies between different system components, reducing complexity and the risk of system-wide errors. This design philosophy encourages a transparent and organized system, avoiding the entanglements associated with spaghetti architecture.

Conclusion

Plug-in architecture, supported by decoupled design principles, offers flexibility, scalability, and maintainability, making it a powerful approach for building modern software systems. By isolating functionality, promoting reusable components, and supporting dynamic updates, this architecture ensures that systems remain adaptable and future-proof in the face of changing system requirements.

Practical Example: NI PXI Systems

National Instruments PXI (PCI eXtensions for Instrumentation) systems are a prime example of how modern engineering platforms effectively incorporate the foundational pillars of modularity, hierarchy architecture, and plug-in architecture with decoupled design. These principles are integral to creating flexible, adaptable, and high-performance systems suitable for a wide range of testing and measurement applications.

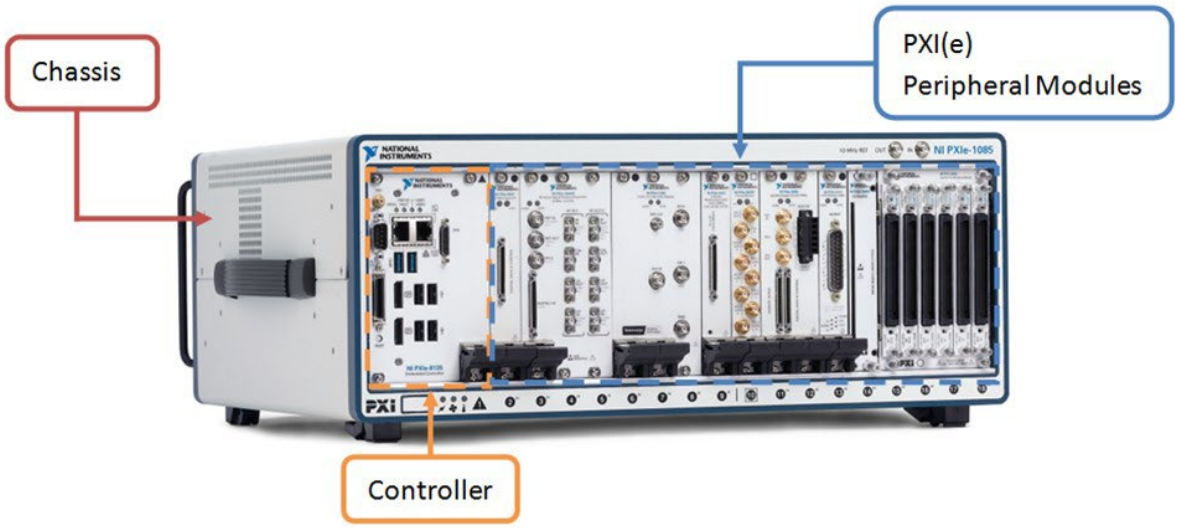

Figure 4 – NI PXI system (ni.com, 2024)

Modularity in PXI Systems

Modularity is a central feature of PXI systems, emphasizing the separation of a complex system into distinct, self-contained units or modules. Each module within a PXI chassis performs a specific function, such as data acquisition, signal generation, or communication. For example, one module might be a high-speed digitizer, while another is an arbitrary waveform generator. This modular approach allows users to configure their systems precisely according to their needs, facilitating customization and adaptation to various testing scenarios.

In a PXI system, modularity allows for easy interchangeability of components. Users can add, remove, or replace modules within the chassis without impacting the operation of the entire system. This capability simplifies maintenance and reduces downtime because individual modules can be serviced or upgraded independently. The modular design also enhances the system's reliability and reduces the complexity involved in troubleshooting and repairing faults.

Plug-in Architecture in PXI Systems

Plug-in architecture is another key pillar embodied by PXI systems. This design pattern enables the system to be extended with new features through separate modules that can be dynamically integrated. In PXI systems, each module acts as a plug-in that provides specific functionalities, such as various types of measurements or signal processing tasks.

The plug-in nature of PXI systems allows users to expand their capabilities without altering the core system. As testing requirements evolve, new modules can be added to introduce additional functionalities. For instance, if a project requires RF signal generation, an RF generator module can be plugged into the existing chassis. This flexibility makes PXI systems highly adaptable, allowing them to cater to diverse applications across multiple industries.

Scalability and Extensibility in PXI Systems

Scalability and extensibility are critical attributes that ensure a system can grow and adapt over time. PXI systems are designed with these principles in mind, allowing them to handle increased complexity and incorporate new functionalities efficiently.

The modular architecture of PXI systems supports scalability by enabling users to add additional modules to increase the system's capabilities. Whether the need is for more data channels or advanced measurement features, users can extend their systems incrementally as their requirements grow. This scalability ensures that the system can accommodate higher demands and maintain optimal performance even as the scope of applications expands.

Extensibility in PXI systems is evident through the ease with which they can incorporate new technologies or standards. As technological advancements occur, users can integrate new modules that leverage these innovations without needing to redesign the entire system. This ability to adapt to new requirements ensures that PXI systems remain valuable and effective in meeting future challenges.

Decoupled Design in PXI Systems

Decoupled design focuses on reducing dependencies between different parts of a system. In PXI systems, this design is implemented by ensuring that each module operates independently, communicating through standardized interfaces like the PXI bus. This decoupling allows for the independent operation of modules, enabling them to be replaced or upgraded without affecting the rest of the system.

The decoupled design of PXI systems simplifies troubleshooting and maintenance. When issues arise, they can often be isolated to specific modules, making it easier to diagnose and resolve problems without disrupting the entire system. Additionally, this design supports concurrent operation, allowing multiple modules to perform measurements simultaneously. This capability is crucial in complex, multi-channel testing environments where different measurements must be coordinated yet operate independently.

Conclusion

National Instruments PXI systems demonstrate the successful implementation of modularity, plug-in architecture, hierarchical scalability, and decoupled design. These principles work together to create a flexible, customizable, and high-performance platform capable of meeting a wide range of testing and measurement needs. By integrating these foundational pillars, PXI systems not only enhance functionality and adaptability but also serve as a model for designing modern engineering platforms that can meet a wide range of technological demands.

Framework Concepts

In this section, we will dive into the specifics, presenting the fundamental components of the framework that are essential for building scalable and modular applications.

What is a Worker?

Concept definition

What is a Worker, and why is the framework called Workers for LabVIEW? Consider a company where employees perform tasks independently, each assigned specific duties by their managers within a structured hierarchy. Each employee operates as a modular unit, focusing on tasks that utilize their unique skills, while managers oversee multiple employees and delegate responsibilities. This layered structure ensures efficiency and clear lines of responsibility, with the CEO at the top managing the overall organization.

Similarly, in the Workers framework, a **Worker** is a modular process that operates independently within a hierarchical system. Each Worker performs specific functions based on its unique abilities and reports to its manager or **Caller** upon completing its tasks. Just as in a company, Workers can also manage **sub-Workers**, creating a structured and organized modular hierarchy. This design allows multiple processes to work concurrently, asynchronously, and collaboratively, ensuring the overall application scales well and functions efficiently. The hierarchical structure ensures clear communication pathways between Workers, helping the system remain modular and adaptable to changing requirements.

Framework definition

In the framework, a **Worker** is a modular, asynchronous process that implements the LabVIEW Queued Message Handler (QMH) design pattern within a hierarchical architecture.

Each Worker exists as a LabVIEW class that inherits from the common base class *Worker.lvclass*, leveraging LabVIEW Object-Oriented Programming (LVOOP) principles to enhance flexibility and code reuse.

Workers operate independently to perform specific tasks and communicate with other Workers through well-defined asynchronous public APIs. This structured approach promotes modularity, scalability, and maintainability, enabling developers to build large, multi-process applications by assembling and reusing Workers within a clear and organized hierarchy.

> **Good to know**
>
> For developers without LVOOP experience, for now you can simply think of a LabVIEW class as a LabVIEW library that also has the benefit of being a LabVIEW datatype.

A Worker's Default Items

You can create new Workers and add them to your LabVIEW projects using the **Create/Add Worker** tool (see page 39). A new Worker created by the tool contains only the following items:

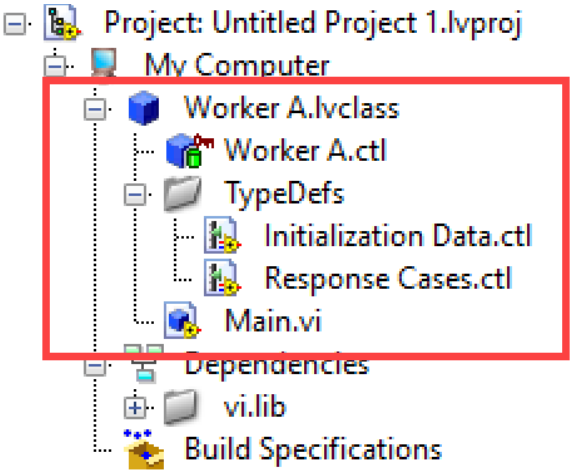

Figure 5 - Every new Worker is created with only these items

Worker A.lvclass

The Worker itself - a class that encapsulates the Worker's VIs and data files (i.e., *.vi* and *.ctl* files). Since every Worker is a LabVIEW class, it has the *.lvclass* file extension.

Worker A.ctl

A typedef that is private to the Worker and its VIs. You can add any data elements here that you wish to store between iterations of the Worker's Message Handling Loop (MHL).

Initialization Data.ctl

A typedef used to pass data into the Worker's <Initialize> MHL case during initialization by the Worker's Caller or Launcher VI.

Response Cases.ctl

A typedef that holds multiple MHL case names from the Worker's Caller. It is used to register the Worker's Public API Response messages with its Caller when the Worker is initialized.

Main.vi

Known as the Worker's Main VI, this VI contains the Worker's Queued Message Handler (QMH) on its block diagram. Unless you are developing on an NI real-time target (e.g. a CompactRIO), this VI is a reentrant shared clone.

A Worker's Main VI

A Worker's Queued Message Handler (QMH) is located on the block diagram of its Main VI (Main.vi). It is designed in the same style as the LabVIEW QMH template taught in the NI LabVIEW Core 3 course, which is one of the standard design-pattern templates included with LabVIEW. The block diagram of a Worker's Main VI may look like this:

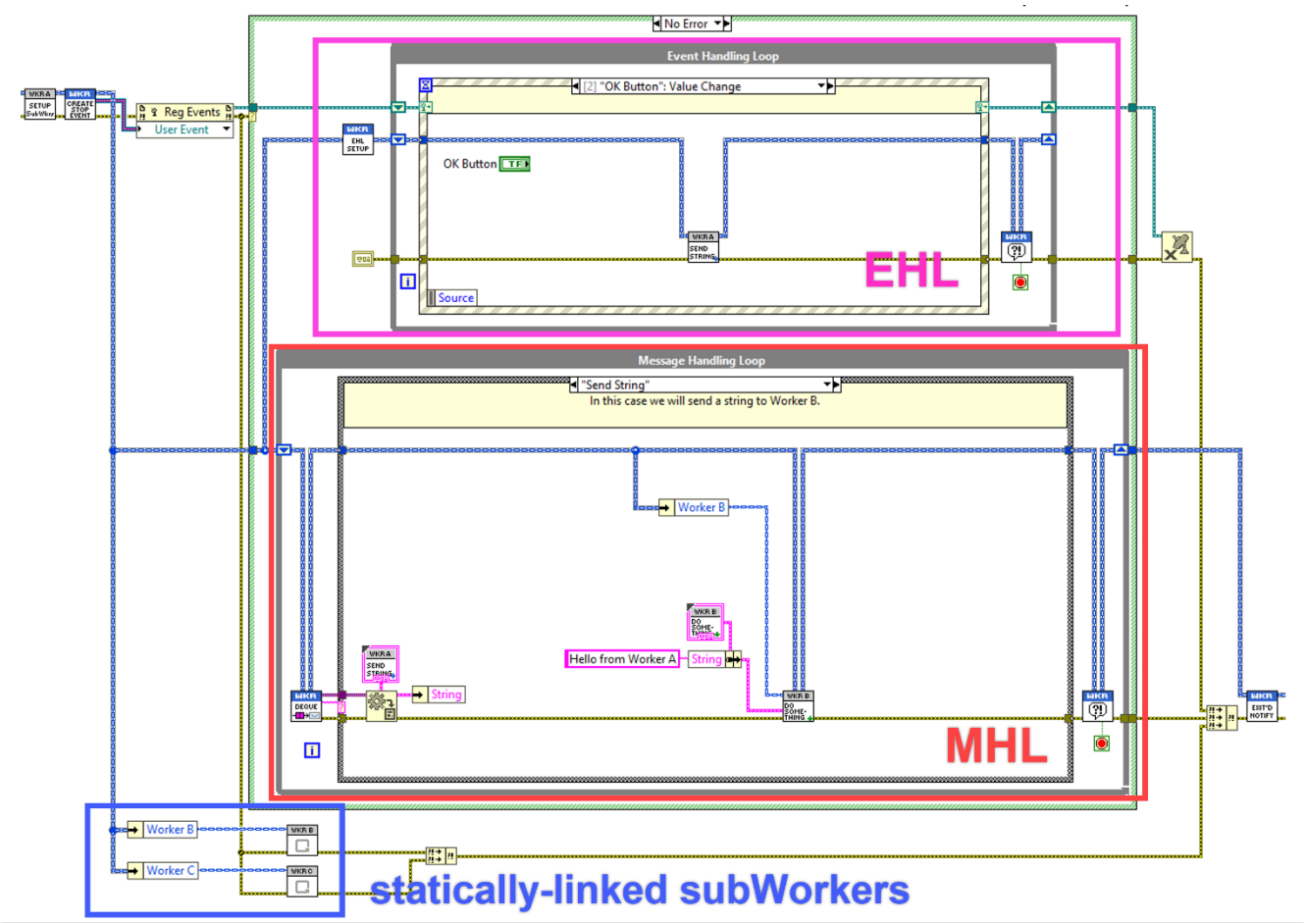

Figure 6 – The block diagram of a Worker's Main VI

Good to know

A Worker's *Main.vi* is a reentrant shared clone, allowing to you to run multiple instances of a Worker's Main VI in simultaneously.

Event Handling Loop (EHL)

The Event Handling Loop (EHL) at the top of a Worker's Main VI is optional. A Main VI can exist with or without an EHL and can even have multiple EHLs if necessary. The EHL is required only if you need to handle front panel events or user-generated events.

Message Handling Loop (MHL)

Every Worker's Main VI is required to have a Message Handling Loop (MHL). The Worker's MHL processes all incoming messages sent to the Worker, whether they originate from within the Worker itself by use of the Worker's Local API or are sent to the Worker from external sources through use of the Worker's Public API.

The main parts of a Worker's MHL include the following:

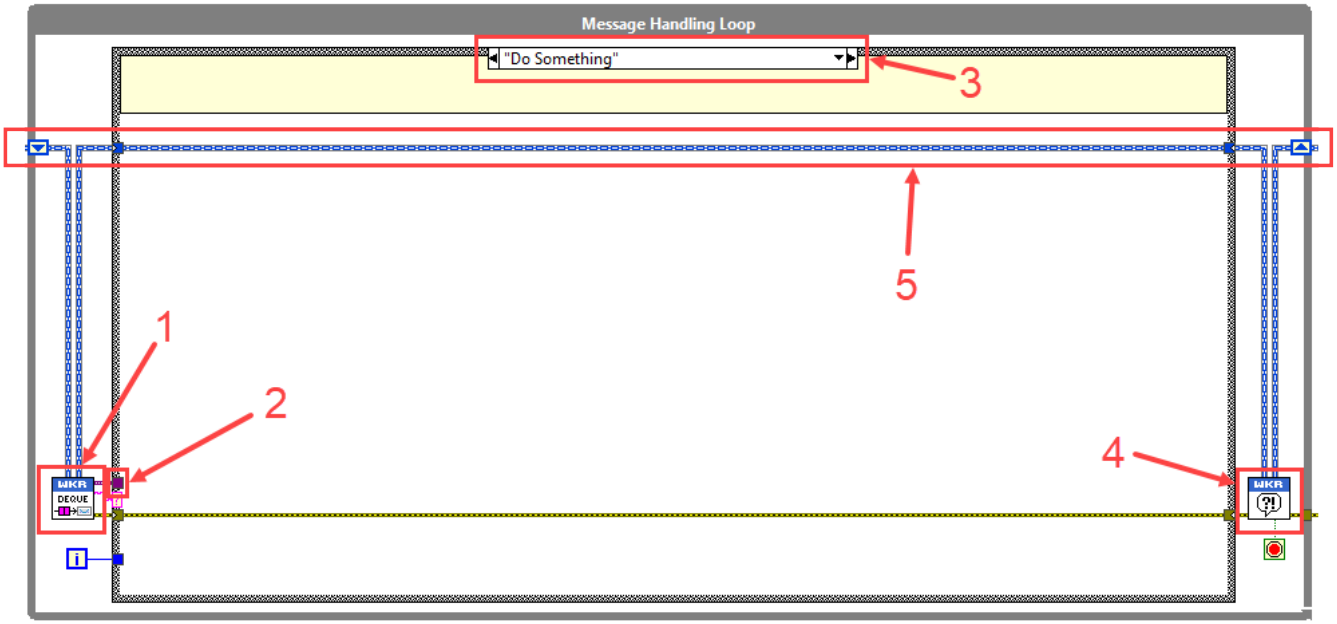

Figure 7 - Parts of a Worker's MHL

1. ***Dequeue Message.vi***

   A VI that removes and retrieves messages from the Worker's message queue.

2. **Data**

   An optional data payload (as a variant) sent along with a message, which can be accessed within the corresponding case of the case structure.

3. **Case Structure**

   A structure where each case corresponds to a specific message, selected based on a string identifier contained in the message dequeued by *Dequeue Message.vi*.

4. ***Error Handler.vi***

   A VI that handles any errors encountered within the MHL, forwarding them to the Workers Debug Server for logging and debugging.

5. ***Worker (X).ctl***

   Represents the Worker's private data cluster (the blue wire), maintained within a shift register in the MHL for state management between case iterations.

Worker Main Data Wire

The Worker's Main Data Wire is the Worker class wire that originates at the Worker's object constant and flows through the Worker's MHL. It holds the Worker's state and private data throughout the execution of the loop.

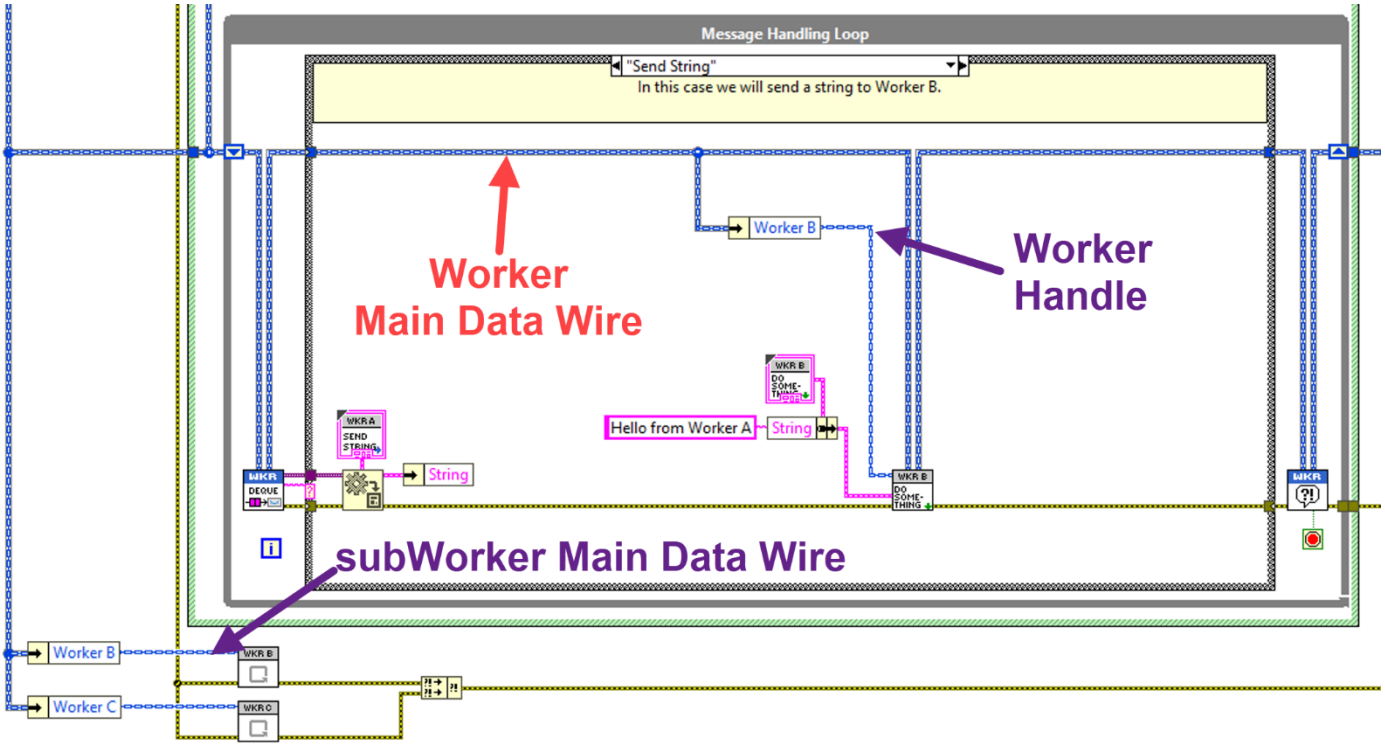

Figure 8 - Worker Main Data Wires and Worker Handles

Worker Handles

Worker Handles are class wires that contain a Worker's public properties, such as the Worker's message queue reference. They are used to send messages to a specific Worker instance via the Worker's Public Request VIs. Examples of Worker handles include:

1. **Caller's MHL statically-linked Worker Class Wires**

   Statically-linked subWorker class wires used by a Caller's MHL to communicate with its subWorkers.

2. **Asynchronous Launcher VI Output**

   The Worker class wire that exits an Asynchronous Launcher VI after launching a Worker.

3. **Dynamically Load Worker Public API VI Output**

   The Worker class wire that exits a Dynamically Load Worker Public API VI.

Worker Relationship Terms

A Workers application consists of one or more Workers connected in a call-chain hierarchy resembling a tree structure. Understanding these relationships is important for designing applications with the framework. For example, consider an application built with four Workers, as represented in Figure 9.

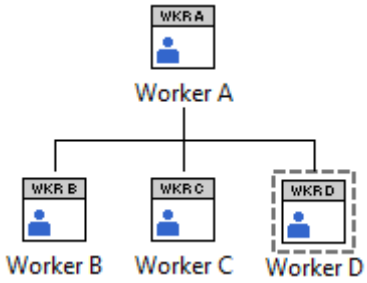

Figure 9 - an application's Worker call-chain consisting of four Workers

Head Worker

Every Workers application has a head Worker. The head Worker is the Worker that sits at the top of the application's Worker call-chain hierarchy. In Figure 10, **Worker A** is the application's head Worker.

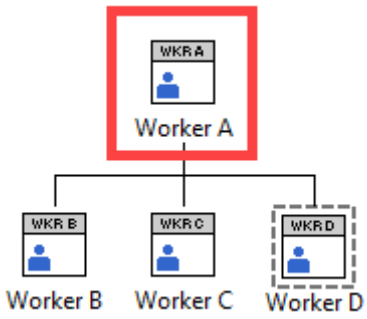

Figure 10 - Worker A is the head Worker of the application

Caller

A Caller is a Worker that loads and integrates one or more Workers into an application's Worker call-chain hierarchy. These Workers are known as the Caller's subWorkers. The Caller initiates and manages its subWorkers, creating their message queues and facilitating communication with them. In Figure 11, **Worker A** is the Caller of **Worker B**, **Worker C**, and **Worker D**.

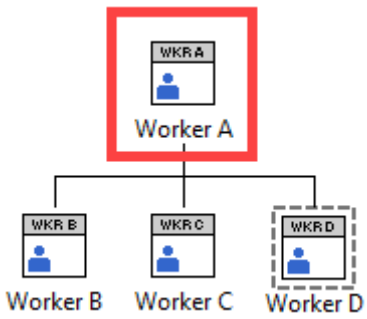

Figure 11 - Worker A is the Caller of Worker B, Worker C and Worker D

subWorker

A subWorker is a Worker that is loaded and integrated into an application's Worker call-chain hierarchy by another Worker, known as its Caller. SubWorkers perform specific tasks as part of the overall application and are managed by their Caller. In Figure 12, **Worker B**, **Worker C**, and **Worker D** are subWorkers of **Worker A**.

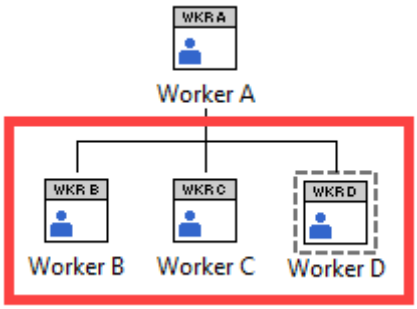

Figure 12 - Worker B, Worker C and Worker D are subWorkers of Worker A

SubWorker Types

Workers can be integrated into their Callers in two different ways: either statically-linked or dynamically loaded. Understanding these methods allows you to choose the best approach for your application's needs.

Statically-Linked subWorkers

Statically-linked subWorkers are incorporated into the application at compile-time. This means they are always loaded when the application starts, which can be advantageous for performance and simplicity when the required Workers are known in advance.

When you add a Worker to another Worker using the **Create/Add Worker** tool, you are adding it as a **statically-linked subWorker**. The Main VI of the statically-linked subWorker appears below the Message Handling Loop (MHL) of its Caller, as shown in Figure 13.

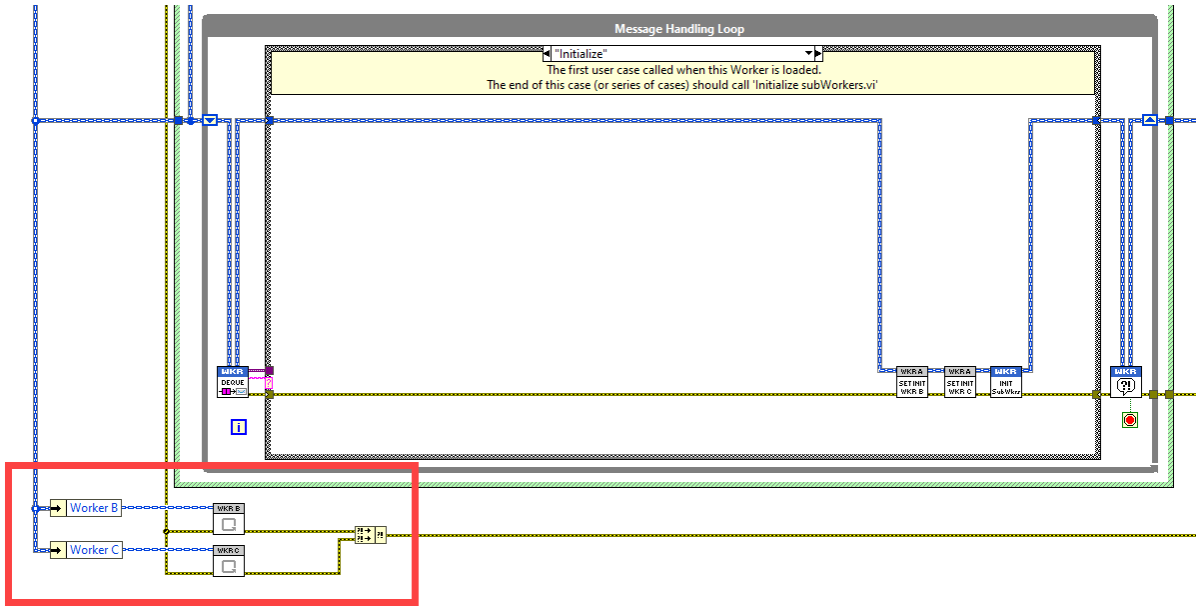

Figure 13 - Main VIs of statically-linked subWorkers located below a Caller's MHL

In an application's Worker call-chain hierarchy diagram, statically-linked subWorkers are represented as **Workers without an outer dashed line**. In Figure 14, the Workers inside the red box represent the statically-linked subWorkers from the block diagram in Figure 13.

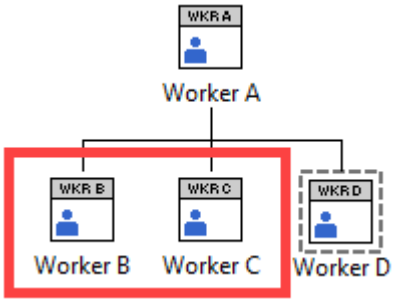

Figure 14 - Statically-linked subWorkers of Worker A

Dynamically Loaded subWorkers

Alternatively, Workers can be integrated into an application's Worker call-chain hierarchy at runtime by dynamically loading them using a Worker's **Dynamically Load Worker Public API VI**.

An example of how a Worker is dynamically loaded is shown in Figure 15, where **Worker D** is dynamically loaded from the MHL case of another Worker (i.e. **Worker A** in Figure 16). The returned Worker handle is stored in the Caller's Main Data Wire.

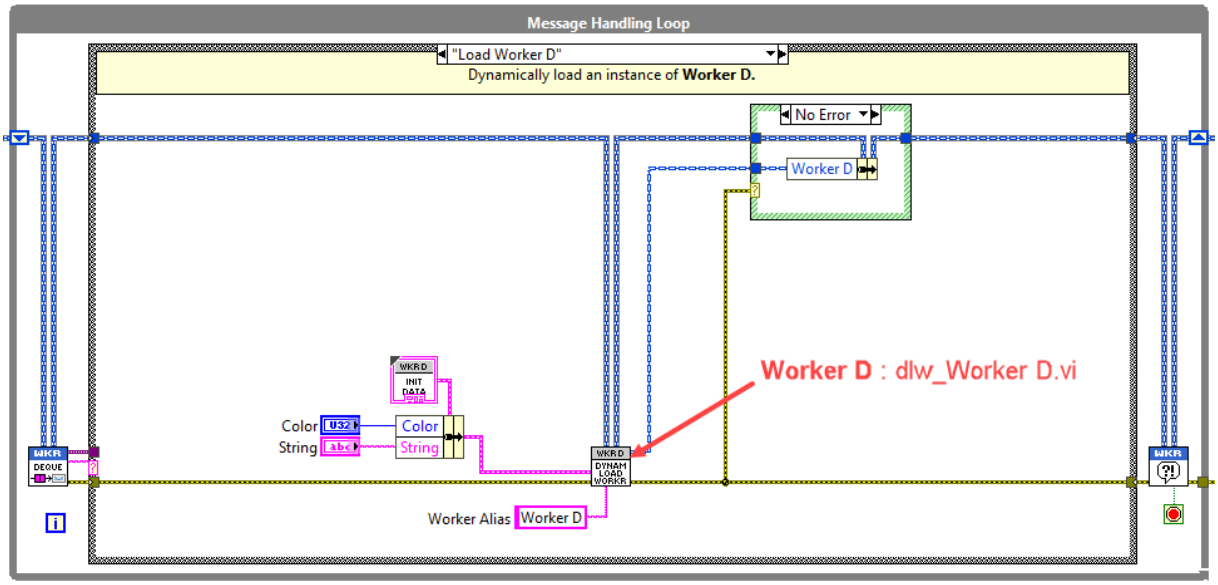

Figure 15 - Worker D is dynamically loaded by Worker A

In an application's Worker call-chain hierarchy diagram, dynamically loaded subWorkers are represented with an outer dashed line. In Figure 16, the Worker inside the red box represents the dynamically loaded subWorker from the block diagram in Figure 15 above.

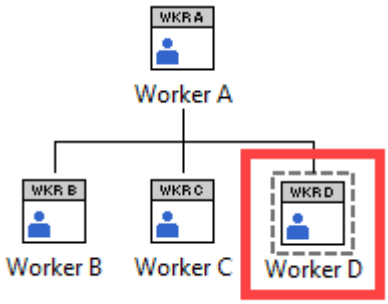

Figure 16 – A dynamically loaded subWorker of Worker A

Good to know

Dynamically loaded Workers are loaded with LabVIEW's *Start Asynchronous Call* node, using flags: 0x100 flag (Call and Collect) and 0x40 (Enable simultaneous calls on reentrant VIs).

Required MHL Cases

The Message Handling Loop (MHL) of a Worker's Main VI must include the four MHL cases shown in Figure 17 for the Worker to function properly. These cases can exist either within a Worker's MHL case structure, or they can exist in the *MHL Cases.vi* of a Worker base class that the Worker inherits from (see page 96). These cases are used for the initialization and shutdown sequences of a Worker and its subWorkers. Understanding these required MHL cases is essential for ensuring that Workers initialize and shut down properly within a Workers application.

Note: All VIs referenced in this section can be found in the Workers palette within LabVIEW.

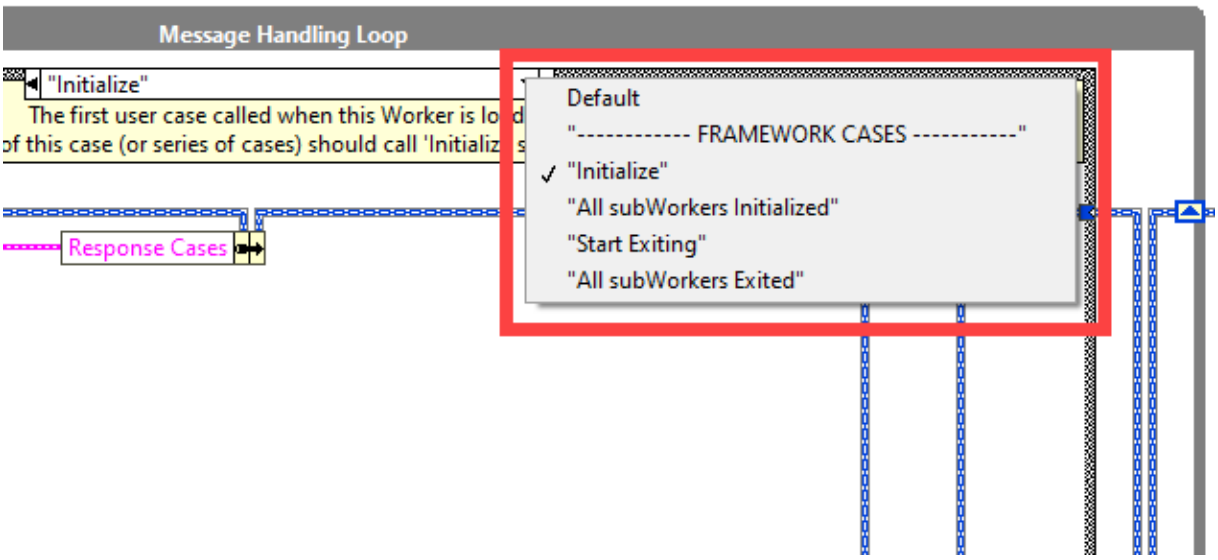

Figure 17 - The four required framework cases in a Worker's MHL

<Initialize> Case

The **<Initialize>** case is the first case in a Worker's MHL that is called by a Worker's Caller or by a Launcher VI. It receives user-defined initialization data required by the Worker, sent from the Worker's Caller or a Launcher VI via its *Initialization Data.ctl* typedef.

In this MHL case, you can perform any initial tasks required to be performed **before** the Worker's subWorkers are initialized. This may include setting up the Worker's initial conditions or configurations.

In this case you should also write data into the *Initialization Data.ctl* clusters of the Worker's statically-linked subWorkers (if it has any), in preparation for their own initialization.

The final step in this case (or series of cases) is to call *Initialize subWorkers.vi*, which will call the <Initialize> case of the Worker's statically-linked subWorkers.

Worker without statically linked subWorkers

If the Worker has no statically-linked subWorkers, calling *Initialize subWorkers.vi* will instead call the Worker's own <All subWorkers Initialized> case.

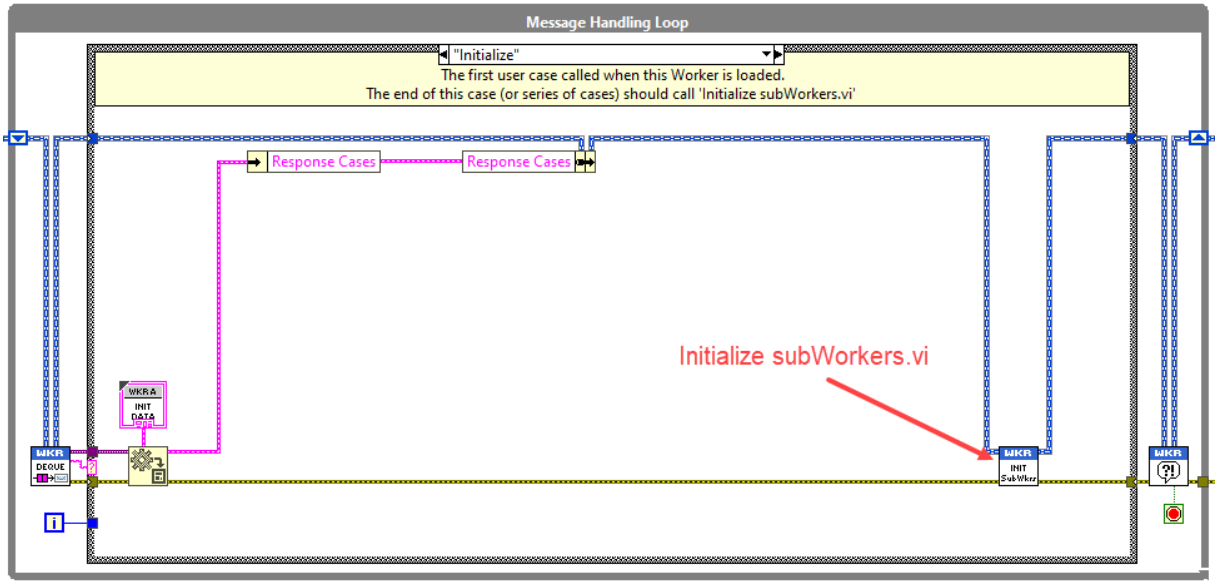

Figure 18 – A Worker's MHL <Initialize> case

<All subWorkers Initialized> Case

The **<All subWorkers Initialized>** case is called internally by the Worker when all of the Worker's statically-linked subWorkers have successfully been initialized. Each subWorker notifies their Caller by calling *Initialized Notify.vi*, indicating the completion of their own initialization.

In this case, you can perform tasks that should occur **after** all statically-linked subWorkers have been initialized.

The final step in this case (or series of cases) is to call *Initialized Notify.vi*, which notifies the Worker's Caller that the Worker itself has completed all its initialization tasks.

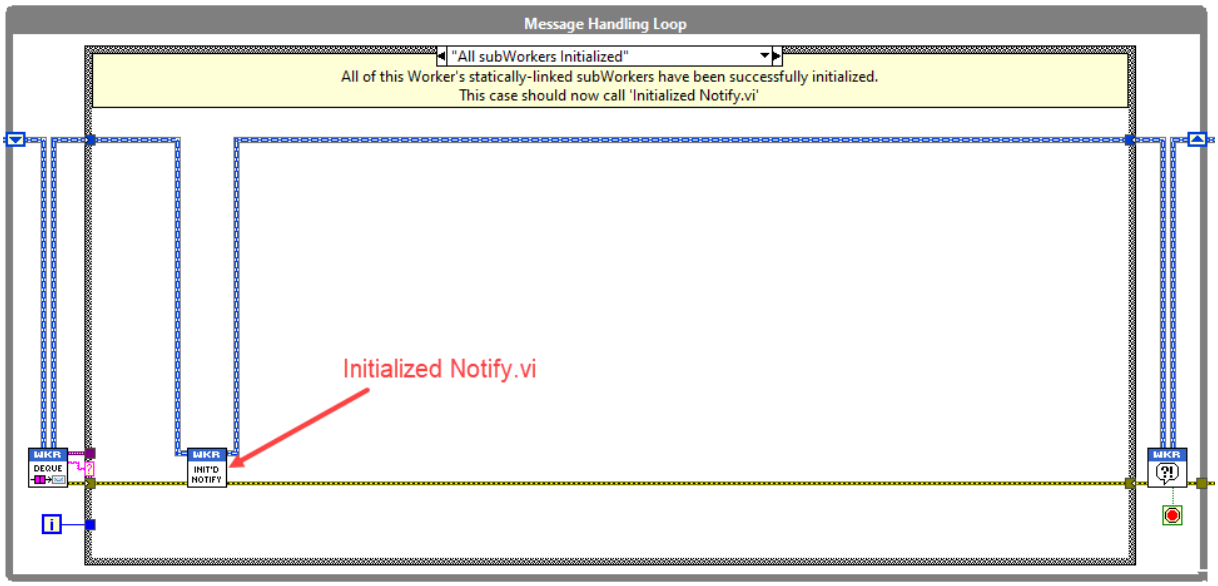

Figure 19 – A Worker's MHL <All subWorkers Initialized> case

<Start Exiting> Case

The **<Start Exiting>** case initiates the shutdown sequence of the Worker and its subWorkers. This case is called using *Start Exiting Worker.vi*, available from the Workers palette. It can also be called from the Workers Debug Server's **Application Manager** (see page 49).

In this case, you can perform any necessary tasks required to be performed **before** the Worker's subWorkers shut down.

The final step in this case (or series of cases) is to call *Start Exiting subWorkers.vi*, which triggers the <Start Exiting> case of the Worker's statically-linked and dynamically loaded subWorkers.

Worker without subWorkers:

If the Worker has no subWorkers, calling *Start Exiting subWorkers.vi* will instead call the Worker's own <All subWorkers Exited> case.

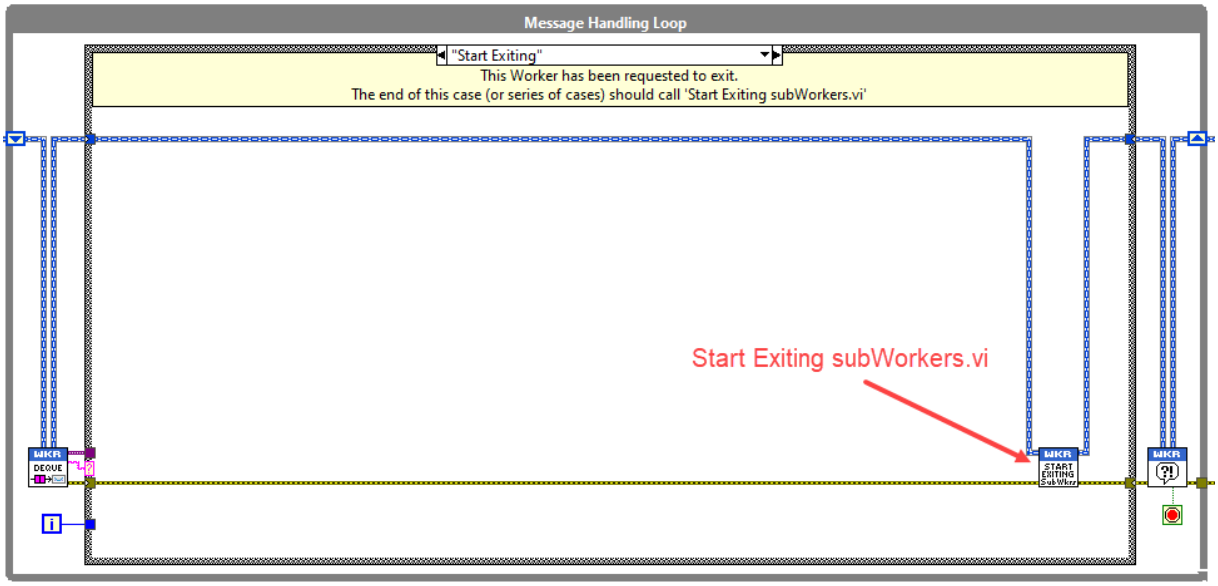

Figure 20 – A Worker's MHL <Start Exiting> case

<All subWorkers Exited> Case

The **<All subWorkers Exited>** case is called internally by the Worker when all of the Worker's subWorkers have successfully exited. Each subWorker notifies their Caller that it has exited by calling *Exited Notify and Cleanup.vi*.

In this case, you can perform tasks that should occur **after** all subWorkers have exited, such as final cleanup operations or releasing resources.

The final step in this case (or series of cases) is to call *Exit.vi*. This VI will stop both the Worker's MHL and EHL, allowing the Worker's Main VI to run to completion.

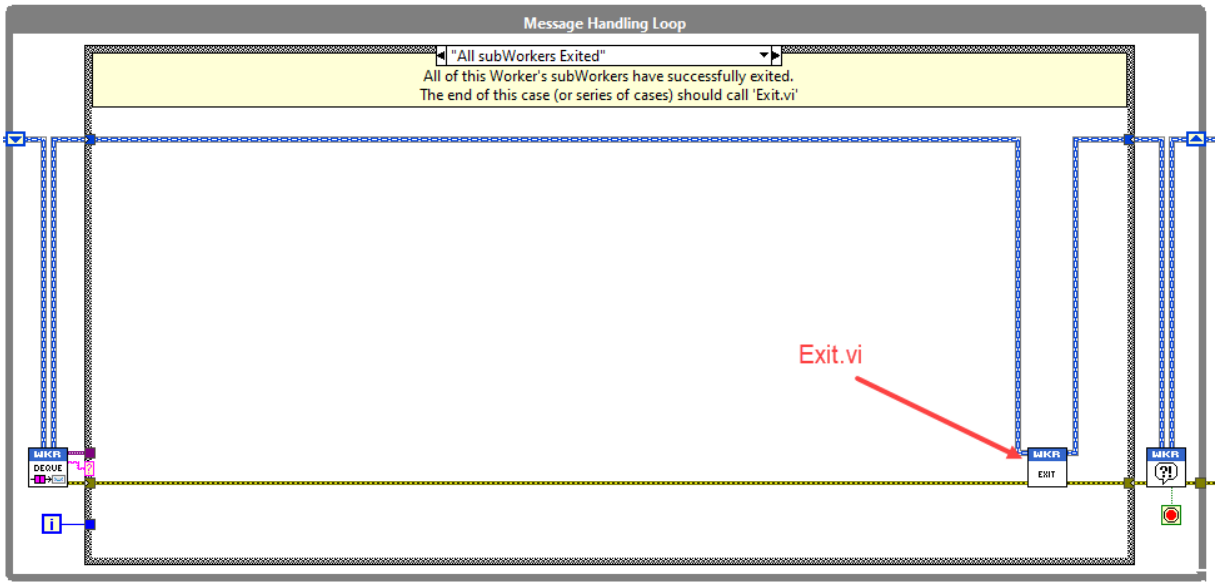

Figure 21 – A Worker's MHL <All subWorkers Exited> case

Application Initialization and Shutdown Sequences

The framework's initialization and shutdown sequences are performed through cascading calls down and up the application's Worker call-chain hierarchy. Understanding these sequences is important for effectively managing the application's lifecycle. Let's consider an application of four Workers arranged in a call-chain hierarchy, as shown in Figure 22.

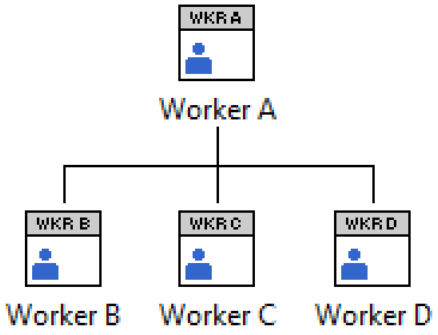

Figure 22 – An application's Worker call-chain hierarchy containing four Workers

Initialization Sequence

The initialization sequence of an application always starts with the head Worker's <Initialize> case being called first. In Figure 22, this means that **Worker A**'s <Initialize> case is the first to be called. The sequence of events from this point is as follows and is illustrated in Figure 23:

1. **Worker A's** <Initialize> case is called by a Launcher VI. Then, by calling *Initialize subWorkers.vi*...
2. All of **Worker A's** subWorkers' <Initialize> cases are called. Each subWorker calls *Initialized Notify.vi*, informing **Worker A** that it has been initialized. When all of **Worker A's** subWorkers have reported they are initialized...
3. **Worker A's** <All subWorkers Initialized> case is called internally by **Worker A**. Finally, **Worker A** *calls Initialized Notify.vi* to inform the framework that it has successfully initialized.

Note: If **Worker A** was not the head Worker, calling *Initialized Notify.vi* would alert its Caller that **Worker A** has been initialized. This process continues up the Worker call-chain hierarchy to the head Worker of the application.

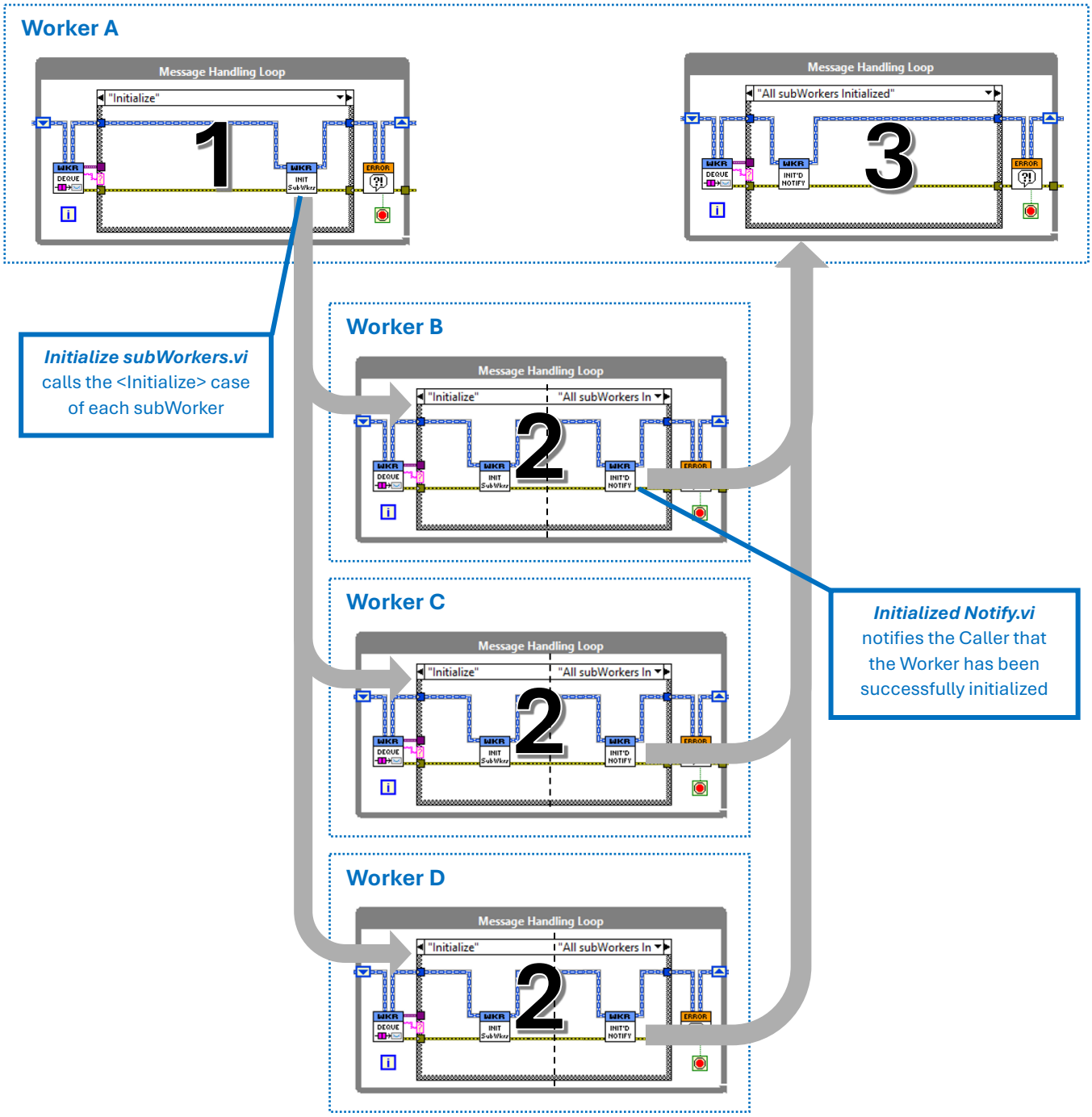

Figure 23 - Initialization sequence of the Workers application in Figure 22.

Shutdown Sequence

The shutdown sequence of an application always begins with the head Worker's <Start Exiting> case being called first. In our example application (in Figure 22), this means that **Worker A**'s <Start Exiting> case is the first to be called when the application needs to shut down. The sequence of events from this point is as follows and is illustrated in Figure 24:

0. The user calls *Start Exiting Worker.vi* (available from the Workers palette), or triggers the shut down of the Worker through the **Workers Debug Server** or through use of the Worker's *rqp_Start Exiting.vi Public API Request VI*.
1. **Worker A's** <Start Exiting> case is called by one of the methods indicated in step 0 above. Then, by calling *Start Exiting subWorkers.vi*...
2. All of **Worker A's** subWorkers' <Start Exiting> cases are called. Each subWorker finally calls *Exited Notify and Cleanup.vi*, informing **Worker A** that it has exited. When all of **Worker A's** subWorkers have reported they have exited...
3. **Worker A's** <All subWorkers Exited> case is called internally by **Worker A**. Finally, **Worker A** calls *Exited Notify and Cleanup.vi* to inform the framework that it has successfully exited.

Note: If **Worker A** was not the head Worker, calling *Exited Notify and Cleanup.vi* would alert its Caller that **Worker A** has exited. This process continues up the Worker call-chain hierarchy to the head Worker of the application.

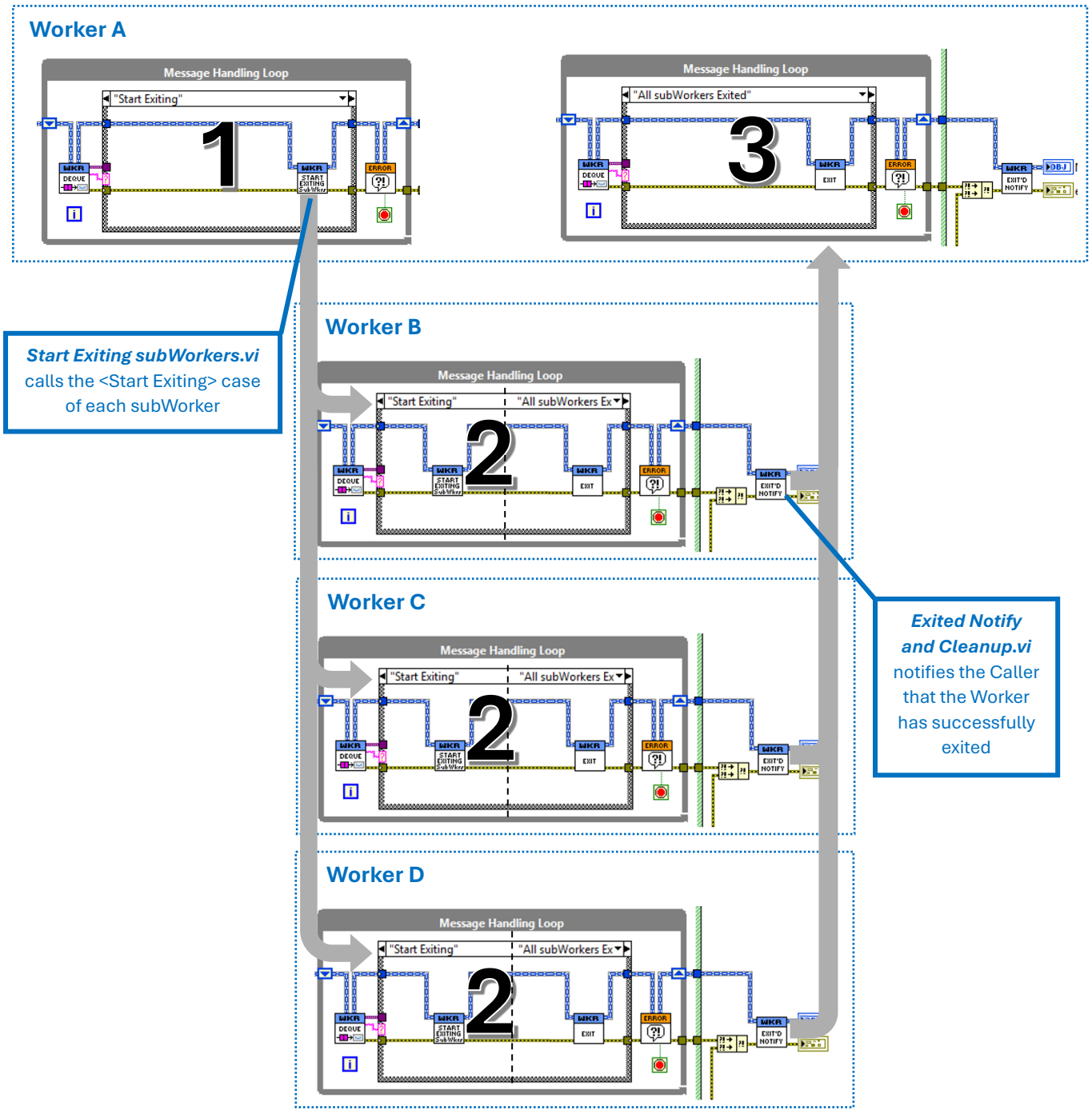

Figure 24 - Shut down sequence of Workers application in Figure 22.

A Worker's Message Queue

Messages are sent between Worker MHL cases via **priority message queues**. Unlike standard LabVIEW FIFO queues and events, the Workers message queue allows you to assign priorities to messages. This gives you more control over the processing order, ensuring that critical and important messages are processed before less important ones. As a result, your applications become more deterministic and responsive. The Workers framework uses the same priority queue implementation as the LabVIEW Actor Framework.

Each Worker is assigned a message queue by its Caller during the Worker's instantiation. For head Workers, the message queue is assigned by the *Setup Head Worker.vi* in the Worker's Launcher VI. Messages are dequeued by *Dequeue Message.vi* in a Worker's MHL (Figure 25).

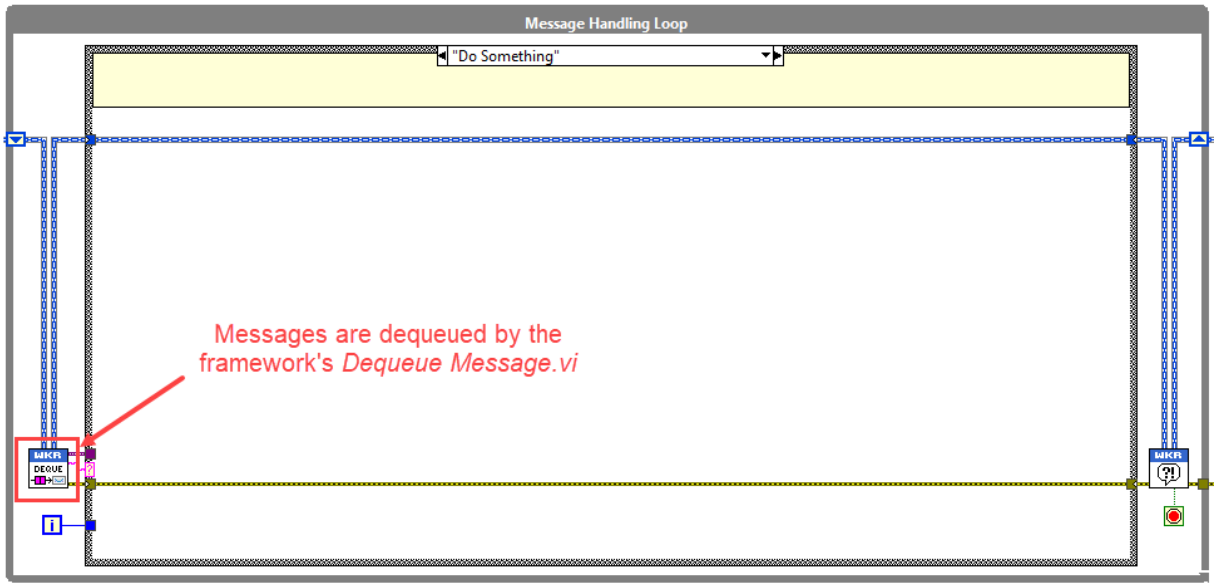

Figure 25 - Messages in a Worker's priority message queue are dequeued by the framework's ***Dequeue Message.vi***

The message priorities and their availability are as follows:

Priority	Availability
Critical	Internal (framework reserved)
High	Developer (Worker API)
Normal	Developer (Worker API)
Low	Developer (Worker API)

The framework uses **Critical** priority messages internally for the application's initialization and shutdown sequences. This ensures that these important messages are propagated immediately through an application's Worker call-chain hierarchy.

Using the Worker API VIs (see page 55) developers can assign their messages a priority of **High**, **Normal** (default), or **Low**, depending on the importance and timing requirements of the messages.

Good to know

For any given priority, the priority queue dequeues messages in the same order they were enqueued (first-in, first-out). This behavior avoids the reverse ordering of messages that can occur when enqueuing elements at the opposite end of a standard FIFO queue. As a result, your messages maintain their intended sequence within each priority level.

Launcher VIs

A **Launcher VI** is a VI that you run to launch the head Worker of a Workers application. Since a Worker cannot launch or run itself by default, a Launcher VI is necessary to initiate the application. You can create Launcher VIs using the **Create Launcher VI** tool (see page 82).

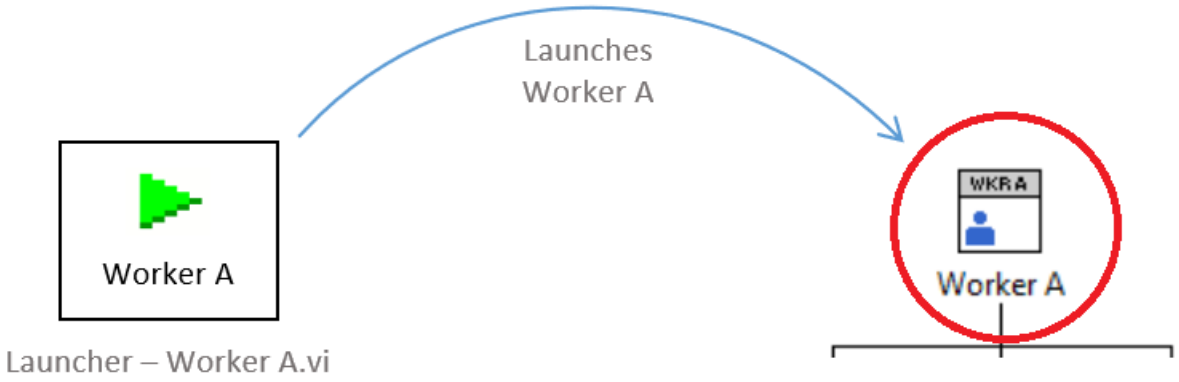

Figure 26 – A Launcher VI launches the head Worker of an application

The following images show a Launcher VI for a Worker in a LabVIEW project (Figure 27) and the block diagram of the Launcher VI (Figure 28).

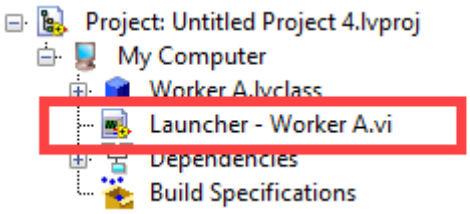

*Figure 27 – **Launcher – Worker A.vi** is the launcher VI for Worker A*

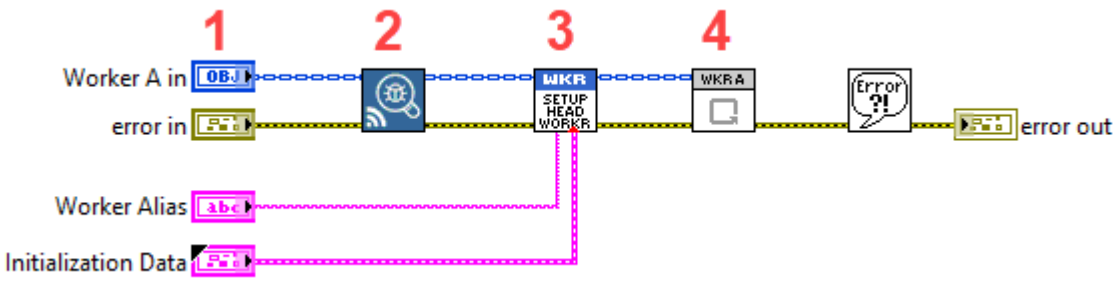

*Figure 28 – The block diagram of **Launcher - Worker A.vi***

Components of a Launcher VI

A Launcher VI typically contains the following elements (Figure 28):

1. **Worker Object Constant of the Application's head Worker**

   Represents the Worker you want to launch.

2. ***Debug Client Loader.vi***

   This VI loads an instance of a Workers Debug Client, which is a background task that connects the Workers Debug Server to a Workers application via TCP data streams. For more information, see the detailed help for this VI in LabVIEW.

3. ***Setup Head Worker.vi***

   A VI that sets up the head Worker by assigning it a message queue and performing the necessary framework initializations for the application.

4. **Main VI (*Main.vi*) of the Application's head Worker**

   The Main VI contains the head Worker's Queued Message Handler (QMH).

Behavior of the Launcher VI

When you run the Launcher VI, it launches the application's head Worker along with all of its statically-linked subWorkers. The Launcher VI remains running until the head Worker's Main VI exits.

If the Launcher VI is aborted (e.g., stopped manually), all Workers in the application's call-chain hierarchy—including both statically-linked and dynamically loaded Workers—will also be immediately aborted.

Reentrancy and Multiple Instances

The Launcher VI is configured as a **reentrant VI**, allowing you to run multiple instances of it simultaneously. Each instance operates independently, launching its own head Worker and associated subWorkers.

Despite being reentrant, the Launcher VI is **synchronous** in nature; it waits for the head Worker's Main VI to exit before it completes execution. This means that while you can run multiple instances of a Launcher VI in parallel, each Launcher VI remains active until its corresponding head Worker stops running.

Asynchronous Launcher VIs

An **Asynchronous Launcher VI** is used to launch a Worker's Launcher VI asynchronously. Unlike a standard Launcher VI, an Asynchronous Launcher VI does not wait for the head Worker's Main VI to exit before completing execution, providing greater flexibility and integration of Workers applications with external applications.

After launching the head Worker, an Asynchronous Launcher VI completes execution immediately, allowing the calling code to continue running without waiting, and returns the head Worker's handle (see page 21), which can be used to interact with the running Worker using its Public API Request VIs.

External applications, such as other LabVIEW VIs, different frameworks, or software like NI TestStand, can use Asynchronous Laucher VIs to launch and interact with head Workers without being tied to their execution lifecycle. For example, by using an Asynchronous Launcher VI, TestStand can launch a Workers application, receive the head Worker's handle, and interact with it using the head Worker’s Public API VIs, all while continuing with other test steps.

In **Exercise 2.4** later in the course, you will learn how to create and use an Asynchronous Launcher VI and interact with a Worker's Public API VIs from an external LabVIEW VI.

Good to know

Asynchronous Launcher VIs dynamically load Launcher VIs using LabVIEW's *Start Asynchronous Call* node, using flags: 0x80 flag (Call and Forget) and 0x40 (Enable simultaneous calls on reentrant VIs).

The Workers Development Tools

Welcome to the Workers Development Tools ! These tools have been created to enhance your efficiency and productivity while developing with the framework. By automating routine tasks, these tools not only save time but also promote adherence to best practices and design principles, helping you build applications that are maintainable, scalable and standardized.

The **Workers tools menu** (Figure 29 below) provides a single location to access the Workers development tools, online help documentation, and the Workers example projects. The Workers tools menu can be accessed in LabVIEW under *Tools >> Workers tools menu*.

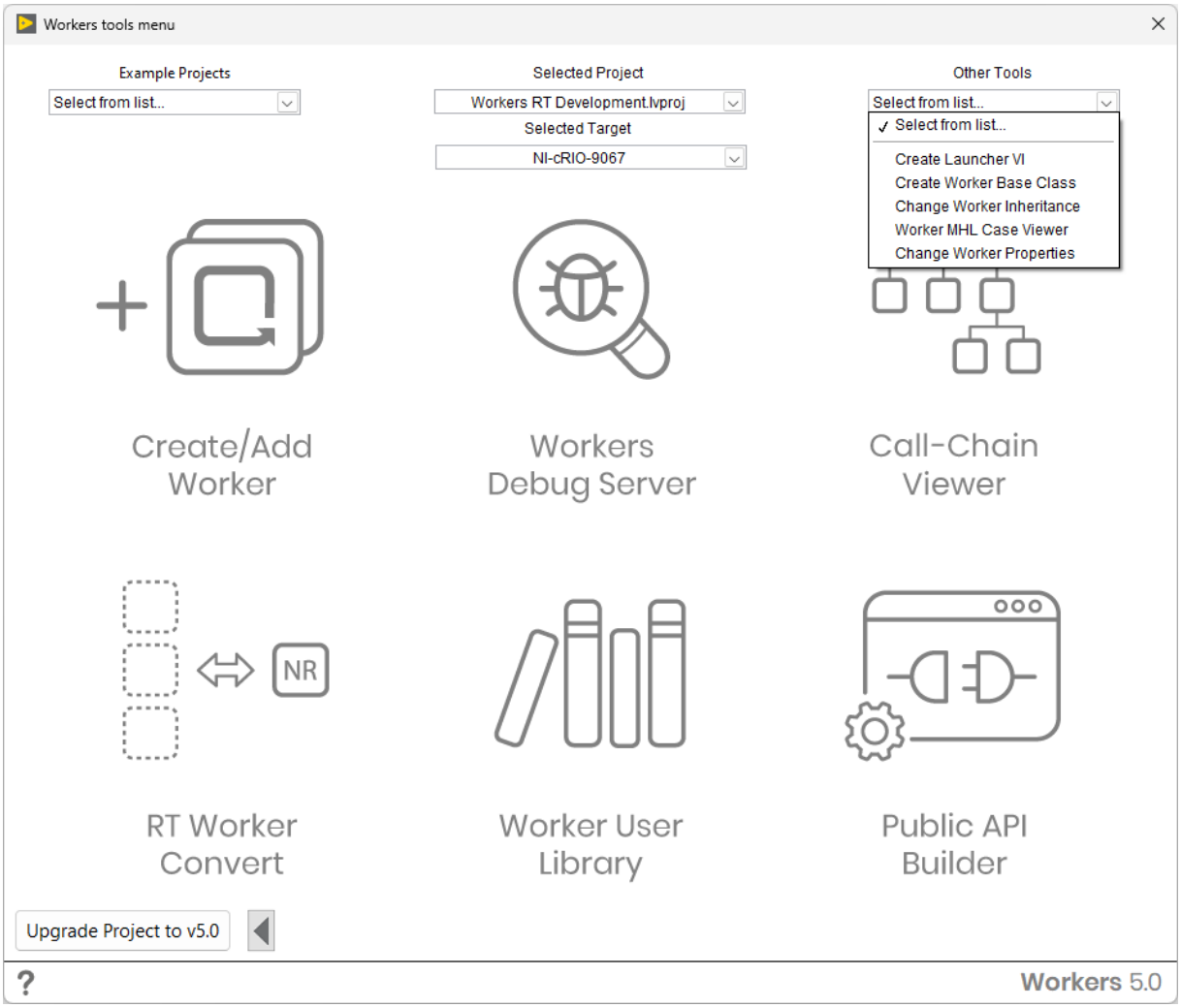

Figure 29 – Screenshot of the Workers 5.0 tools menu

All Workers tools will show the project name and target name that a tool is running on in the tool's window title. The format is: **Tool Name : Project Name : Target Name.** For example:

*Figure 30 - A Workers tools window title: **Tool Name : Project Name : Target Name***

Create/Add Worker tool

The **Create/Add Worker** tool is used to build the statically-linked Worker call-chain hierarchy of a Workers application. It allows you to create new Workers and integrate them into your application without any manual wiring required.

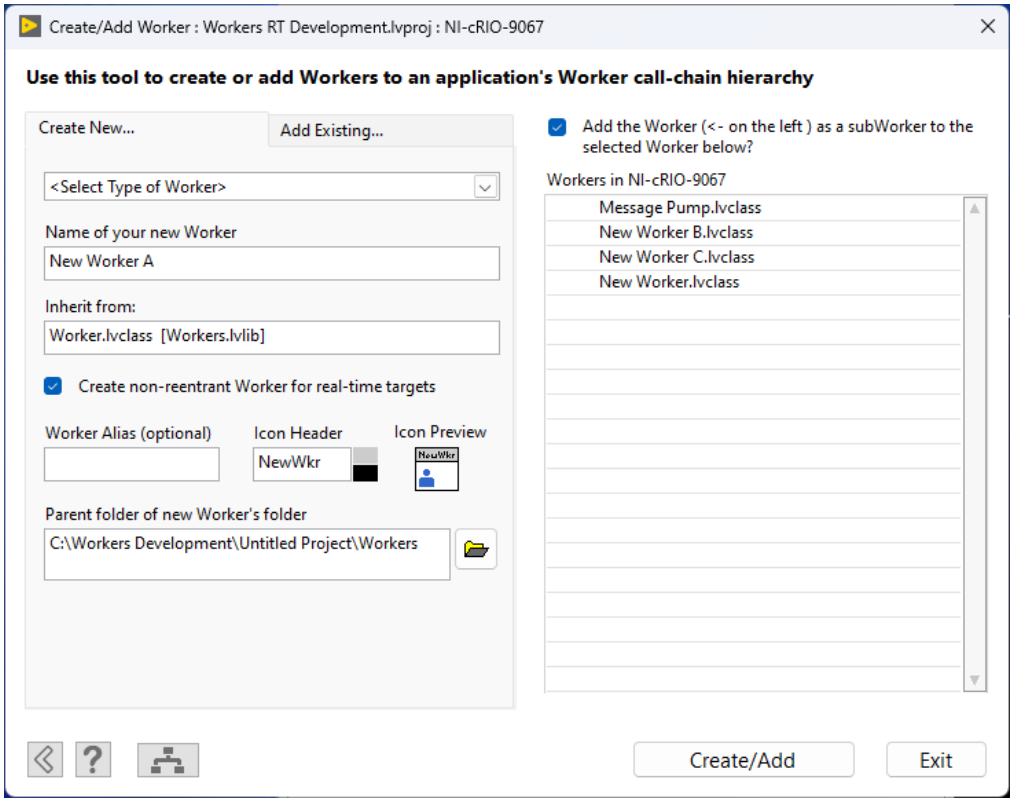

Figure 31 – Screenshot of the Create/Add Worker tool

Options

Select Type of Worker

- **Worker with EHL**: The Worker's Main VI will contain an Event Handling Loop (EHL). Choose this option if your Worker needs to handle LabVIEW events, such as front panel control events.
- **Worker without EHL**: The Worker's Main VI will **not** include an EHL. Select this option if your Worker does not need to handle LabVIEW events.
- **Worker from Template**: Create a new Worker based on your custom Worker template.

Good to know

If you select ***Worker without EHL*** and later realize that you need to handle LabVIEW events, you can simply add an EHL yourself, copied from another Worker with an EHL.

Name of your New Worker

Enter a name for your new Worker, which will also be the name of the Worker's class. Ensure the name is suitable for use as a filename.

Inherit from

Choose the class from which your new Worker will inherit. By default, Workers inherit from *Worker.lvclass [Workers.lvlib]*. Inheritance enables your Worker to utilize and extend the functionality of the parent class.

Create non-reentrant Worker for real-time targets

This option **only** appears if a real-time target was selected in the Workers tools menu. By default, this flag is selected, meaning that a Worker with a *Main NR.vi* will be created (the non-reentrant version of a Worker's Main VI).

Worker Alias

The Worker's Alias is the name the Worker will be known as within a Workers application. If left blank, the Worker's Alias will be the same as the **Name of your New Worker**. If your Worker's name contains many characters, it is recommended you choose a shorter alias for your Worker.

Icon Header

A string for the header of your Worker's icon. Here you can also select the background and text color of the icon's header.

Icon Preview

Displays what the icon of your Worker will look like, based on the header and text/background colors chosen.

Good to know

Right clicking over the icon preview and selecting **Copy header colors from class** allows you to extract the text/background icon colors from another Worker or Worker base class.

Parent folder of new Worker's folder

Specify the directory where the new Worker's folder will be created. The tool will create a folder named after your Worker to contain its class file and related resources. The parent folder is the directory one level above the new Worker's folder.

Add the Worker as a subWorker to the Worker selected below

Check this option to add the new Worker as a statically-linked subWorker to an existing Worker. Below this option, select the Worker from the **Workers in *target*** list to which you want to add the new subWorker (i.e. the Caller of your new Worker).

Open Worker Call-Chain Viewer 

Click this option to open the **Worker Call-Chain Viewer** tool. This tool visualizes the call-chain hierarchy of your Workers application, helping you understand and manage the relationships between Workers and subWorkers.

Manually Removing Workers

The **Create/Add Worker** tool cannot yet remove Workers that it adds to your applications. (The scripted removal of statically-linked Workers is planned in the next version of the framework). However, for now, it is simple enough to remove a statically-linked Worker yourself by deleting the items that the **Create/Add Worker** tool created by following the steps below.

Let's take an example where we want to remove **Worker B** from **Worker A**.

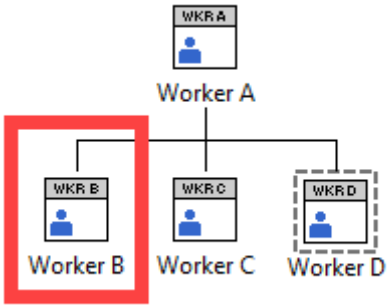

Figure 32 - Removing Worker B from Worker A

Step 1: Remove Worker object constant from *Statically-Linked subWorkers.ctl*.

Remove **Worker B**'s object constant from **Worker A**'s *Statically-Linked subWorkers.ctl*.

Important

If no Worker object constant exists in a Caller's *Statically-Linked subWorkers.ctl*, you will need to also delete *Statically-Linked subWorkers.ctl* from the Caller, because LabVIEW does not allow empty clusters to exist.

Step 2: Delete *Set Initialization Data - XXX.vi*

Delete *Set Initialization Data - Worker B.vi* from **Worker A** in the LabVIEW project explorer, and from any block diagrams where it has been placed within any VIs owned by **Worker A**.

Step 3: Delete *Setup subWorker - XXX.vi*

Delete *Setup subWorker - Worker B.vi* from **Worker A** in the LabVIEW project explorer, and from any block diagrams where it has been placed within any VIs owned by **Worker A**.

Step 4: Delete Worker's Main VI from Caller's Main VI

Delete **Worker B**'s Main VI (*Main.vi* or *Main NR.vi*) from the block diagram of **Worker A**'s Main VI.

Step 5: Final cleanup

You have now removed the statically-linked subWorker but you might still need to clean-up any broken wires now that you have removed and deleted the VIs in the previous steps. The Worker should now be removed from its Caller.

Creating Worker Templates

Creating custom Worker templates allows you to streamline your development process by providing your own predefined structure and functionality to new Workers. By using templates, you can ensure consistency, adhere to your own coding standards, and save time when developing new Workers within your application.

A **Worker template** is a Worker class whose filename ends with *template.lvclass*. By adhering to this naming convention, the **Create/Add Worker** tool recognizes the class as a template when generating new Workers.

For example, if your Worker is named *MyCustomWorker.lvclass*, **manually** rename the Worker class filename in the LabVIEW project to *MyCustomWorker template.lvclass*.

Creating a New Worker from a Template

When you create a new Worker from a Worker template using the **Create/Add Worker** tool, it performs the following actions:

1. **Copies the Worker Template**

   The tool creates a new Worker by making a copy of your Worker template, ensuring that all the customized features and structures are included.

2. **Applies Icon Header Customizations**

   The tool takes the icon header colors and settings from the Worker template and applies them to the new Worker's icon. This maintains visual consistency across Workers derived from the same Worker template.

3. **Updates Control and Indicator Labels**

   The tool replaces all the template VIs' Worker object control and indicator labels with the name of the new Worker.

Troubleshooting

If the **Create/Add Worker** tool fails to add a Worker as a subWorker to another Worker, several common issues might be responsible. This section outlines potential causes and provides guidance on how to resolve them.

Placement of Default Framework VIs in SubVIs

Issue: Placing the default framework VIs from a Worker's Main VI (i.e. the framework API VIs that exist in every new Worker's Main VI) into subVIs can cause the tool to fail, as it may not be able to locate the necessary VIs during the addition and wiring process.

Solution: Keep the default framework VIs visible in the Worker's Main VI. Avoid moving them into subVIs to ensure the tool can find them when adding new Workers with the tool.

Strange Wiring of Newly Placed VIs on Caller's Main VI

Issue: The tool may not correctly place or wire new VIs if it is unsure where to position them, especially if the Caller's Main VI is complex or crowded.

Solution: After using the tool, manually adjust the placement and wiring of the new VIs as needed. Organize the Caller's Main VI to provide clear spaces where new items can be added, reducing the likelihood of placement issues.

Caller Class is Locked

Issue: If the Caller class is locked (due to source code control, password protection, or being open in another project), new items cannot be added to it.

Solution: Ensure that the Caller class is unlocked before adding a new subWorker. If using source code control, check out or unlock the necessary files. If the class is password-protected, enter the password to unlock it.

Caller's Main VI is Broken

Issue: A broken Caller's Main VI can prevent the tool from adding new subWorkers.

Solution: Before adding a new subWorker, ensure that the Caller's Main VI is functional. Check for and fix any broken wires or other errors in the VI.

Worker or Files Already Exist on Disk

Issue: If a Worker with the same name already exists on disk (even if it's not included in your project), the tool may fail due to naming conflicts or file overwriting concerns.

Solution: Delete the existing Worker files from disk before attempting to create a new Worker with the same name. Alternatively, choose a different name for the new Worker to avoid conflicts.

VIs Are Not Wired Correctly

Issue: The tool may fail if it cannot find the required elements on the Caller's Main VI necessary for wiring up the new subWorker's Main VI. For instance, the Merge Errors node is required to exist for correct automatic wiring.

Solution: Ensure that required nodes, such as Merge Errors, are present on the Caller's Main VI before adding a new subWorker. If they are missing, add them to facilitate proper wiring.

A Worker's *Statically-Linked subWorkers.ctl* Is Empty

Issue: If any Workers in your project have a *Statically-Linked subWorkers.ctl* control that contains an empty cluster, the **Create/Add Worker** tool may not load or function correctly.

Solution: Open the affected Worker's *Statically-Linked subWorkers.ctl* control. Either remove the empty cluster or add at least one element to it, apply the changes, and save the control. Once it is no longer broken, try running the tool again.

Workers Debug Server

The **Workers Debug Server** provides your Workers applications with runtime transparency and plays an important role in developing applications efficiently with the Workers framework. It allows developers to easily access, view, and navigate their Worker instances while they are running. By offering real-time insights into your Workers applications, the Workers Debug Server helps you monitor and debug applications running in the LabVIEW development environment, as standalone executables, or on NI real-time targets (e.g. a CompactRIO) over a local network.

	Workers application running in			
Debug Server features	LabVIEW	EXE	PPL	RT (eg. cRIO)
Application Manager and Message Logs	✓	✓	✓	✓
Jump into Running VIs, Show MHL Case	✓	✗	✓*	✗

*Figure 33 - Available Debug Server features with Workers applications (*Enable Debugging Flag = True)*

Basic Principles

The Debug Server communicates with Workers applications via TCP data streams over a local network. It acts as a TCP server, while each Workers application functions as a TCP client through use of a **Debug Client**—a lightweight background task loaded by *Debug Client Loader.vi* within a Worker's Launcher VI. The Debug Server can accept multiple connections from Debug Clients over a user-defined TCP port.

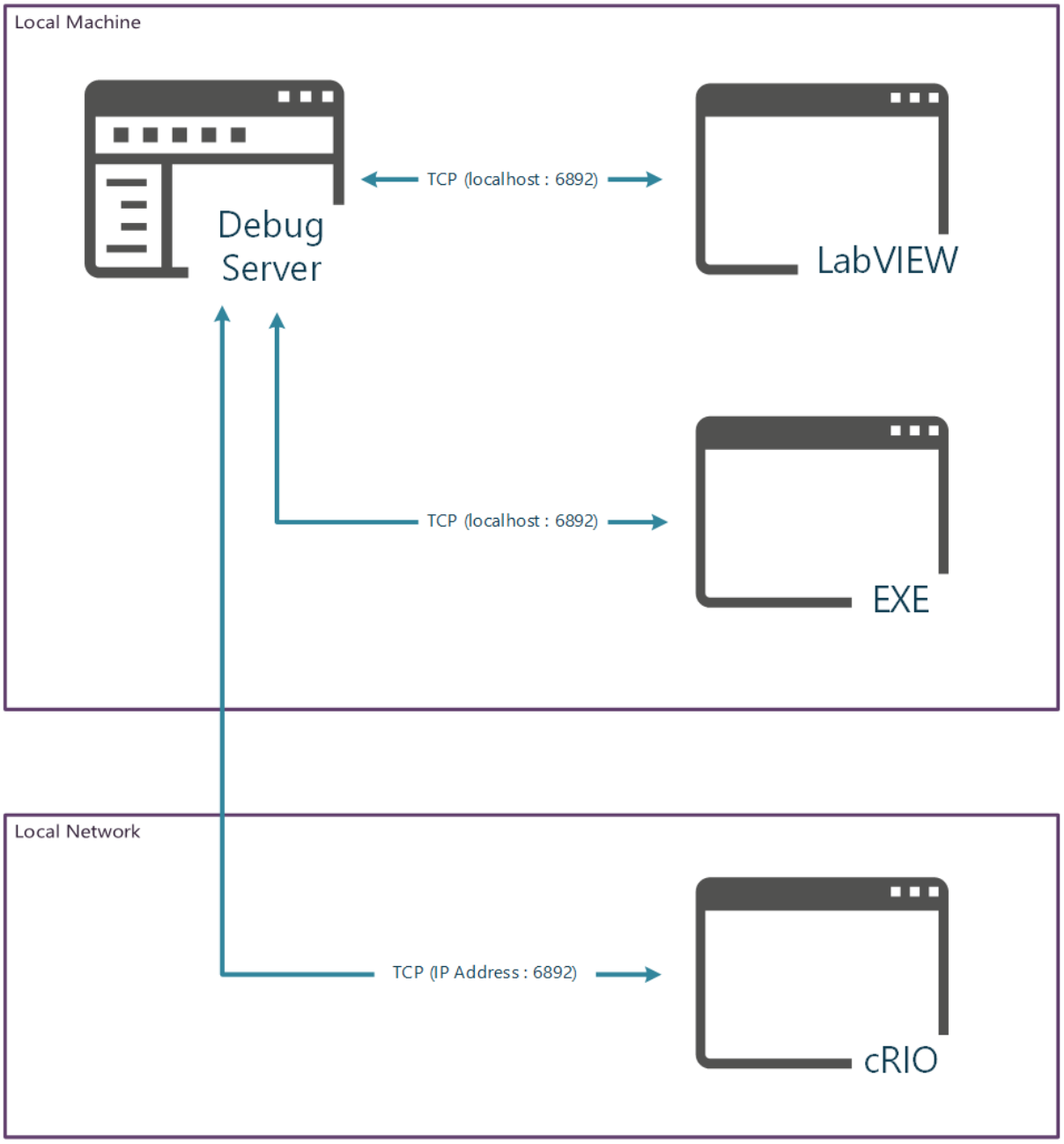

Figure 34 - Debug Server connected to multiple Workers applications running on different targets

IP Addresses and Ports

When you start the Debug Server application, it begins listening for incoming Debug Client connections on all available IP addresses on the host platform (e.g., localhost and network interfaces). By default, the Workers 5.0 Debug Server listens on port **6892**.

Within the Debug Server's **Settings** dialog (see Figure 35) you can:

- View the IP addresses the Debug Server is listening on.
- Change the listening port to a different value if needed.

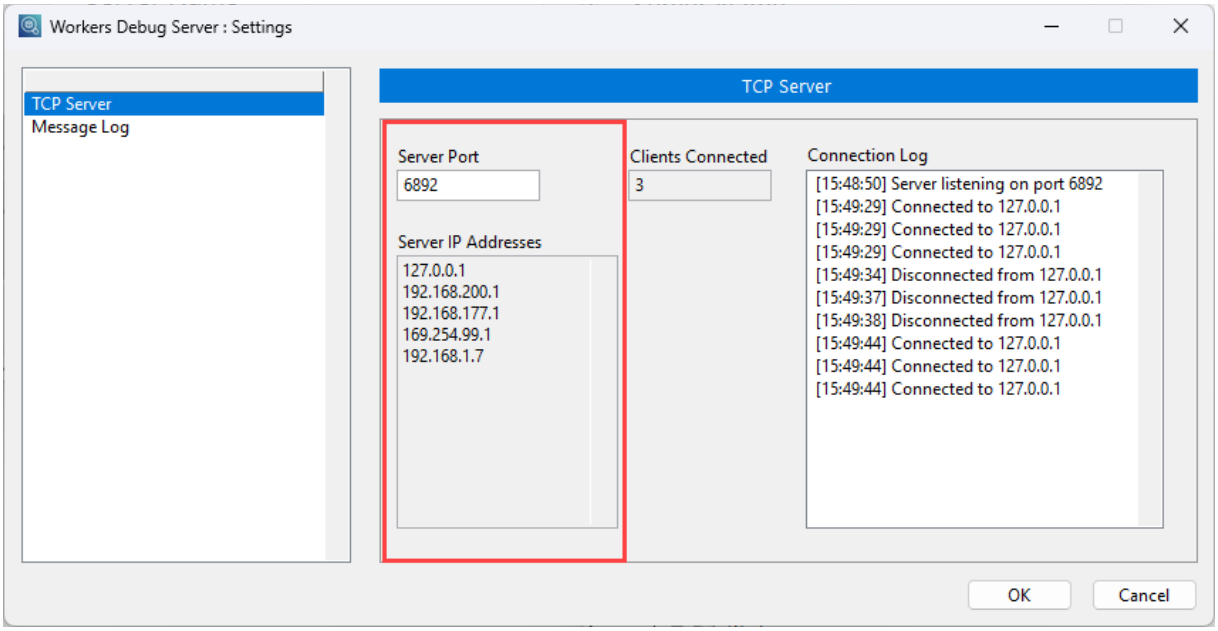

Figure 35 - IP Address and selected port for TCP communication to Debug Clients

Debug Clients

A **Debug Client** is a background task loaded by *Debug Client Loader.vi* within a Worker's Launcher VI.

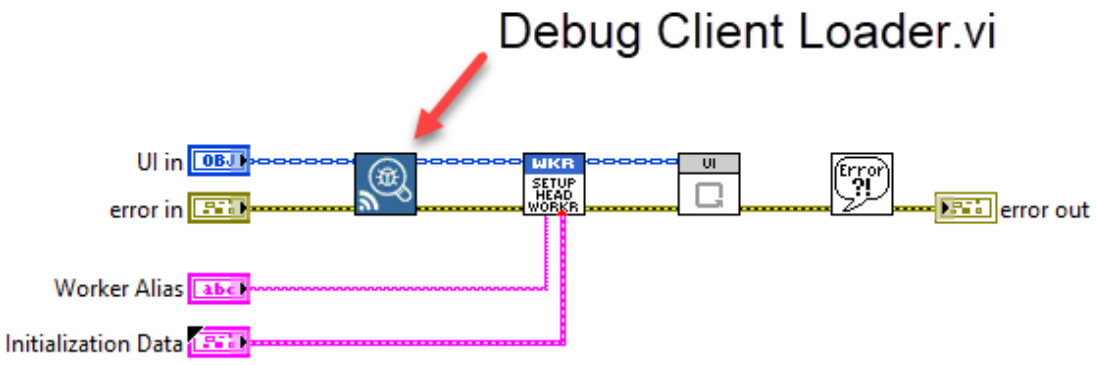

Figure 36 - Debug Client Loader.vi within an application's Launcher VI

Important

Never fork a Worker's Main Data Wire at the output of *Debug Client Loader.vi*. Use a **single** *Debug Client Loader.vi* for **every** head Worker's application instance you run.

Important Points:

- **Initialization Order:** The Debug Client must be loaded **before** launching the application's head Worker to establish proper communication with the Debug Server.
- **Connection Attempts:** The Debug Client attempts to connect to the Debug Server periodically (every few seconds) until a successful connection is made.
- **Automatic Shutdown:** Debug Clients shut down automatically when the application's head Worker shuts down or is terminated.
- **Reconnection Behavior:** If the connection to the Debug Server is lost while the Workers application is still running, the Debug Client continues to attempt reconnection periodically.

The Debug Client is lightweight and resource-friendly background task, making it suitable for inclusion in your applications regardless of the target platform or packaging format.

Good to know

It doesn't matter whether a Workers application is launched **before or after** the Debug Server. The application's debug data is stored within the Debug Client and remains **persistent**. This data will be transmitted to the Debug Server upon a successful connection between a Debug Client and the Debug Server.

Workers Application Manager

When you run the Debug Server application, the **Workers Application Manager** (Figure 37) appears as a central hub for monitoring and managing your running Workers applications. The Application Manager provides a single location where you can access and observe all your running Workers applications, view their statuses, and interact with them as needed. This section will explain the various features and functionalities of the Application Manager.

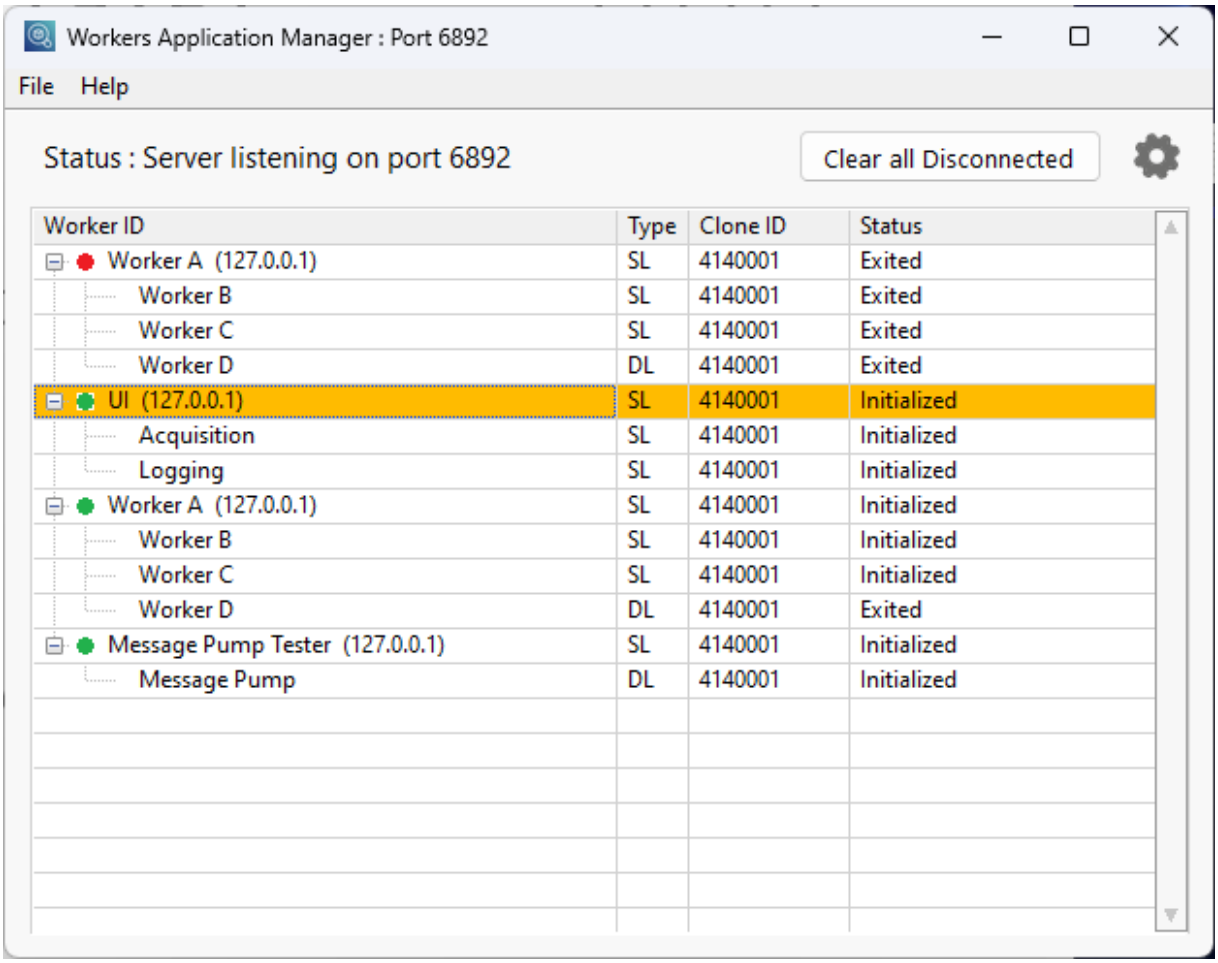

Figure 37 – Screenshot of the Workers Application Manager

Good to know

Every **Debug Client** is identified by a unique random ID number (which is not visible to the user). Therefore, it is possible to have more than one Workers application with the same **head Worker ID** running on the same target appear in the Application Manager. This allows you to monitor and differentiate between multiple instances of applications, even if they share the same head Worker ID.

Columns

The Workers Application Manager displays several columns that provide important information about your running Workers applications. Below is an explanation of each column:

Worker ID

This column shows the Workers applications that are or have been connected to the Debug Server, displaying a tree list of an application's Worker call-chain hierarchy. The individual names in each row represent the **aliases** of the Workers in the application.

In addition to the Workers themselves, the following information is also displayed:

	Application connected to Debug Server
	Application disconnected from Debug Server
(127.0.0.1)	IP Address of connected Application

Figure 38 - Worker ID column key

Type

This column shows how a Worker is linked to its Caller:

- **SL**: The Worker is **statically linked** to its Caller.
- **DL**: The Worker is **dynamically loaded** by its Caller.

Clone ID

This column displays the **Clone ID** of the Worker's Main VI. If the Worker's Main VI is reentrant, each instance running in memory has a unique Clone ID. If the Main VI is not a clone, then **Not a Clone** will be displayed for the Worker.

Status

This column displays the **status** of a Worker. The framework assigns various statuses to Workers to provide developers with state feedback during the initialization and shutdown routines of a Workers application. Developers can also assign their own statuses to their Workers by using *Update Worker Status.vi*, available on the Workers palette.

For more information about the various statuses, please visit the online help for the Workers Debug Server tool. The webpage can be accessed by clicking the **Help** button within the tool.

Worker Message Logs

Worker Message Logs are essential tools that provide insight into the runtime behavior of your Workers applications. They allow you to:

- See the flow and sequence of messages sent between Workers at runtime.
- Identify when and where errors have occurred in your applications.
- Access a running Worker's MHL cases via a right-click menu option.
- View custom debug messages generated during execution.

An example of a Workers Message Log is shown in Figure 39, illustrating the sequence of messages that have been sent to Worker MHL cases.

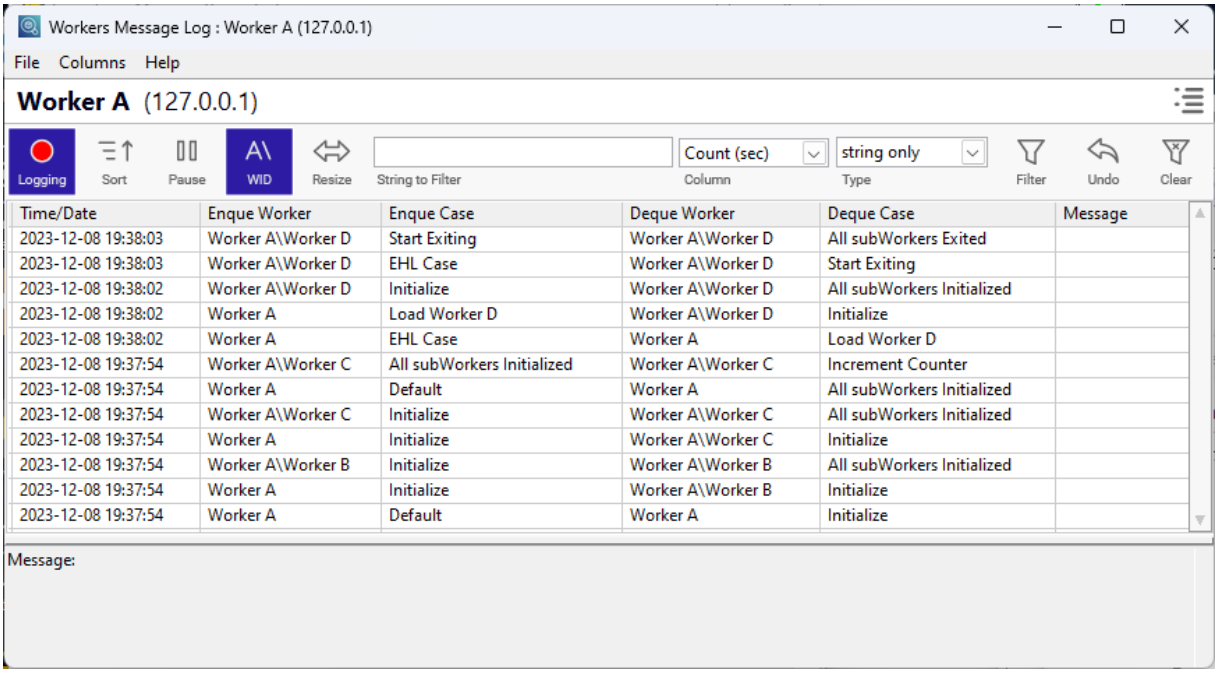

Figure 39 – A Workers application's Message Log - Worker A is the application's head Worker.

A single Message Log window is created for each Workers application, identified by the combination of the **head Worker's Alias** and the **IP Address** on the target where the application is running. Therefore, if multiple Workers applications have the same head Worker Alias and are running on the same IP Address, they will send messages to the same Message Log window. This can be useful for monitoring multiple instances collectively but may require careful interpretation to distinguish between them.

Note: To ensure that each application has its own Message Log window, assign a unique alias to the head Worker or run the applications on targets with different IP addresses

Columns

Whenever a message is dequeued within a Worker's Message Handling Loop (MHL), **metadata** about the message is forwarded to the Worker's Debug Client (if one exists) and then sent over the local network to the Debug Server. The Message Log displays the following information in its columns.

Good to know

When a Debug Client connects to the Debug Server, messages appear in the order they were **dequeued** by their respective MHLs, **not** in the order they were **enqueued**. Pressing the **Sort** button will arrange the messages based on the time they were enqueued.

Count (sec)

This column, located on the far right of the Message Log, contains a high-resolution timestamp (in seconds) of when the message was **enqueued**. The timestamp is provided by the LabVIEW VI: *High Resolution Relative Seconds.vi*. This column cannot be disabled.

Time/Date

Provides a human-readable, low-resolution timestamp of when the message was **enqueued**.

Enque Worker

Displays the Worker's ID (WID) or alias of the Worker that **enqueued** the message.

Note: If this field is blank, it may be because the message was enqueued without the **Caller's Main Data Wire** connected to the **Caller** input terminal of a Public Request VI. This terminal is not required if the Public Request VI is being used from an external LabVIEW application or from NI TestStand.

Enque Case

Shows the MHL case from which the message was **enqueued**.

Note: If this field is blank, it may be due to the same reason as above—the **Caller's Main Data Wire** was not connected to the **Caller** input terminal of a Public Request VI.

Deque Worker

Displays the Worker's ID (WID) or alias of the Worker that **dequeued** the message.

Deque Case

Shows the MHL case that **received** the message.

Priority

Indicates the priority of the message. Ie. **Low**, **Normal**, **High**, or **Critical**.

Message

Messages sent between MHL cases do not contain an additional message string. This column is reserved for use by **Error** and **User** messages (see below).

Error Message

Sent by the Workers palette VI *Error Handler.vi* which exists after the case structure in every Worker's MHL. If an error is received on this VI's **error in** input terminal, the error will be sent to the Message Log. The timestamp for the message is created within *Error Handler.vi*.

Good to know

Error Handler.vi can be placed **anywhere** in your code where you want to catch errors to be sent to the Message Log.

User Message

Sent by use of the Worker's palette VI: *Send Debugger Message.vi*. You can place this VI **anywhere** in your code where you want to send a custom message to the Message Log. The timestamp for the message is created within *Send Debugger Message.vi*.

Good to know

The Enque and Deque columns contain data for both error and user messages. This data comes from the MHL case that sent the error or user message, allowing you to see, for example, which Worker MHL case sent the error or user message (i.e. Deque Worker and Deque Case).

Finally, we arrive at the first exercise !

Don't worry if you haven't understood all the information presented up until now. This training manual is both ***a guide and reference manual*** that you can use and come back to whenever you wish to gain a greater understanding of a particular topic. The exercises in this course will walk you through the steps required to create Workers applications using the framework's development tools, so you can start working on them immediately. In time, these steps will become second nature, and at any time you can revisit the content detailed in this course manual.

In the first exercise, you will get hands-on experience with the framework's fundamental development tools and begin building your first application using the framework. Grab the accompanying **Training Course Exercise** book and let's start creating your first Workers application!

Exercise 1.1 – Creating a new Worker

In this exercise you will use the **Create/Add Worker** tool to create a new Worker and will study the various parts of the Worker within the LabVIEW project. The Worker's Main VI will also be studied.

The **Workers Debug Server** will also be used to view and access the Worker's Main VI during its execution.

Exercise 1.2 – Adding a statically-linked subWorker

In this exercise, you will start to build up an application's Worker call-chain hierarchy, by learning how to add a new Worker as a statically-linked subWorker to the Worker that was created in Exercise 1.1.

Again, the **Create/Add Worker** tool will be used to accomplish this task.

Creating Worker APIs

In the development of modular and maintainable applications using Workers for LabVIEW, creating well-defined Application Programming Interfaces (APIs) for your Workers is essential. Worker APIs provide an encapsulated and abstractable means of interacting with the internal processes of a Worker, specifically its Message Handling Loop (MHL) cases. By defining APIs, you enable both internal communication within a Worker and external communication from sources such as a Worker's Caller, other LabVIEW frameworks, or external applications like NI TestStand.

Developing Worker APIs promote good software engineering practices by allowing you to:

Decouple Workers from their Callers: Workers can operate independently of their Callers, enhancing modularity and allowing for easier testing and maintenance.

Standardize Communication: Create standardized protocols for the sending and receiving of messages, ensuring consistent datatypes and structures.

Abstract Functionality: Use base classes and inheritance to create mock Workers or simulate hardware devices, facilitating testing and development without the need for physical hardware.

Enhance Documentation: Provide clear definitions of inputs and outputs, making it easier for other developers to understand and utilize your Workers.

The Workers SDK provides a suite of development tools that assist in creating both local and public APIs for your Workers. These tools help you generate standardized, type-defined APIs that can be consistently used across Workers and Worker base classes. They also simplify the process of editing and updating your APIs, ensuring that changes propagate correctly throughout your application.

There are **four types of Worker API VIs** you can create using the Workers API tools:

1. **Local Requests**: Enable a Worker to send messages to itself for internal communication between its Event Handling Loop (EHL) and MHL or between its own MHL cases.
2. **Public Responses**: Enable a Worker to send messages to its Caller, with the Caller registering to receive these messages. This is useful for asynchronously notifying the Caller of events or data updates within the Worker.
3. **Public Requests**: Allow external sources, such as a Worker's Caller or other applications, to send messages to a Worker, initiating actions or requesting data.
4. **Public Request with Reply**: Facilitate synchronous communication where an external source sends a message to a Worker and waits for a reply before proceeding. This is essential when immediate feedback or data is required from the Worker.

Each API VI and its associated type definitions (typedefs) are designed with specific filename prefixes and distinctive icon glyphs to easily identify their purpose (Figure 40). These visual cues help developers quickly recognize the type and intended use of each API VI within the LabVIEW environment.

Name	Local Request	Public Response	Public Request	Public Request w/Reply
Type	Asynchronous	Asynchronous	Asynchronous	Synchronous
Purpose	Send message to self	Send message to Caller* from Worker. *Caller must register to receive the Response	Send message to Worker	Send message to Worker, and wait for reply
Icon Glyph				
File prefix	rql_	rsp_	rqp_	rwr_

Figure 40 - The four Worker API VIs created by the Workers API tools

Good to know

All API VIs created in Workers 5.0 are by default **static dispatch** VIs.

MHL Case Sections

The **Public API Builder** tool and the **Create Worker MHL Case** tool automatically create the MHL cases required to receive messages sent by Worker API VIs (see Figure 41). These MHL cases are added to specific sections within the Worker's MHL case structure (which are also created by the API tools), organizing them based on their functionality. The three main MHL case sections are.

Local API Cases

These cases are created by the **Create Worker MHL Case** tool to receive **Local Request** messages. Local API MHL cases process messages sent internally within a Worker to itself, allowing for structured internal modular communication.

Public API Cases

These cases are created by the **Public API Builder** tool to receive **Public Request** messages and **Public Request with Reply** messages. Public API cases process messages sent to a Worker from external sources, such as the Worker's Caller or other external applications, enabling communication with the Worker from outside its own scope.

Response Cases

These cases are created by the **Create Worker MHL Case** tool when you create an MHL case with the option **"Add to Response Cases section"** selected. Response cases process messages that are sent to a Caller from a subWorker, enabling decoupled asynchronous communication up the Worker call-chain hierarchy.

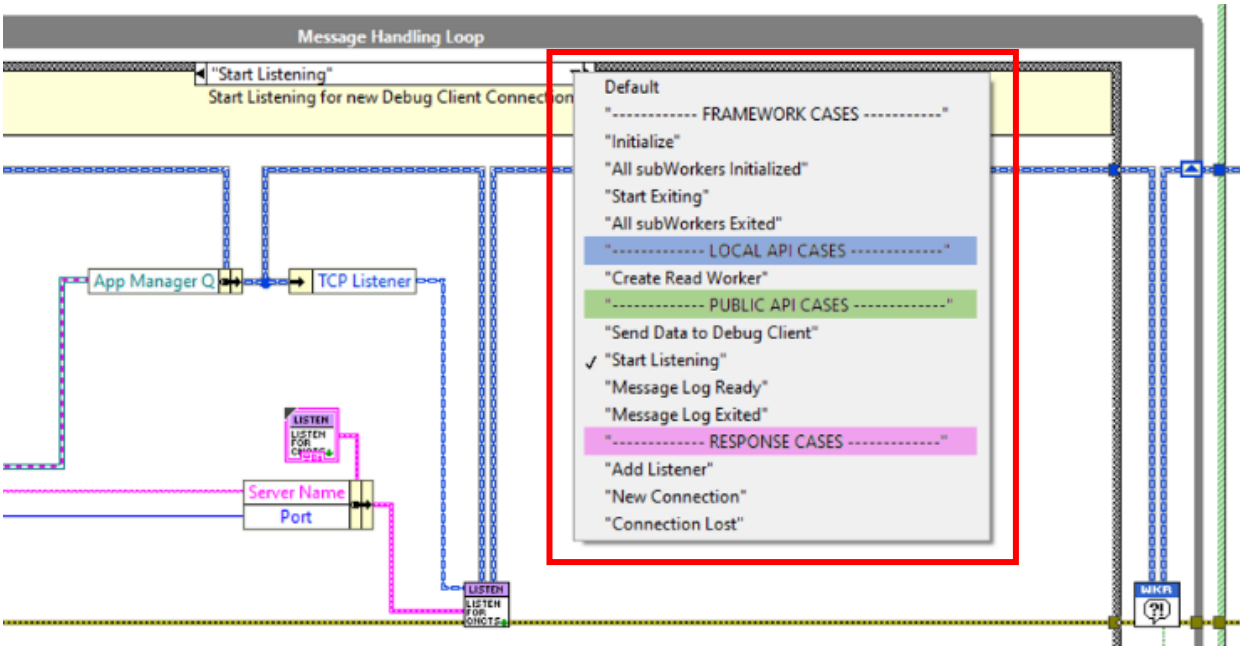

Figure 41 - Three MHL case structure sections created and managed by the Workers API tools

Worker API VI folders

When created, Worker API VIs will be added automatically by the tools to the following folders. VIs that are expected to be used locally (i.e. only by the Worker that owns the VI or by the Worker base class that owns the VI) are set to **Protected** scope. All other VIs are set to **Public** scope.

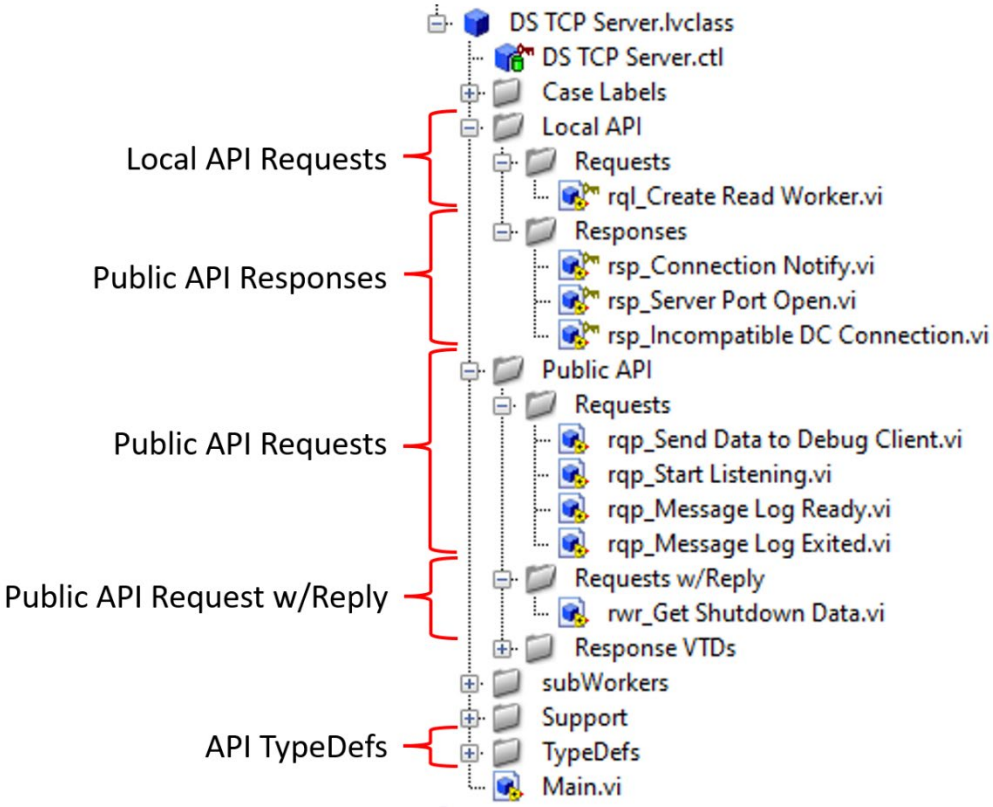

Figure 42 - Worker API VI folder structure

Local Requests

Purpose	Message Type	Icon Glyph	File Prefix
Send message to self	Asynchronous		rql_

A Local Request is a message that a Worker sends to itself. You can use Local Request VIs to send a message from a Worker's EHL to its MHL, or between MHL cases of the same Worker.

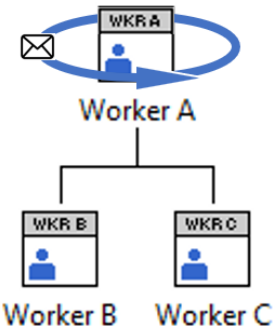

Figure 43 - Illustration showing that a Local Request is a message a Worker sends only to itself

Local Request usage example

Figure 44 demonstrates the use of a Local Request VI sending a message from the EHL to the MHL of the Worker's QMH, and between MHL cases of the same Worker.

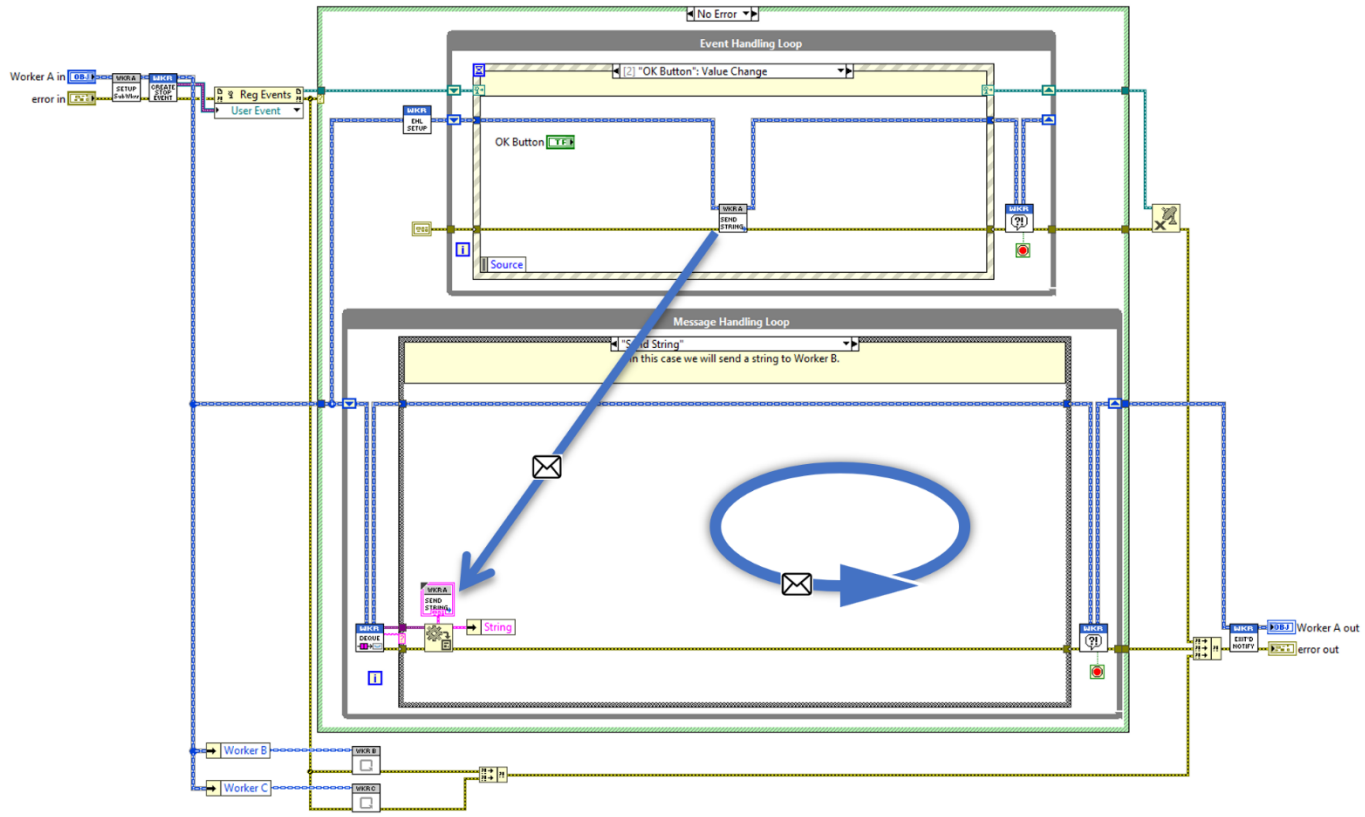

Figure 44 - Example of a Local Request VI sending messages within a Worker's Main VI

Local Request VI

A **Local Request VI** is wired in-line with a Worker's Main Data Wire and can be used within the Worker's EHL or MHL. This VI sends a message to the Worker's MHL case that shares the same name as the Request VI.

Every Local Request VI filename takes the prefix **rql_** and contains a blue glyph in the bottom-right corner of its icon. An example of a Local Request VI is shown in Figure 45.

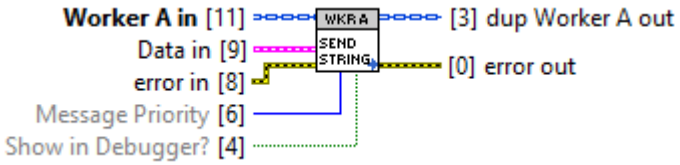

*Figure 45 - Example of a Local Request VI - **rql_Send String.vi** in Worker A*

Data in

The data sent with the message. By default, this is a typedef that shares its filename with the Local Request VI.

Message Priority

Sets the priority of the message—**Low**, **Normal** (default), or **High.**

Show in Debugger?

A Boolean flag; when set to TRUE, the message appears in the Debug Server's Message Logs.

Local Request Typedef

Local Request typedefs are custom data structures (clusters) created with the same icon and filename as their matching Local Request VI. They define the datatype of the message payload sent to the Local Request's MHL case.

Local Request MHL Cases

For every Local Request, an MHL case is created to receive the message sent by the Local Request VI. These cases are automatically added to the Worker's case structure section labeled **--- LOCAL API CASES ---** (see Figure 46). To convert the data sent with the message to its specific datatype, a **Variant to Data** node is included at the beginning of each Local Request MHL case. This node converts the variant data back to the defined typedef.

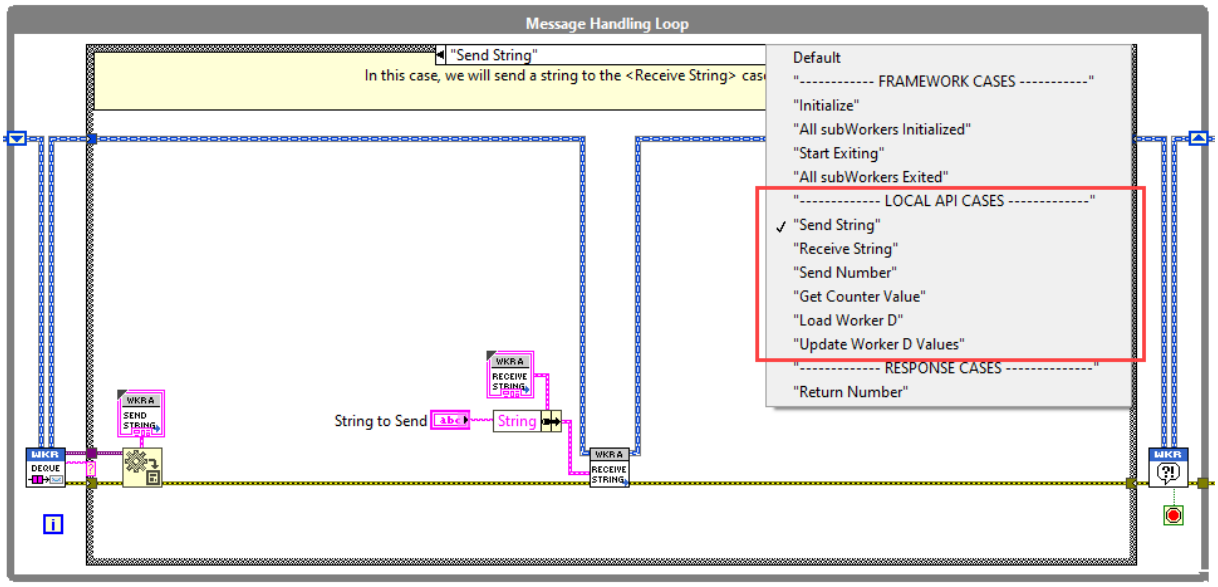

Figure 46 – A Worker's Main VI showing its Local Request MHL cases

Local Request Folders

Local Request VIs created by the **Create Worker MHL Case** tool are added to a Worker's *Local API* >> *Requests* folder. Their scope is set to **Protected** because they are designed for use only within the Worker that owns them.

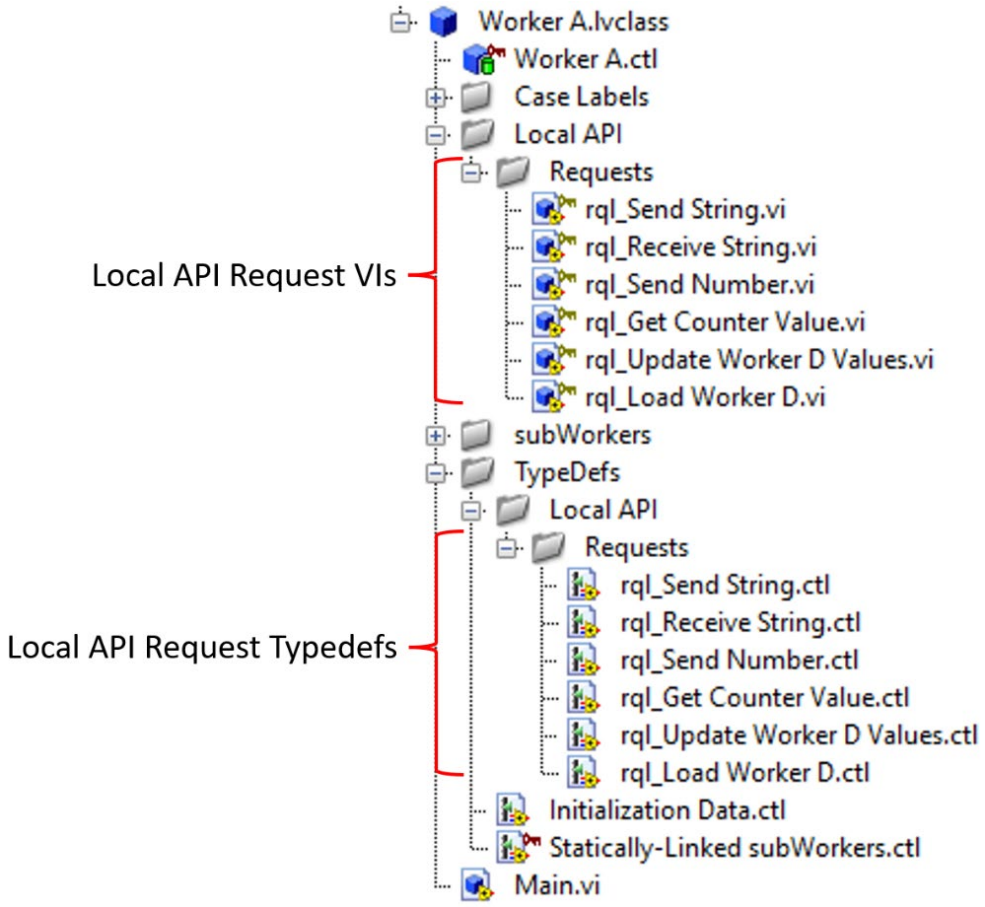

Figure 47 - Local Request folder structure within a Worker

Exercise 1.3 – Creating a Local Request

In this exercise, you will learn how to use the **Create Worker MHL Case** tool to create a Local Request in a Worker. You will then use the Local Request VI to send a message from the Worker's EHL to its MHL.

Public Responses

Purpose	Message Type	Icon Glyph	File Prefix
Send message to Caller from Worker	Asynchronous		rsp_

A **Public Response** is an asynchronous message that a Worker can send to its Caller; however, the Caller must first **register** to receive the Response message from the Worker (see page 67).

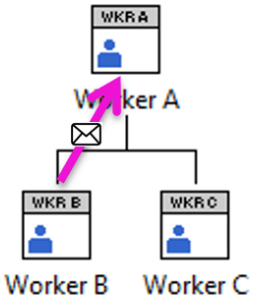

Figure 48 - Diagram showing a Public Response message being sent from Worker B to its Caller, Worker A

Public Response usage example

Figure 49 demonstrates the use of a Public Response VI sending a message from a MHL case of **Worker B** to its Caller. **Worker B** does not know which case of its Caller the Response message will be sent to. It is up to the Caller, during the initialization of **Worker B**, to register the Response message and to provide **Worker B** with the Caller's MHL case that will receive the Response message.

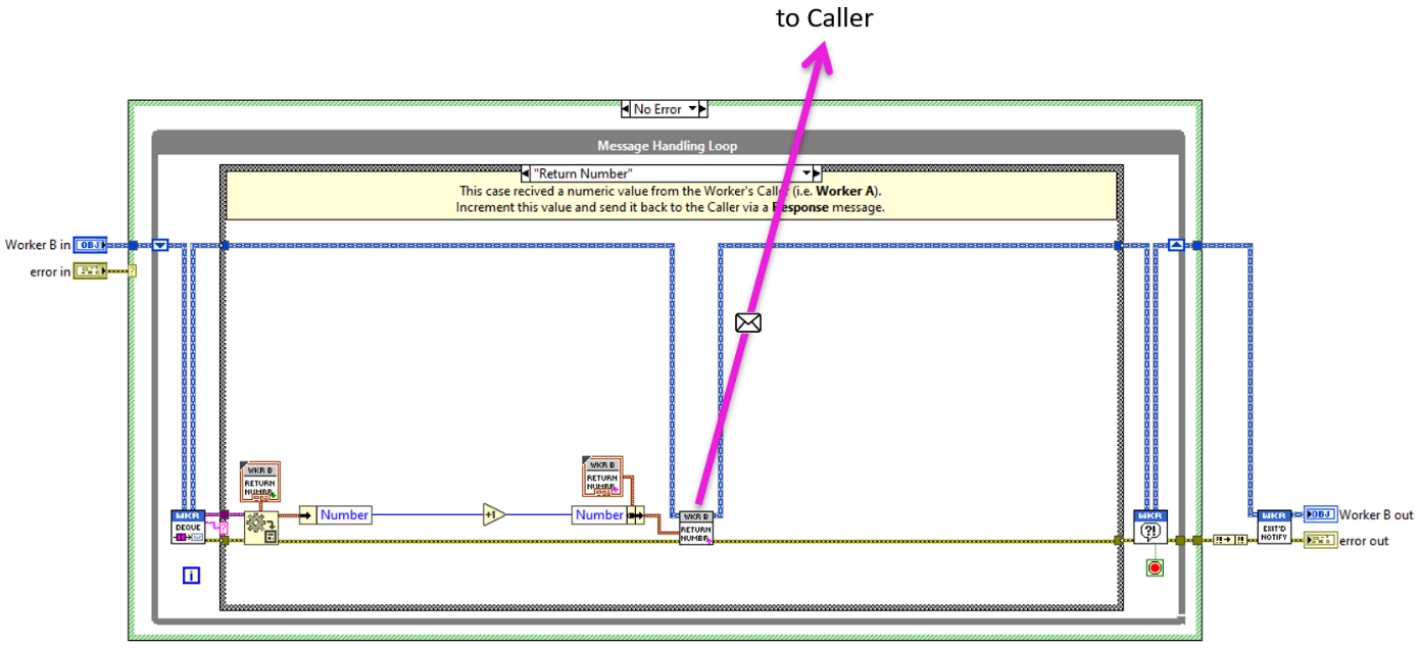

Figure 49 - Worker B's Main VI showing a Public Response VI sending a message to its Caller

Public Response VI

A **Public Response VI** is wired in-line with a Worker's Main Data Wire. The Public Response VI will send a message to the Worker's Caller **if** the Caller has registered to receive the Response message.

Every Public Response VI filename takes the prefix **rsp_** and contains a pink glyph in the bottom-right corner of its icon. An example of a Public Response VI is shown in Figure 50.

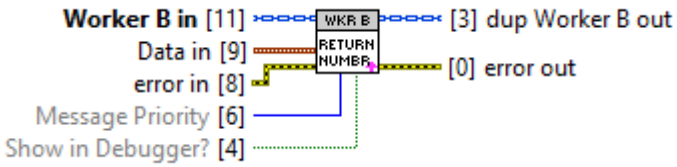

*Figure 50 - Example of a Public Response VI - **rsp_Return Number.vi** in Worker B*

Data in

The data sent with the message. By default, this is a typedef that shares its filename with the Public Response VI.

Message Priority

Sets the priority of the message—**Low**, **Normal** (default), or **High**.

Show in Debugger?

A Boolean flag; when set to TRUE, the message appears in the Workers Debug Server's Message Log.

Public Response VTD VI

Every Public Response is accompanied by a **Variant to Data (VTD) VI**. This VI is used by a Worker's Caller to convert the variant data received with the Response message back to its specific datatype. The VTD VI is placed in the registered Public Response MHL case of the Caller.

Every Public Response VTD VI filename takes the prefix **rspvtd_**. Every Public Response VTD VI icon contains a pink rectangular glyph in its bottom-right corner. An example of a Public Response VTD VI is shown in Figure 51.

*Figure 51 - Example of a Public Response VTD VI - **rspvtd_Return Number.vi** in Worker B*

Data in

The variant data that was sent along with the Response message.

Data out

The strictly typed data converted from the variant, matching the typedef associated with the Public Response VI.

Public Response Typedef

Public Response typedefs are custom data structures (clusters) created with the same icon and filename as their matching Public Response VI. They define the datatype of the message payload sent to the registered Public Response MHL case of the Caller.

Public Response MHL Cases

The MHL case that receives a Public Response message exists in the Worker's Caller. The Caller must register to receive a Response message from a subWorker. Figure 52 shows an MHL case in **Worker A** (from Figure 48) that will receive a Response message from **Worker B**.

MHL cases created to receive Response messages are added to the Caller's case structure section labeled **--- RESPONSE CASES ---** (see Figure 52). To convert the variant data sent with the Response message back to its specific datatype, a **Response VTD VI** is used at the beginning of each registered Response MHL case.

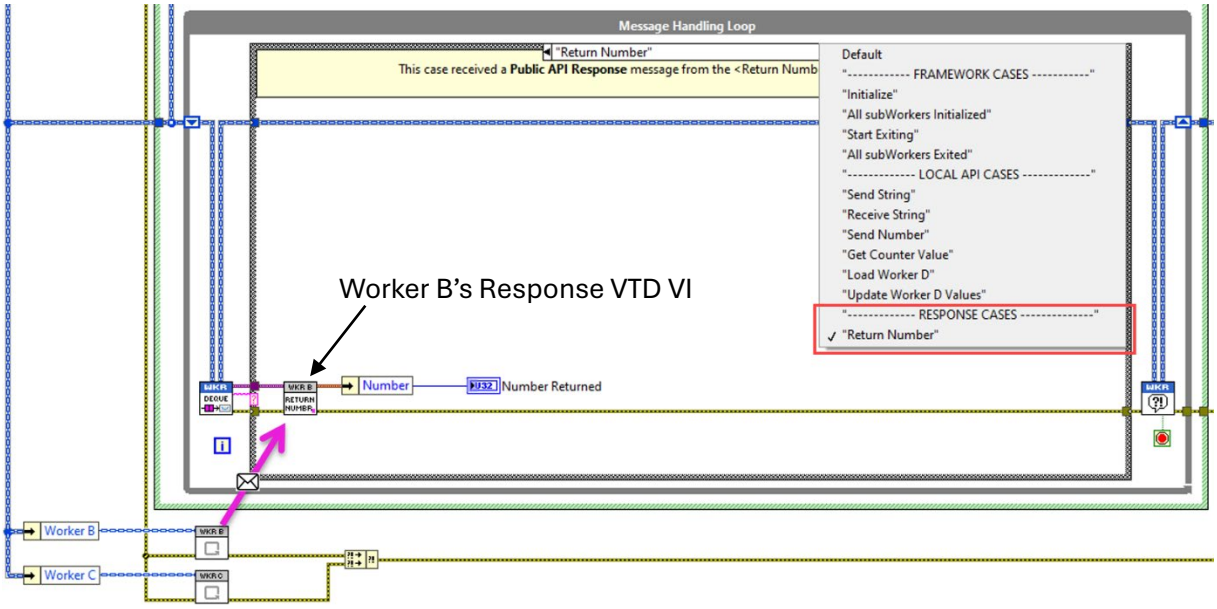

Figure 52 - Worker A's Main VI showing a registered Response MHL case

Public Response Folders

Public Response VIs created by the **Public API Builder** tool are added to a Worker's *Local API >> Responses* folder. Their scope is set to **Protected** because they are intended for use only within the Worker that owns them.

Public Response VTD VIs are added to a Worker's *Public API >> Response VTDs* folder. Their scope is set to **Public** because they are intended to be used in the MHL cases of a Worker's Caller.

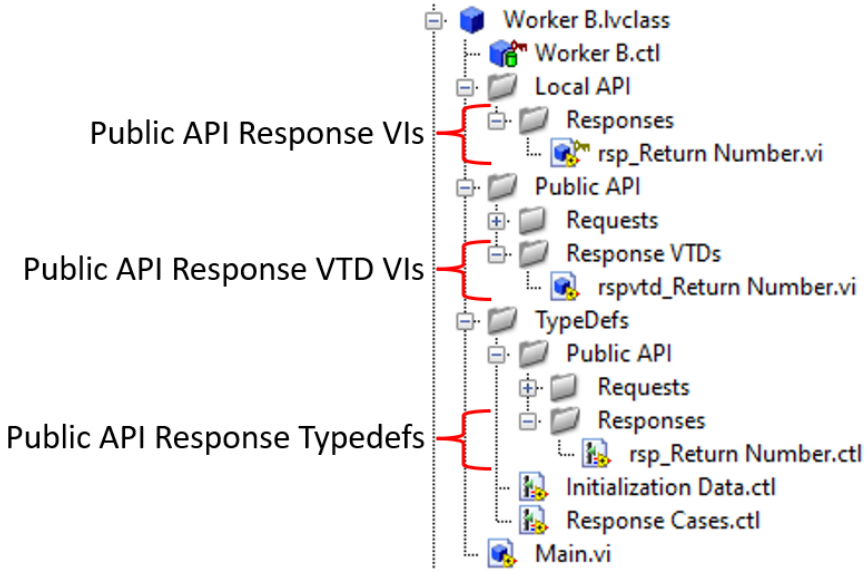

Figure 53 - Public Response folder structure within a Worker

Response Cases.ctl

A Public Response requires the name of the Caller's MHL case to be **passed** into the Worker through the Worker's *Initialization Data.ctl*. This process occurs when a Worker's Public Response is registered with the Caller.

Every time a new Public Response is created using the **Public API Builder** tool, a **Case Label** string control is automatically added to the Worker's *Response Cases.ctl* typedef. Figure 54 shows the Case Label string controls that have been added to a Worker's *Response Cases.ctl* typedef.

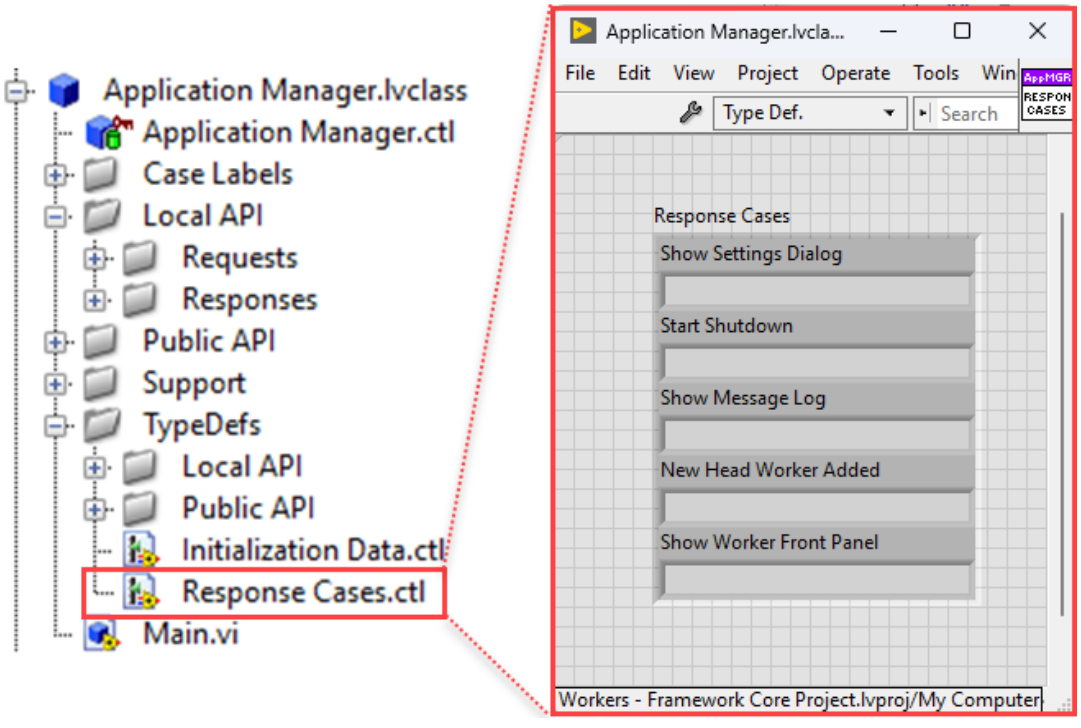

*Figure 54 - A Worker's **Response Cases.ctl** used to register Public Responses with a Caller*

Every new Worker and Worker base class (from Workers 5.0 onwards) wires the Worker's *Response Cases.ctl* from the *Initialization Data.ctl* typedef into the *Response Cases.ctl* within the Worker's Main Data Wire as shown in Figure 55. Without this **transfer of information**, the Public Response VIs will not have the necessary data to know which of the Caller's MHL cases to send its Public Response messages to.

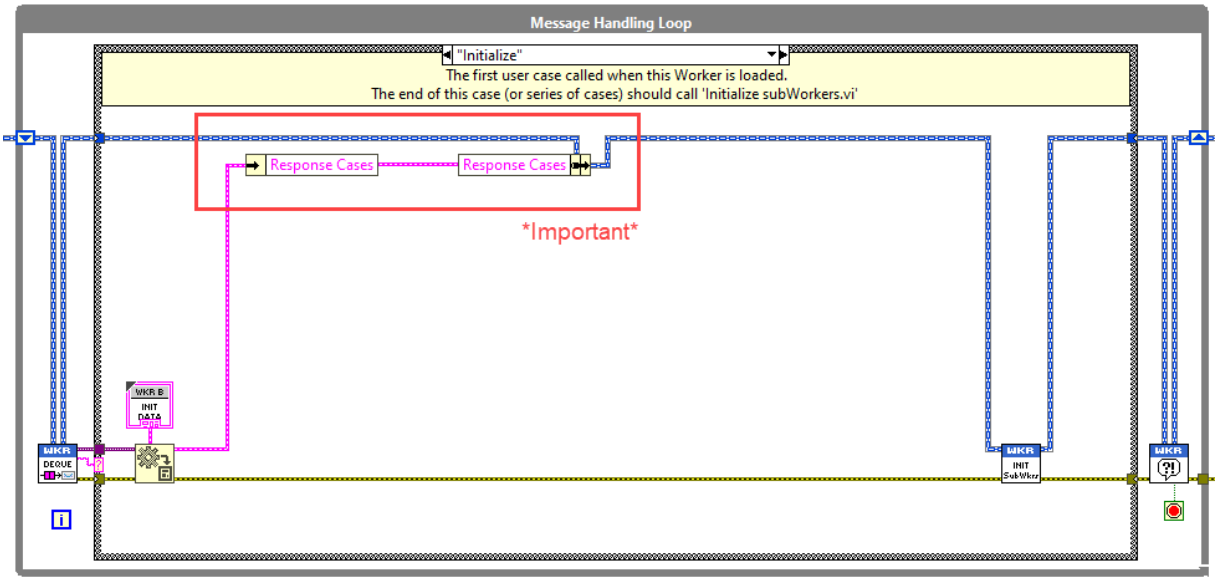

Figure 55 - Transferring a Caller's registered Response MHL Cases into a Worker

Exercise 1.4 – Creating a Public Response

In this exercise, you will use the Public API Builder tool to create a Public Response in a Worker. You will then use the Public Response VI to send an asynchronous message from the Worker to its Caller.

Registering Public Responses with a Caller

A Worker is not inherently coupled to its Caller; therefore, a Caller must register to receive Public Response messages from its subWorkers. It is up to the Caller to decide whether it wants to receive any Public Response messages from a subWorker.

How to register a subWorker's Public API Responses

Let's take an example where we want to register a Public Response from **Worker B** with its Caller, **Worker A**. Follow these steps to complete the registration process:

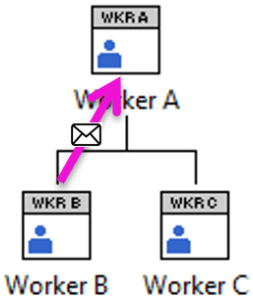

Figure 56 - Diagram showing a Public Response message being sent from Worker B to its Caller, Worker A

Step 1: Create an MHL Case in the Caller to Receive the Response Message

1. Open the block diagram of **Worker A**'s Main.vi.
2. Activate the **Create Worker MHL Case** tool by pressing the QuickDrop Shortcut: **Ctrl+9**.
3. In the **Create Worker MHL Case** tool, enter the name of the new MHL case that will receive the Response message (e.g., Return Number).
4. Check the option "**Add to Response Cases Section**" so that the tool will create the MHL case and add it to the **--- RESPONSE CASES ---** section of the Caller's MHL case structure.

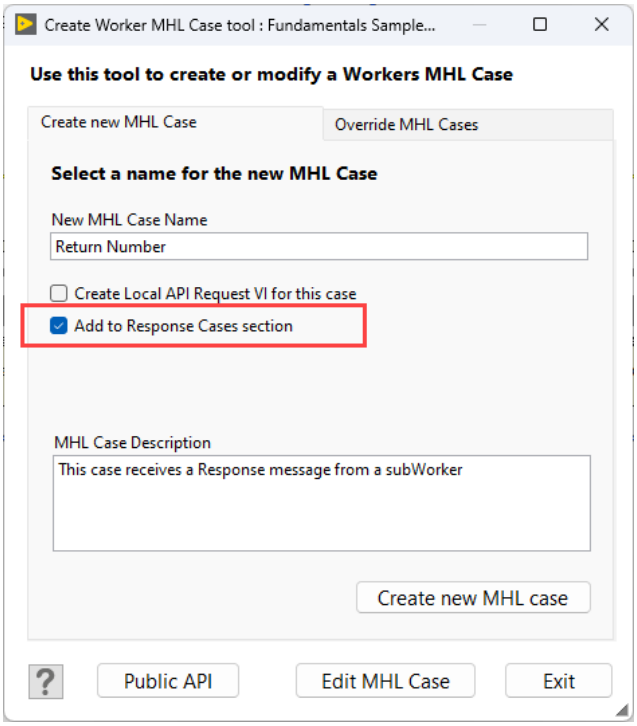

Figure 57 - Using the Create Worker MHL Case tool to create an MHL case to receive a Response message

Step 2: Pass the Caller's MHL Case Name to the subWorker

1. In **Worker A's <Initialize> MHL case**, locate the Set *Initialization Data - Worker B.vi*.
2. Open Set *Initialization Data - Worker B.vi* and view its block diagram.
3. Drop the **Case Label** that was created for the new MHL case that you created in **Step 1** (e.g. *c_Return Number.vi*), into the VI. You can find the Case Label that was created in the Caller's (**Worker A**) **Case Label** virtual folder.
4. Wire the **Case Label** string output to the corresponding input in the Initialization Data cluster of **Worker B** of the Public Response that you want to register.

   Note: This step informs **Worker B** of the exact MHL case in **Worker A** that should receive the Public Response message.

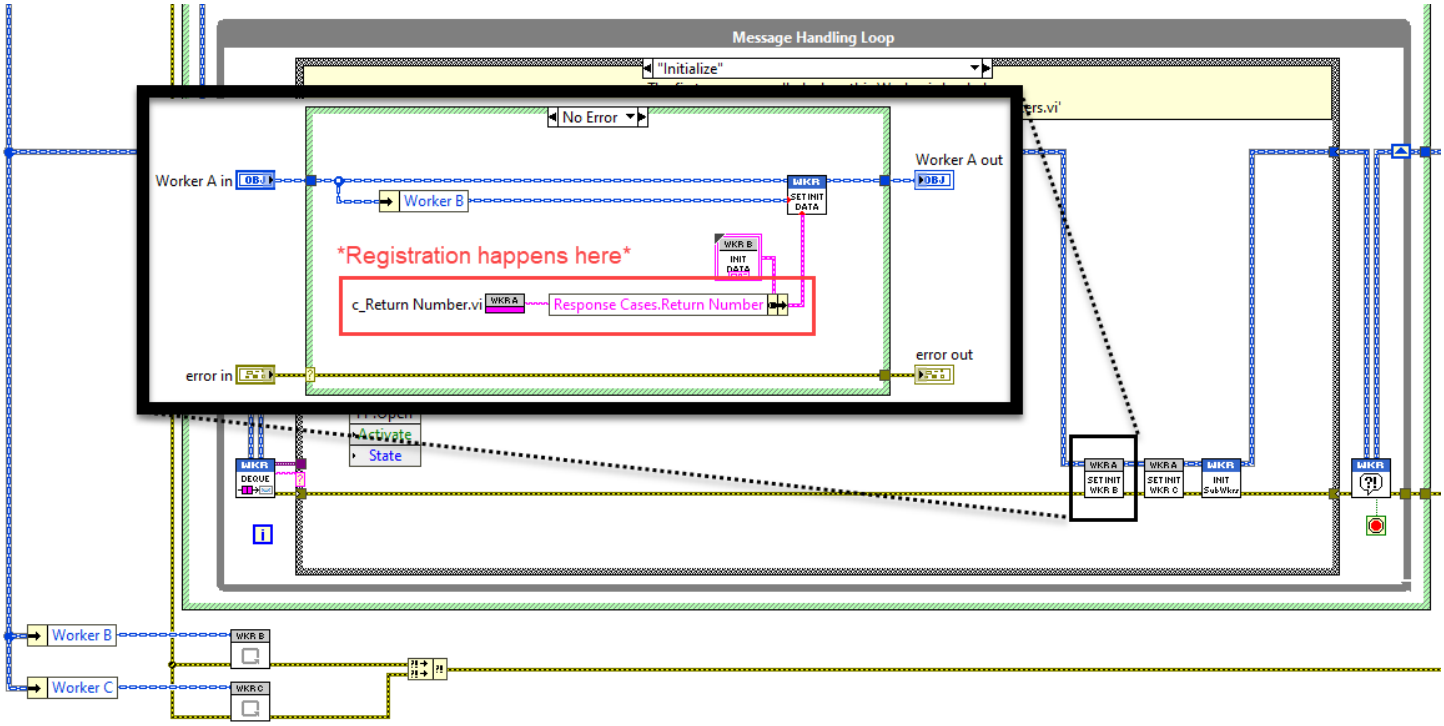

Figure 58 – Passing Worker A's MHL Response case name into Worker B

Step 3: Convert Public Response Variant Data to its Specific Datatype

1. In the MHL case you created in **Worker A** to receive the Public Response message (e.g., <Return Number>), place the **Public Response VTD VI** (from **Worker B**).
    - You can find this VI in **Worker B**'s *Public API >> Response VTDs* folder (e.g., *rspvtd_Return Number.vi*).
2. Connect the Data terminal of the MHL case structure to the **Data In** input of the Public Response VTD VI.
3. The **Data Out** output of the VTD VI will provide the message data converted to the correct datatype, as defined by the typedef associated with the Public Response.

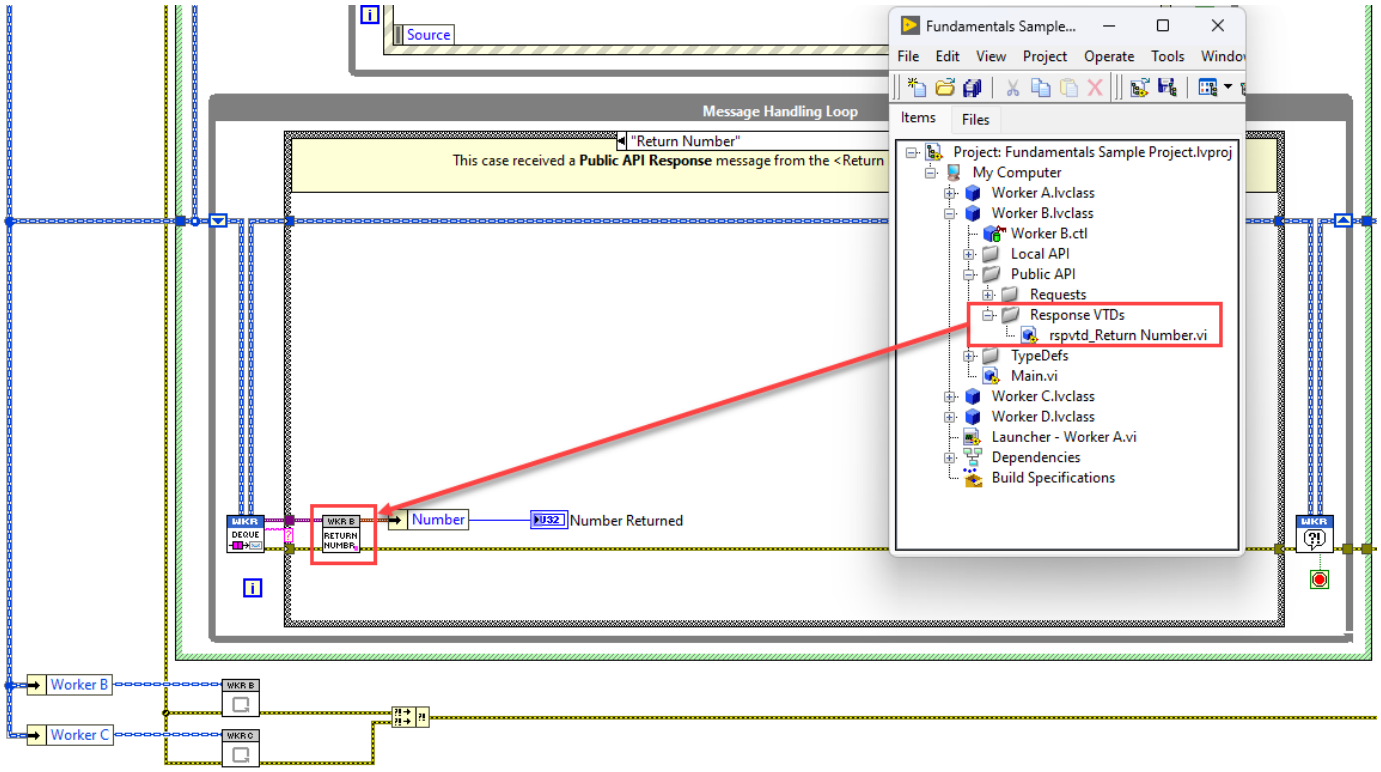

*Figure 59 - Placing **rspvtd_Return Number.vi** in Worker A's MHL case to convert message data*

This completes the registration process for a Public API Response. **Worker B** can now send the Public Response message to its Caller, **Worker A**. Worker A will receive the Response message in the <Return Number> case of its MHL, and the data in the message will be converted to its specific datatype using **Worker B**'s Public Response VTD VI.

Exercise 1.5 – Registering a Public Response

In this exercise, you will learn how to register a Public Response with a Worker's Caller. Registration of Public Responses is required for a Caller to receive a Public Response from a subWorker.

Public Requests

Purpose	Message Type	Icon Glyph	File Prefix
Send message to Worker from external source	Asynchronous		rqp_

A **Public Request** is an asynchronous message that can be sent to a Worker from an external source, such as from a Worker's Caller (within a Workers application), another LabVIEW application or framework, or from external applications like NI TestStand. Public Requests enable external entities to interact with a Worker by sending messages to its MHL.

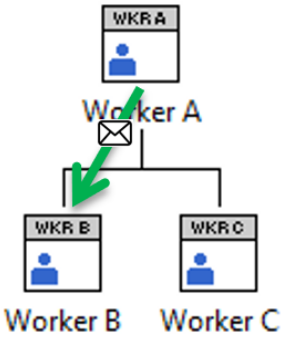

Figure 60 - Diagram illustrating that a Public Request is a message sent from an external source to a Worker

Public Request usage example

Figure 61 demonstrates the use of a Public Request VI sending a message from a MHL case of **Worker A** to **Worker B**. To ensure the Public Request VI knows which instance of **Worker B** to send the message to, the **Worker's Handle** must be wired to the top-left input terminal of the Public Request VI.

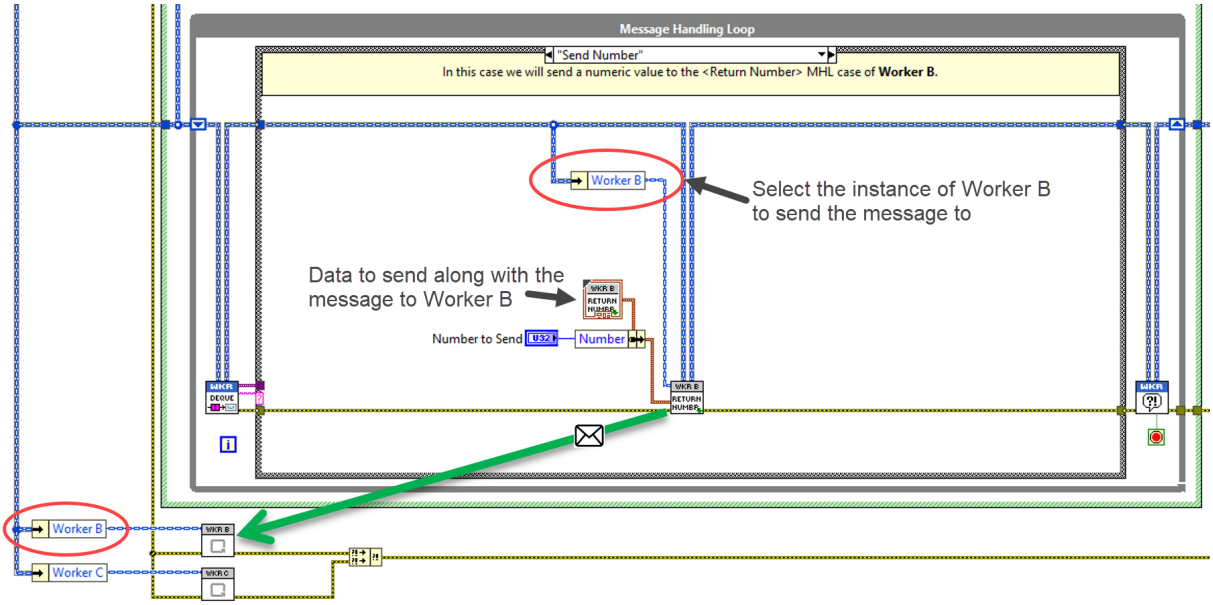

Figure 61 - Worker A's Main VI showing a Public Request VI sending a message to Worker B

Public Request VI

A **Public Request VI** is wired in-line with the Caller's **Main Data Wire**. The Public Request VI sends a message to the Worker whose **Worker Handle** is wired to its top-left input terminal. The message will be received by the Worker's MHL case that has the same name as the Public Request VI.

Every Public Request VI filename takes the prefix **rqp_** and contains a green glyph in the bottom-right corner of its icon. An example of a Public Request VI is shown in Figure 62.

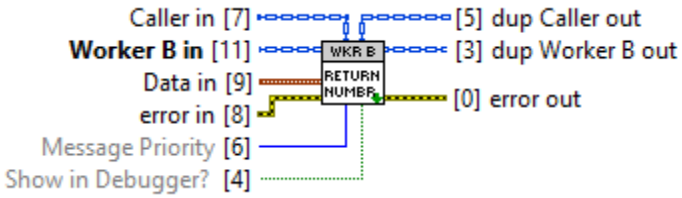

*Figure 62 - Example of a Public Request VI - **rqp_Return Number.vi** in Worker B*

Data in

The data sent with the message. By default, this is a typedef that shares its filename with the Public Request VI.

Message Priority

Sets the priority of the message—**Low**, **Normal** (default), or **High**.

Show in Debugger?

A Boolean flag; when set to TRUE, the message appears in the Workers Debug Server's Message Log.

Public Request Typedef

Public Request typedefs are custom data structures (clusters) created with the same icon and filename as their matching Public Request VI. They define the datatype of the message payload sent to the Public Request's MHL case.

Public Request MHL Cases

For every Public Request, an MHL case is created to receive the message sent by the Public Request VI. Public Request MHL cases are automatically added to the Worker's case structure section labeled **--- PUBLIC API CASES ---** (see Figure 63). To convert the data sent with the message back to its specific datatype, a **Variant to Data** node is required at the beginning of each Public Request MHL case. This node is automatically added when the MHL case is created using the **Public API Builder** tool.

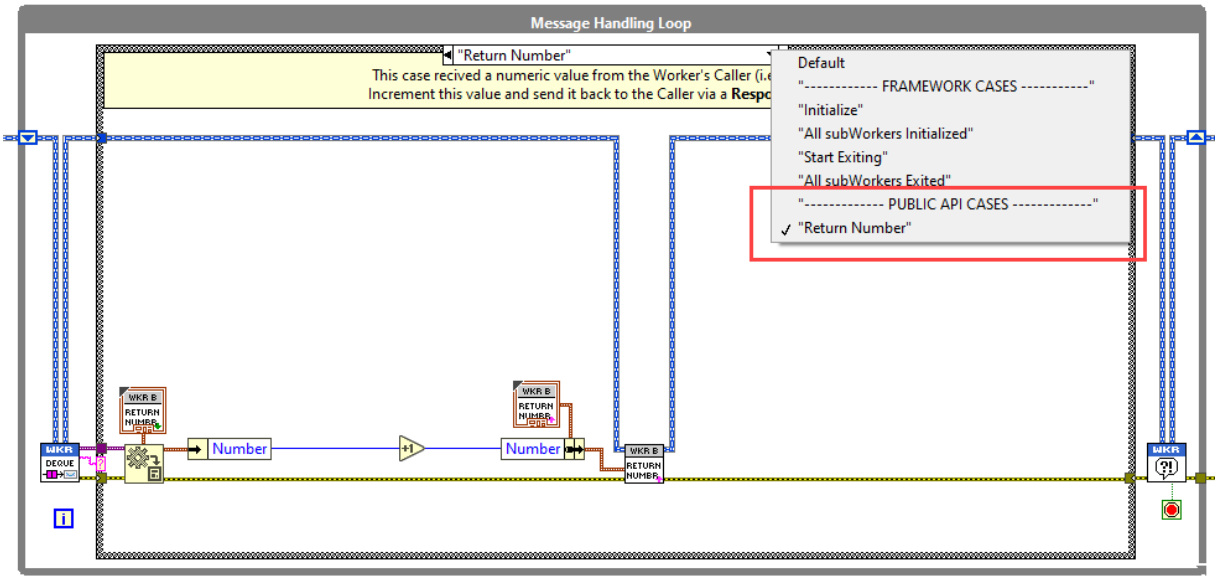

Figure 63 - Worker B's Main VI showing a Public Request MHL case

Public Request Folders

Public Request VIs created by the **Public API Builder** tool are added to a Worker's *Public API >> Requests* folder. Their scope is set to **Public** because they are designed to be used by code external to the Worker that owns them.

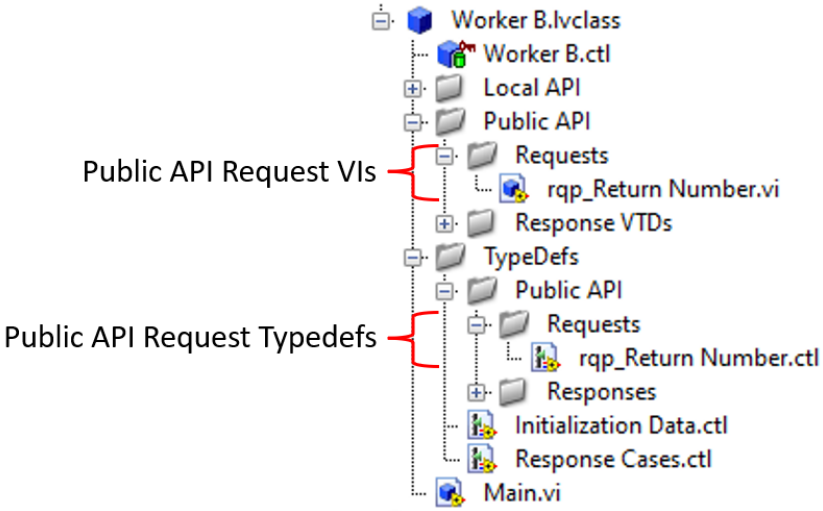

Figure 64 - Public Request folder structure within a Worker

Public Request with Reply

Purpose	Message Type	Icon Glyph	File Prefix
Send message to Worker from external source, and wait for reply from Worker	Synchronous		rwr_

A **Public Request with Reply** is a synchronous message that can be sent to a Worker from an external source, such as a Worker's Caller (within a Workers application), another LabVIEW application or framework, or from external applications like NI TestStand. Unlike a standard **Public Request** VI, a **Public Request with Reply** VI will ***wait*** until it receives a **Reply** message back from the Worker to which the Request message was sent, or until a timeout occurs.

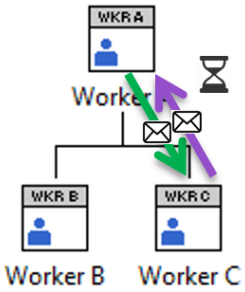

Figure 65 - Diagram illustrating that a Public Request with Reply is a message sent from an external source to Worker C, with the sender waiting for a reply.

Public Request with Reply usage example

Figure 66 demonstrates the use of a Public Request with Reply VI sending a message from an MHL case of **Worker A** to **Worker C**, and receiving reply data back from **Worker C**. For the Public Request with Reply VI to know which instance of **Worker C** to send a message to, the **Worker's Handle** must be wired to its top-left input terminal.

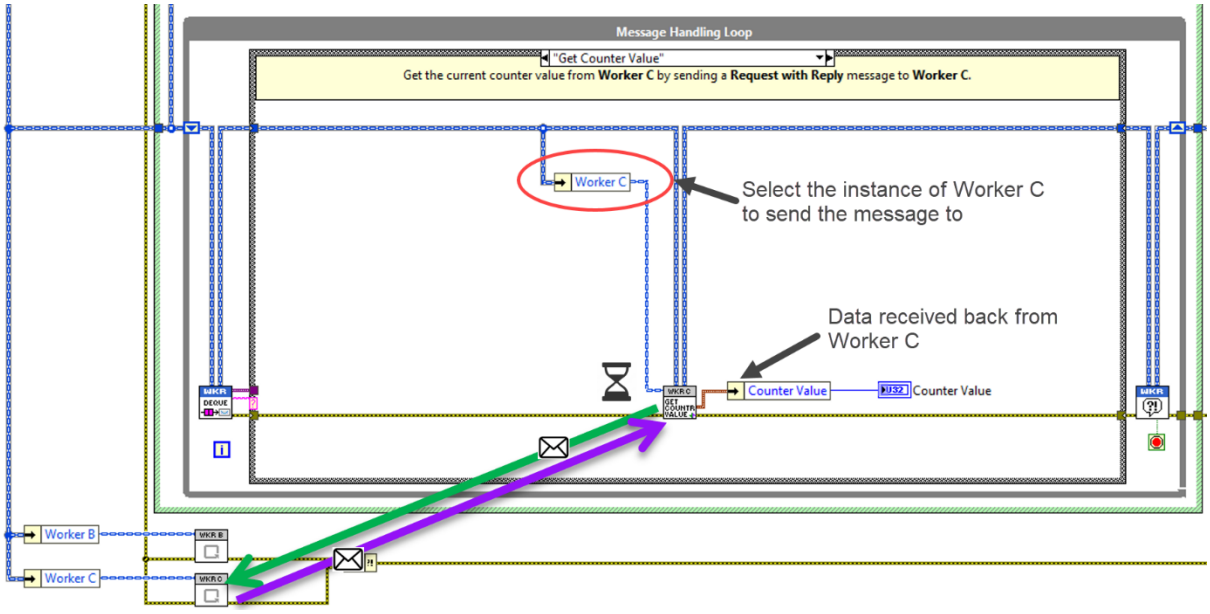

Figure 66 - Worker A's Main VI showing a Public Request with Reply VI sending a message to Worker C and receiving reply data back from Worker C

Public Request with Reply VI

A **Public Request with Reply VI** is wired in-line with the Caller's **Main Data Wire**. The VI sends a message to the Worker whose Worker Handle is wired to its top-left input terminal. The message is received by the Worker's MHL case that has the same name as the Request with Reply VI. The Public Request with Reply VI then waits until it receives a Reply message back from the Worker or until a specified timeout duration is exceeded.

Every Public Request with Reply VI filename takes the prefix **rwr_** and contains a green/purple glyph in the bottom-right corner of its icon. An example of a Public Request with Reply VI is shown in Figure 67.

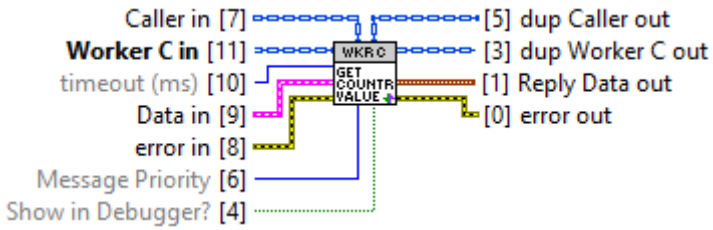

*Figure 67 - Example of a Public Request with Reply VI - **rwr_Get Counter Value.vi** in Worker C*

Data in

The data sent with the message. By default, this is a typedef that shares its filename with the Public Request with Reply VI.

Message Priority

Sets the priority of the message—**Low**, **Normal** (default), or **High**.

Show in Debugger?

A Boolean flag; when set to TRUE, the message appears in the Workers Debug Server's Message Log.

timeout (ms)

The time in milliseconds that the VI will wait for a Reply message back from the Worker it sent the message to. A value of -1 means that the VI will wait indefinitely.

Reply Data out

The data returned from the Worker that the Request message was sent to.

Public Request with Reply Typedefs

For every **Public Request with Reply** two typedefs are created. These typedefs are created with the same icon and filename as their Public Request with Reply VI**.**

Request Typedef (green icon glyph): Defines the data structure for the Request message sent from the external source to the Worker.

Reply Typedef (purple icon glyph): Defines the data structure for the **Reply** message sent back to the **Public Request with Reply VI**.

Public Request with Reply MHL Cases

For each Public Request with Reply, an MHL case is created in the Worker to receive the message sent by the Public Request with Reply VI. These MHL cases are automatically added to the Worker's case structure section labeled **--- PUBLIC API CASES ---** (see Figure 68). To convert the data sent with the message to its specific datatype, a **Variant to Data** node is included at the beginning of the MHL case. This node is automatically added when the MHL case is created using the **Public API Builder** tool. Within the MHL case, after processing the Request, the Worker sends a Reply message back to the sender using the framework's *Callback Easy Reply.vi*, providing the data to be returned to the Public Request with Reply VI.

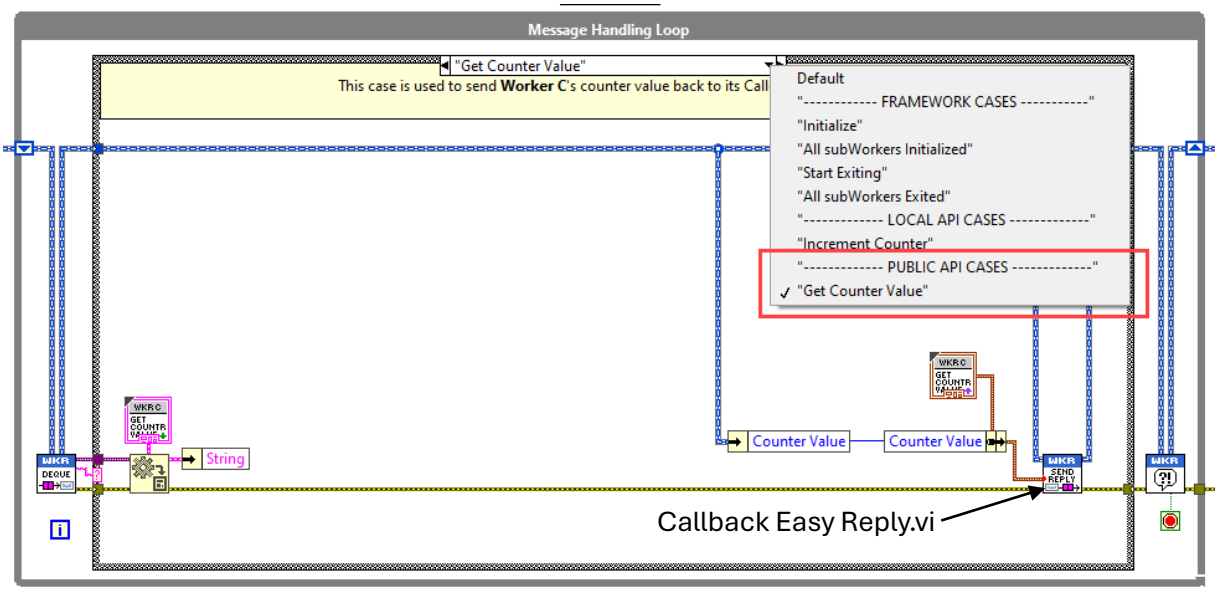

Figure 68 - Worker C's Main VI showing a Public Request with Reply MHL case

Public Request with Reply Folders

Public Request with Reply VIs created by the **Public API Builder** tool are added to a Worker's *Public API* >> *Request w/Reply* folder. Their scope is set to **Public** because they are intended to be used by code external to the Worker that owns them.

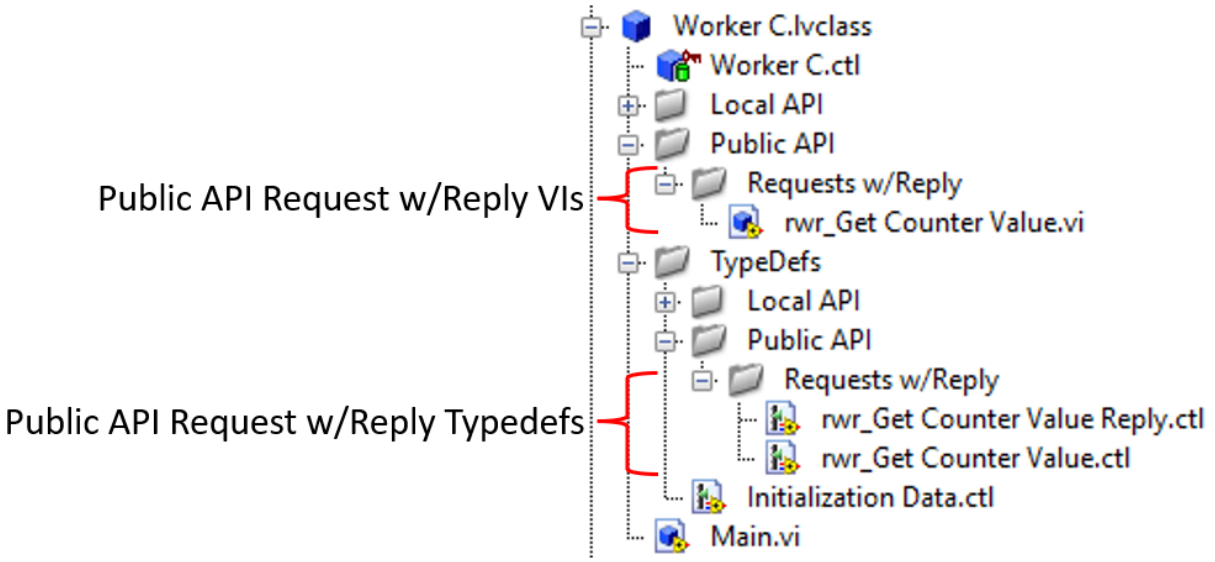

Figure 69 - Public Request with Reply folder structure within a Worker

Exercise 1.6 – Creating a Public Request

In this exercise, you will use the Public API Builder tool to create a Public Request in a Worker. You will then use the Public Request VI to send an asynchronous message to the Worker from the Worker's Caller.

Worker User Library tool

The **Worker User Library Tool** provides you with a collection of **functional Workers**—pre-developed Workers with public APIs—that are ready to use. These Workers help you to save development time and promote code reuse by offering Workers with reliable and tested functionalities that you can plug into your applications.

Additionally, the Worker User Library allows you to include third-party items, enabling you to create and add your own libraries of functional Workers to the tool. These can be reused in your projects or distributed to customers or the community.

In Figure 70, the **Items in User Library** list displays the available functional Workers and libraries that are supplied with the Workers SDK by default. Items in the list can be either single Workers (*.lvclass) or libraries (*.lvlib) containing multiple Workers. Selecting an item from the list will display detailed information about it in the information box on the right.

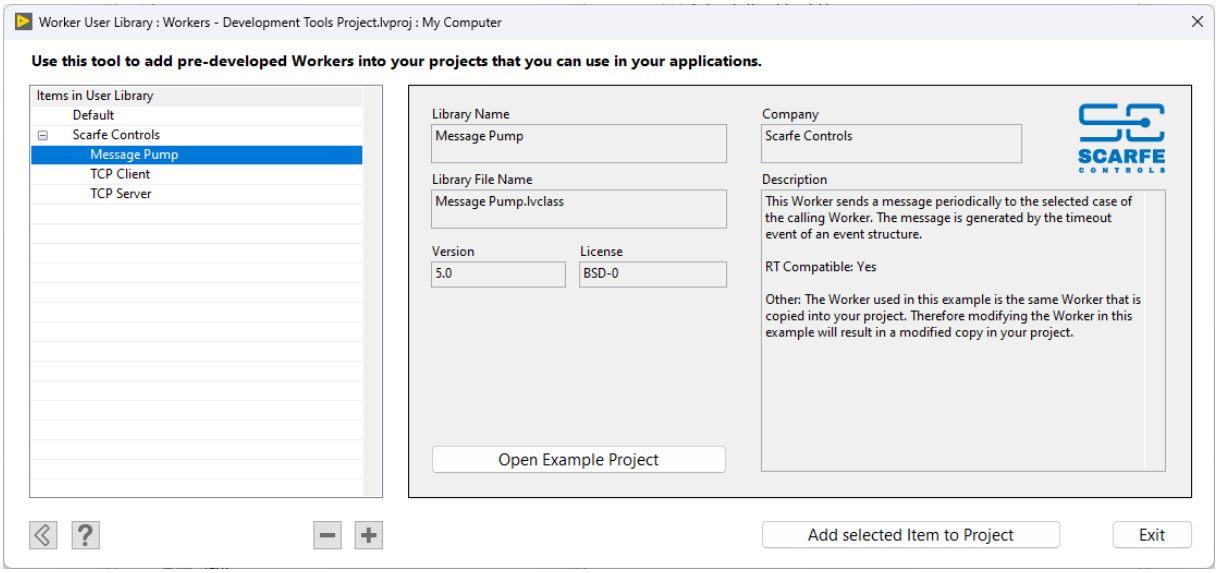

Figure 70 - Screenshot of the Worker User Library tool

Options

Items in User Library

Displays a list of available Workers or LabVIEW libraries that contain multiple Workers. Selecting an item populates the information box to the right with details about the selected item.

Open Example Project

If the selected item contains an example project, clicking this button opens it, providing practical insights into how the Worker functions.

Add selected Item to Project

Creates a copy of the selected Worker or library and adds it to your current project.

Default Functional Workers

The Worker User Library includes three default functional Workers that you can add to your projects:

Message Pump

This Worker is ideal for triggering periodic actions or updates within your application, such as refreshing data displays or polling sensors. The Worker sends a Public Response periodically to a selected MHL case of its Caller. The periodic generation of the Response message is created by the timeout event of an event structure.

RT Compatible : Yes, will run on an NI real-time target such as a Compact RIO (cRIO)

Example Project : Yes.

TCP Server

Useful for applications requiring network communication, such as remote monitoring systems or distributed data acquisition. The library provides a TCP Server Worker that can send and receive string data to and from multiple **TCP Client Workers**. It can handle connections to multiple clients simultaneously.

RT Compatible: Yes, will run on an NI real-time target such as a Compact RIO (cRIO)

Example Project : Yes.

TCP Client

Suitable for applications where you need to connect to a TCP server for data exchange, such as fetching configurations or reporting statuses from remote devices. This library offers a TCP Client Worker that can communicate with the **TCP Server Worker** above. It sends and receives string data, enabling interaction with a TCP Server over a local network.

RT Compatible: Yes, will run on an NI real-time target such as a Compact RIO (cRIO)

Example Project : Yes.

Config File Editor tool

The Worker User Library **Config File Editor** tool allows you to add your own functional Workers, Worker base classes, or Worker libraries to the **Worker User Library** tool. Each item in the Worker User Library requires a configuration file containing essential information about the item. This tool facilitates the creation and editing of these configuration files, enabling you to integrate your own custom functional Workers into your library.

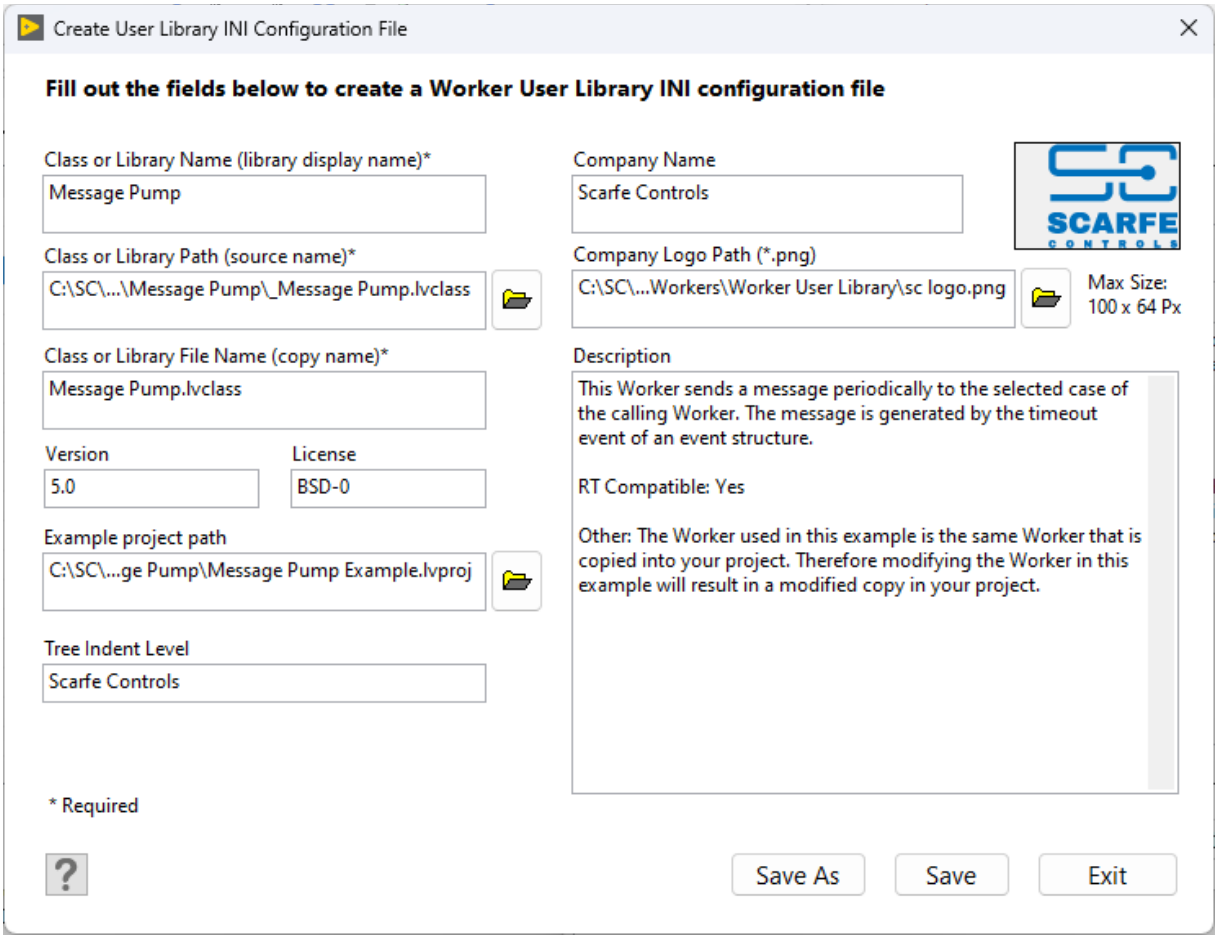

Figure 71 - Screenshot of the Worker User Library Config File Editor tool

Class or Library Name (library display name) **Required Field*

The name that will be displayed in the **Library Name** field and the **Items in User Library** tree list within the Worker User Library tool. E.g. Message Pump

Class or Library Path (source name) **Required Field (must be *.lvclass or *.lvlib)*

The path to the source file (class or library) that the tool will copy into a project. The source file's name cannot be the same as the name of the item that will be copied into the project to avoid naming conflicts.

It is recommended that you rename the source file by adding an underscore (_) to the front of the filename before adding it to the Worker User Library. For example, if your Worker is named *Message Pump.lvclass*, **manually** rename the source file to *_Message Pump.lvclass*.

Class or Library File Name (copy name) **Required Field (must be *.lvclass or *.lvlib)*

The name of the Worker or library that will be added to your project. This name will also be displayed in the **Library File Name** field of the Worker User Library tool.

Note: This name must be different from the source filename specified in the previous field called **Class or Library Path (source name)**. E.g. *Message Pump.lvclass*

Version *(Optional Field)*

The version number of the Worker or library. E.g. 1.0.0

License *(Optional Field)*

The license under which the Worker or library is distributed. E.g. MIT license

Example Project Path *(Optional Field)*

The path to an example project (*.lvproj) that demonstrates how to use the Worker or library.
E.g. *C:\LabVIEW Projects\Workers\Examples\Message Pump Example.lvproj*

Tree Indent Level *(Optional Field)*

Determines how the Class or Library Name is categorized in the **Items in User Library** tree list. An empty value places the item under the default category. You can organize items into custom categories using semicolons (;) to separate levels.
E.g. **My Company;cRIO** would categorize the item under "My Company" > "cRIO".

Company Name *(Optional Field)*

The name of your company, displayed in the **Company** field of the Worker User Library tool.
E.g. My Company Ltd

Company Logo Path *(Optional Field)*

The path to your company's logo image, which will be displayed in the Worker User Library tool for the item. The image must be a PNG file with a maximum size of 100x64 pixels.
E.g. *C:\Company Assets\Logos\MyCompanyLogo.png*

Description *(Optional Field)*

A detailed description of the Worker or library, providing information about its functionality and usage.

Exercise 1.7 – Add a Message Pump Worker to your Project

In this exercise, you will learn how to use the **Worker User Library** tool to add a **Message Pump** Worker to your project. You will then integrate the **Message Pump** Worker into the **Motor Drive** Worker that you created in Exercise 1.6.

Exercise 1.8 – Load a Worker Dynamically

In this exercise, you will learn how to load a Worker into your application at runtime. A second instance of the **Motor Drive** Worker will be loaded and communicated with.

More Development tools...

There are several additional tools included with the Workers SDK that can enhance your development workflow. While not all these tools are demonstrated in the course exercises, they are briefly presented here so you are aware of their existence and understand how they can assist you when developing Workers applications. These tools can be accessed through the **Workers Tools Menu**.

RT Worker Convert tool

The **RT Worker Convert** tool allows you to convert a Worker's Main VI between its **reentrant** (Main.vi) and **non-reentrant** (Main NR.vi) forms.

By default (for non-real-time targets), a Worker's Main VI is reentrant, allowing you to load and run multiple instances of a Worker's QMH within the same application. However, for development on NI real-time (RT) targets such as a Compact RIO, it is preferred to develop using non-reentrant Worker Main VIs (Main NR.vi) so that debugging of the Workers Main VI is possible in LabVIEW.

The RT Worker Convert tool allows you to convert a ***Main.vi*** to and from ***Main NR.vi***.

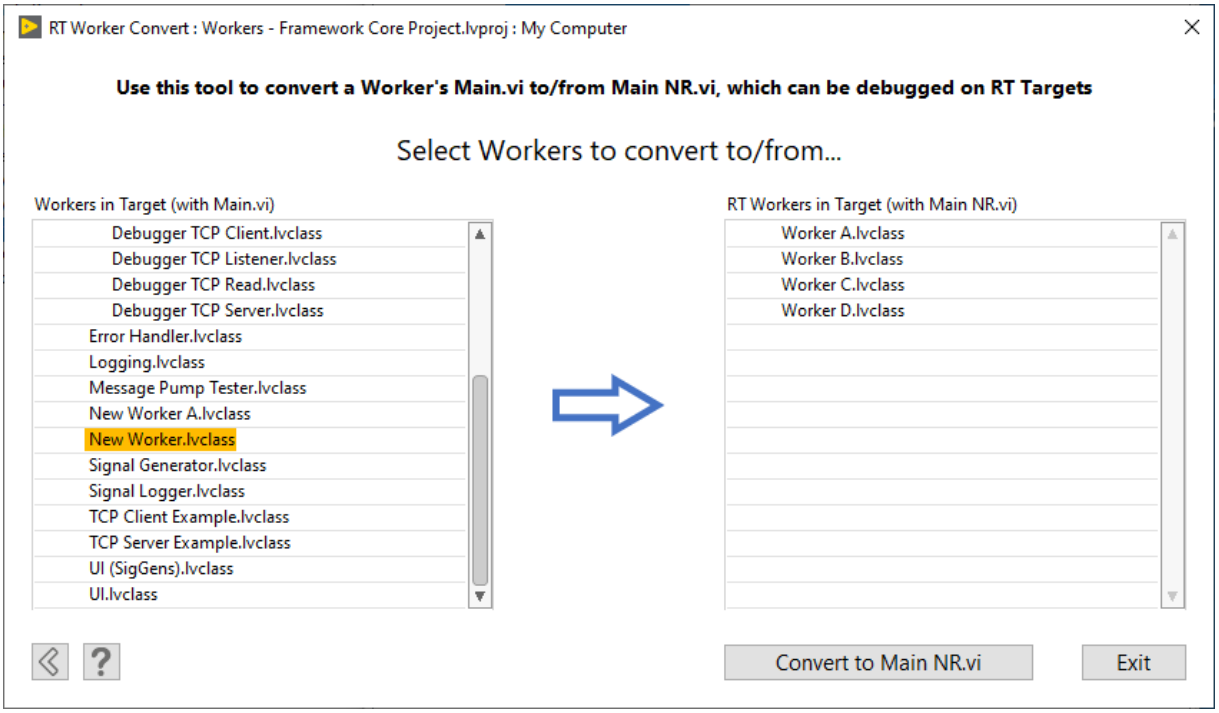

Figure 72 - Screenshot of the RT Worker Convert Tool, showing lists of Workers with Main.vi and Main NR.vi

When you open this tool, you will see the left list show Workers on your target with **reentrant Main VIs**. The list on the right will show Workers on your target with **non-reentrant Main VIs**.

Selecting one or more Workers in either list (mutually exclusive) and then pressing the **Convert to (Main VI)** button will replace the Worker's *Main.vi* with a *Main NR.vi* (or vice versa).

Important

You can use a Worker with a *Main NR.vi* the same way you would a Worker with a *Main.vi*. However, because the VI is non-reentrant, you cannot run multiple instances of a Worker with a *Main NR.vi* simultaneously.

Create Launcher VI tool

The **Create Launcher VI** tool allows you to create Launcher VIs for a Worker or Worker base class.

By default, the **Create/Add Worker** tool creates a Launcher VI for the **first** Worker you add to a blank project. However, you may want to create a Launcher VI for any Worker you wish to launch as a head Worker. This is particularly useful for testing a specific Worker and its Public API independently from the rest of the application.

Launcher VIs are saved in a folder called **Launchers** under your project's main directory.

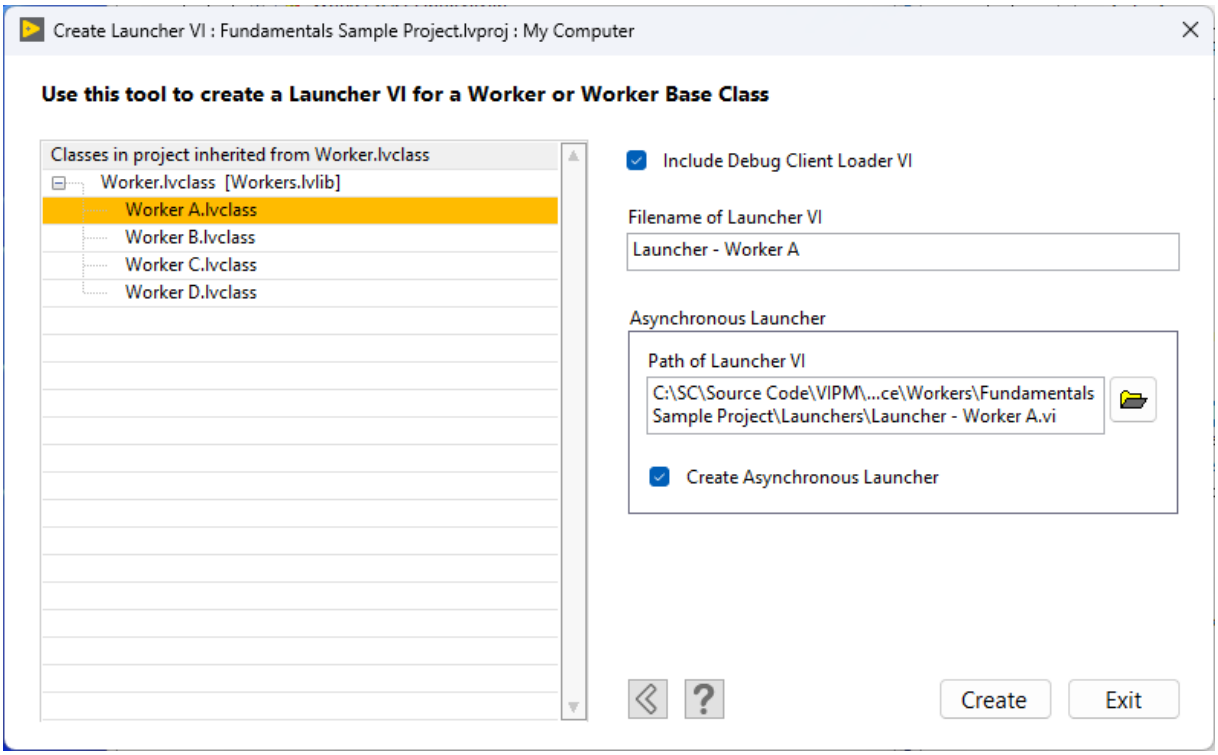

Figure 73 - Screenshot of the Create Launcher VI Tool

Classes in project inherited from Worker.lvclass

Select the Worker or Worker base class for which you want to create the Launcher VI.

Include Debug Client Loader VI

When selected, *Debug Client Loader.vi* is included in the Launcher VI.

Asynchronous Launcher VIs

Specify the path of the Launcher VI that will be dynamically loaded by the Asynchronous Launcher VI.

Create Asynchronous Launcher

Select this option to create an Asynchronous Launcher VI for the specified Launcher VI.

Change Worker Properties tool

The **Change Worker Properties Tool** allows you to modify certain properties of a Worker or Worker base class, which include:

Worker Name: Changing this property updates the Worker's name in multiple places:

1. **Class Filename**: The file name of the class is renamed
2. **Class Folder**: The folder on disk containing the class is renamed
3. **References**: All references to the class and its VIs/typedefs are relinked within the project
4. **Object Labels**: All class object labels on VI front panels (within the class) are renamed

Worker Alias: Change the name used by the Worker in a Workers application. This field is not available for Worker base classes.

Icon Header: Change the text and header colors displayed in the header section of the Worker's icon.

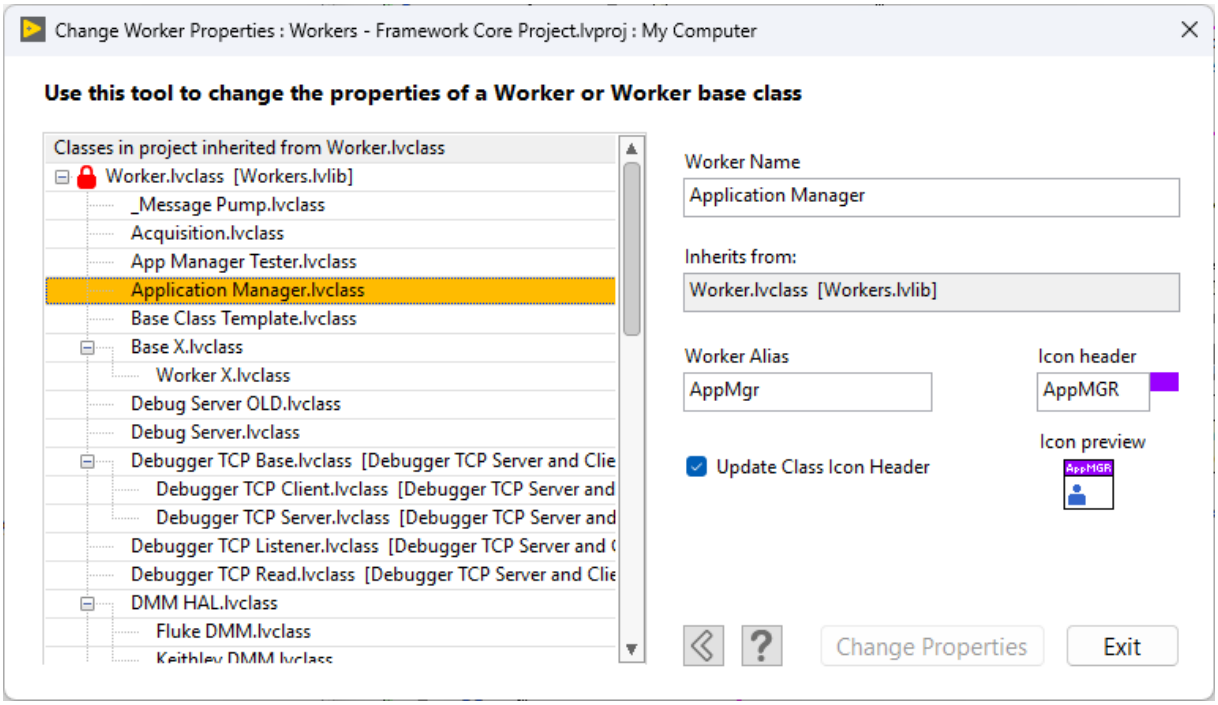

Figure 74 - Screenshot of the Change Worker Properties tool

QuickDrop Shortcuts

The following QuickDrop shortcuts have been created to help streamline your workflow and automate common tasks while developing with the framework.

Ctrl+0 Show Worker's Private Data Cluster

Use this QuickDrop shortcut whenever you want to quickly access the **private data cluster** of a Worker. Activate this shortcut when working on any VI within a specific Worker. This method is faster than searching for the Worker's private data cluster through the LabVIEW Project Explorer.

Ctrl+8 Go to MHL Case

This QuickDrop shortcut allows you to follow the flow of messages within a Workers application while developing in LabVIEW. Activating the **Ctrl+8** QuickDrop shortcut on one of the following VIs will jump directly to the VI's corresponding MHL case:

- **Case Labels**
- **Local API Request VI**
- **Public API Request VI**
- **Public API Request with Reply VI**
- **Public API Response VI** (this will jump to the location of the corresponding Response VTD VI)

Step 1: Select a Case Label or Worker API VI on a block diagram

Select a Case Label or Worker API VI on the block diagram of a VI.

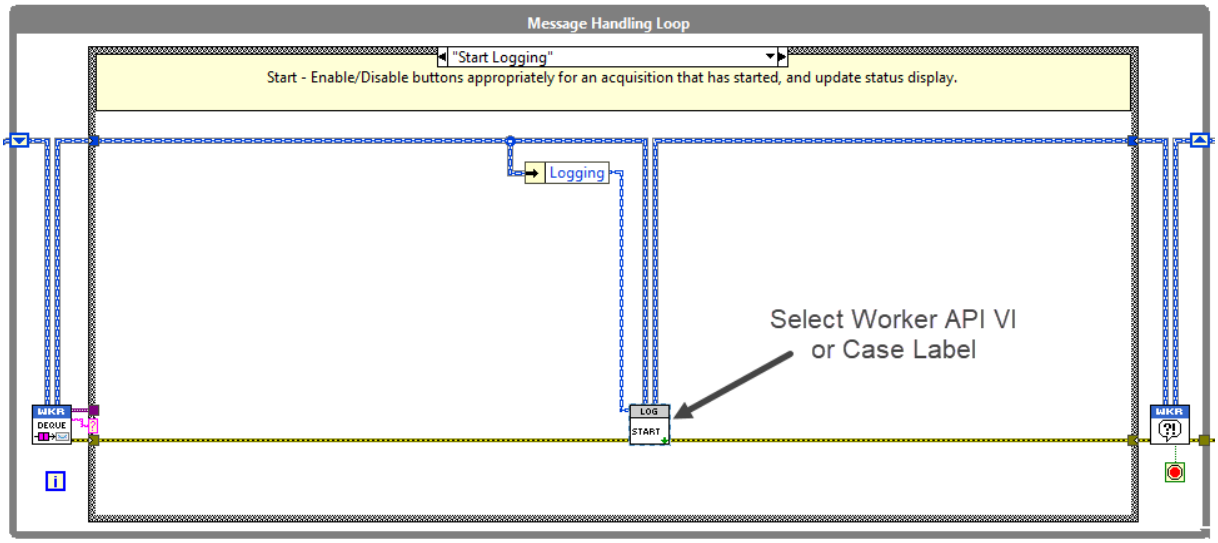

Figure 75 - Example of rqp_Start.vi (a Public Request VI) selected on a block diagram

Step 2: Activate QuickDrop shortcut Ctrl+8

With the VI selected, activate the **Ctrl+8** QuickDrop shortcut. The corresponding MHL case of the Worker API VI or Case Label will appear.

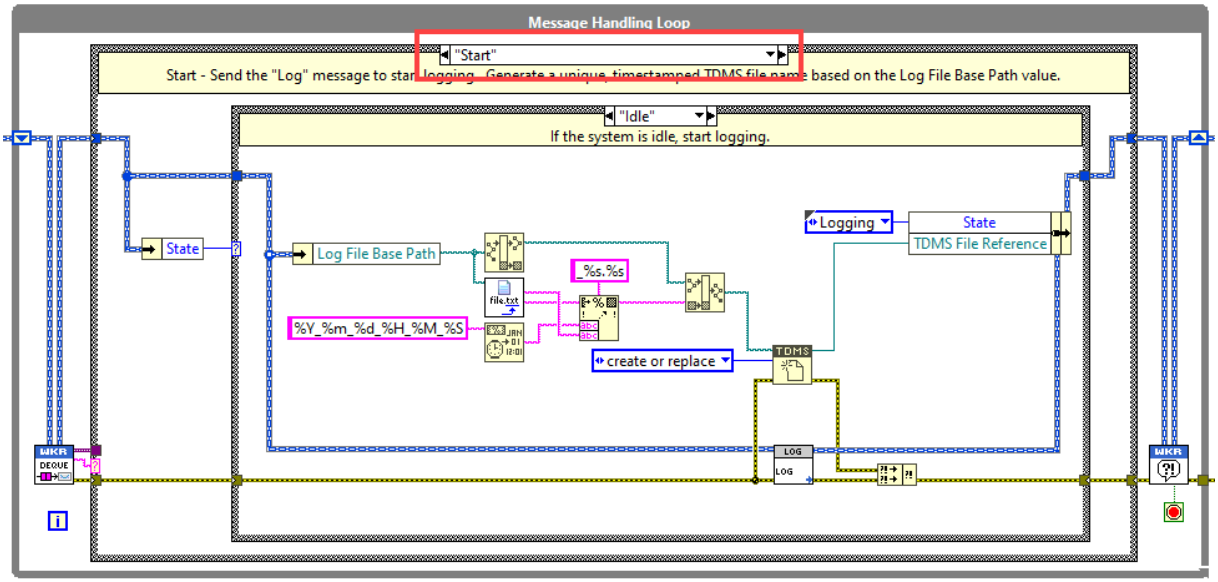

Figure 76 - The Public API MHL case corresponding to rqp_Start.vi appears after activating QuickDrop shortcut Ctrl+8

Ctrl+9 Open the Create Worker MHL Case tool

Activating the **Ctrl+9** QuickDrop shortcut from the block diagram of a Worker's **Main VI** will open the **Create Worker MHL Case** tool. This tool can be used to perform the following tasks (see Figure 77):

1. **Create New MHL Cases** (non-Public API MHL cases)
2. **Create Local API Requests**
3. **Create MHL Cases to receive Public API Response Messages**
4. **Edit non-Public API MHL Cases**
5. **Override non-Public API MHL Cases**
6. **Open the Public API Builder Tool**

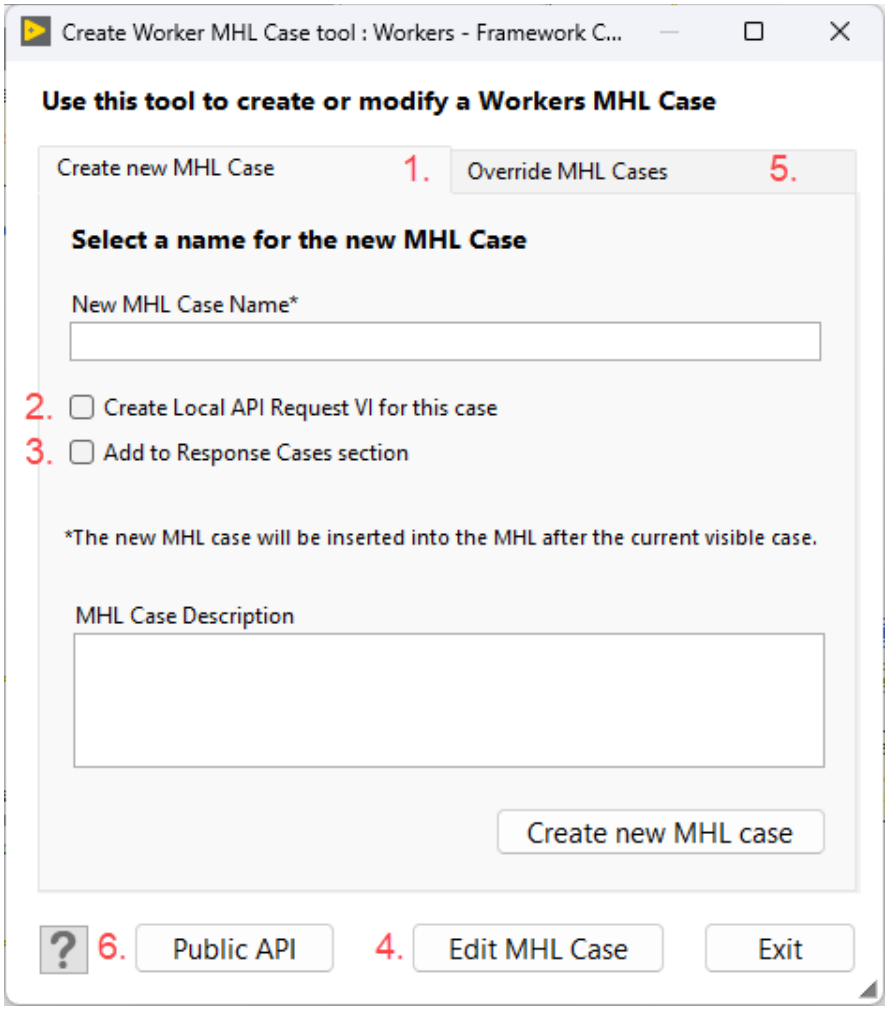

Figure 77 - Screenshot of the Create Worker MHL Case tool

Creating undefined MHL cases

You can use the **Create Worker MHL Case** tool to create an undefined MHL case with a Case Label (i.e. the MHL case is not used to receive a Local or Public API message). To create an undefined MHL case, simply **uncheck** the options at points **2.** and **3.** in Figure 77, and you will be able to create an undefined MHL case with a Case Label.

Part 2

The LVOOP features

Congratulations on completing the first part of the course! We hope you have gained a foundational understanding of what the Workers framework is and how its various components fit together, allowing you to create asynchronous, modular LabVIEW applications.

In the second part of the course, you will learn how to take your Workers application development to the next level by discovering the LabVIEW Object-Oriented Programming (LVOOP) features of the framework. This section will demonstrate how LVOOP can enhance the possibilities of what you can do with the framework to develop more flexible and maintainable applications.

Important Reminder for LVOOP Beginners

If you are not familiar with LVOOP programming practices or object-oriented programming (OOP) practices in other programming languages, we recommend taking some time to learn more about these topics at your own pace. A basic understanding of OOP is a valuable tool that will help you solve various programming problems when needed. While this section will discuss the benefits of using LVOOP in the Workers framework, it will not provide a general background on the benefits of object-oriented programming. Comprehensive information on OOP principles can be found online or in any good book on object-oriented programming. This training course focuses on providing you with a practical guide on how to use the various features of the framework, rather than offering an introductory course in OOP.

LVOOP Benefits

The benefits discussed here are common to developing with any object-oriented programming language and are not specific to LVOOP alone. Object-oriented development can offer straightforward solutions to complex programming problems, but it can also introduce unnecessary complexity to simple problems if overused. Therefore, in LabVIEW, LVOOP should be used thoughtfully as a tool where it adds significant value.

The Workers framework aims to find that **sweet spot**, employing just enough LVOOP to enhance the overall capabilities of the framework while avoiding overuse that could detract from the framework's useability.

Similarities between a LabVIEW Library and a LabVIEW Class

In Part 1, for developers without LVOOP experience, it was mentioned that "you can simply think of a LabVIEW class as a LabVIEW library that also has the benefit of being a LabVIEW datatype."

A **LabVIEW Project Library (*.lvlib)** is a container that programmers use to group a set of VIs and type definitions (typedefs) that are tightly coupled and meant to be used together.

Similarly, a **LabVIEW Class (*.lvclass)** shares many features with a LabVIEW library; in fact, a LabVIEW class is a specialized type of LabVIEW library.

Common Features between LabVIEW Libraries and LabVIEW Classes:

- **Grouping of Items**: Both group their owned items (VIs, typedefs, etc.) together, providing a modular container that clearly defines the purpose of the group.
- **Scope Control**: Both can define the scope of their items (public, private, etc.), which helps control access to their items from outside the library or class.
- **Namespace Provision**: Both provide namespacing for the items they own, preventing naming conflicts and aiding in modular code organization.

These similarities are demonstrated in Figure 79, showing the same contents existing within a LabVIEW library (left) and a LabVIEW class (right).

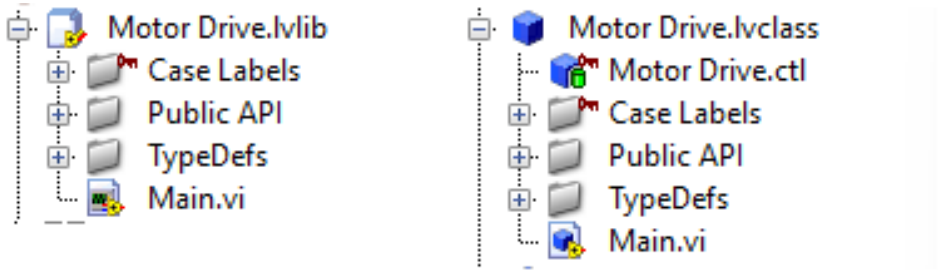

Figure 78 - (a) Items contained within a library *(b) Items contained within a class*

However, this is where the similarities end for our purposes. While both structures help in organizing and managing code, LabVIEW classes introduce additional object-oriented features such as inheritance, encapsulation, and polymorphism, which are not features of LabVIEW libraries. These LVOOP features will now be discussed.

LVOOP Features of a Worker

In the previous section, we discussed how a LabVIEW class shares many similarities with a LabVIEW library, such as grouping related items and providing scope control. However, a LabVIEW class goes beyond these capabilities by offering additional powerful features that significantly enhance the functionality of a Worker within the Workers framework.

Because a Worker is implemented as a LabVIEW class, it benefits from the following LVOOP features:

- A Worker is a LabVIEW datatype
- Encapsulation
- Inheritance
- Dynamic Dispatch (Overriding VIs)
- Abstraction

In this chapter, we will explore each of these LVOOP features in more detail, with practical examples demonstrating how they contribute to making a Worker more than just a modular container. By understanding and leveraging these features, you will be able to create more robust, maintainable, and scalable applications, ultimately enhancing the overall quality of your LabVIEW projects.

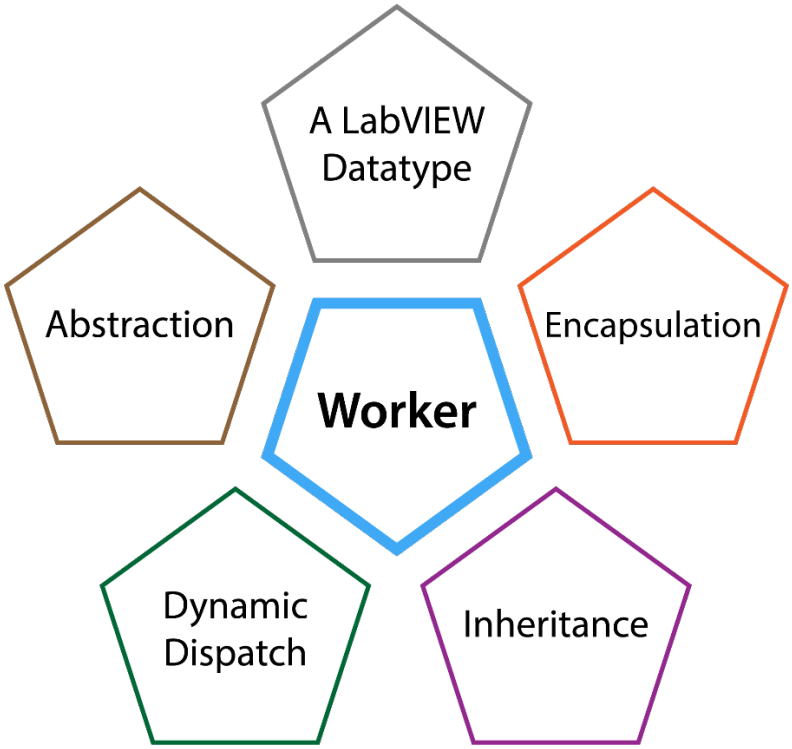

Figure 79 – The five LVOOP features of a Worker

A Worker is a LabVIEW datatype

One of the key benefits of implementing Workers as LabVIEW classes is that they become **LabVIEW datatypes**. Unlike a LabVIEW library, which is merely a container for grouping VIs and other resources, a LabVIEW class defines a new datatype and owns a cluster of data known as its **private data cluster** (Figure 80), which holds data specific to each instance of the class. This capability enables you to load multiple instances of a Worker, each with its own unique data.

In addition, you can place a LabVIEW class on the block diagram of a VI, where it exists as an object representing an instance of the class. Within the class's own VIs, you can use **bundle** and **unbundle nodes**—functions that allow you to read from or write to elements within a cluster—to access or modify the data stored in the Worker's private data cluster (Figure 81).

This feature is not possible with a LabVIEW library. A LabVIEW library cannot be dropped onto a block diagram as an object with its own datatype, nor does it own any private data natively linked to the library. By being a LabVIEW datatype, a Worker can:

- **Instantiate Multiple Objects**: Create multiple instances of the Worker, each with its own private data, facilitating reuseable and modular application design.
- **Maintain State Information**: Keep track of internal states and variables across different instances of an object.
- **Encapsulate Data**: The ability to maintain private data within each Worker instance not only allows for independent operation but also enhances data integrity and security through encapsulation, which we will explore in the next section.

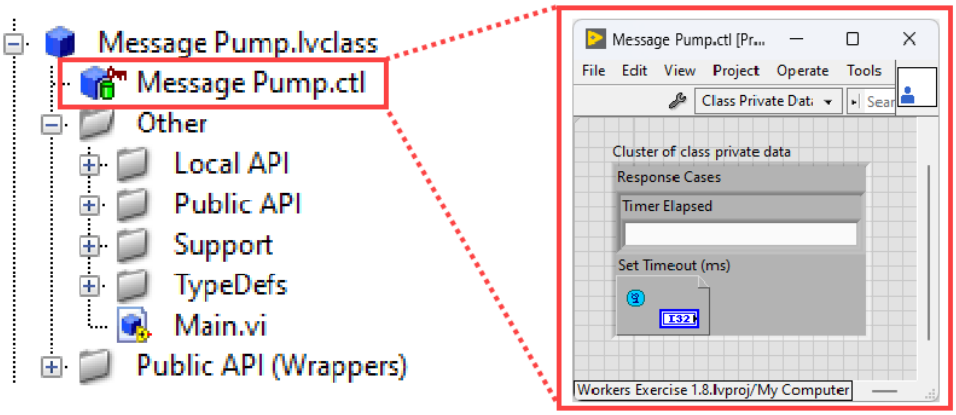

Figure 80 – The private data cluster of a Worker called 'Message Pump'

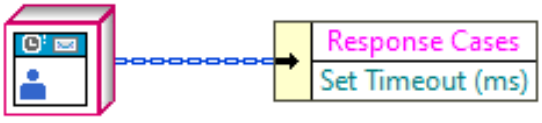

Figure 81 – a Worker object wired to an unbundle node on the block diagram of a VI owned by the Worker, demonstrating how to access the Worker's private data cluster

Encapsulation

Encapsulation is a fundamental concept in object-oriented programming that involves bundling data and the methods that operate on that data within a single unit called a **class**. It also involves restricting direct access to some of an object's components, thereby preventing accidental interference of the data by external entities. In the context of LabVIEW, libraries (*.lvlib) do not provide true encapsulation in this sense. They simply group files together without enforcing strict access control. While you can set the scope of items within a LabVIEW library, it is easy to unintentionally make certain items publicly accessible to code outside the library.

LabVIEW classes (*.lvclass), however, enforce encapsulation by default. They restrict access to a Worker's private data cluster by setting it to **private** scope, and this setting cannot be changed. This means that only the VIs that are members of the class can directly access and modify the class's private data using bundle and unbundle nodes. For example, consider a Worker class called **Message Pump**. As shown in Figure 82, its private data cluster is accessible within its own class VIs (right top), but attempting to access it from external VIs results in a broken wire (right bottom), indicating an error.

To allow external code to interact with a Worker's private data cluster, you must create **read/write accessor VIs** or provide access through a **property node** on the Worker's class wire. These methods offer controlled access to the private data, ensuring that any external interaction adheres to the intended usage and maintains data integrity.

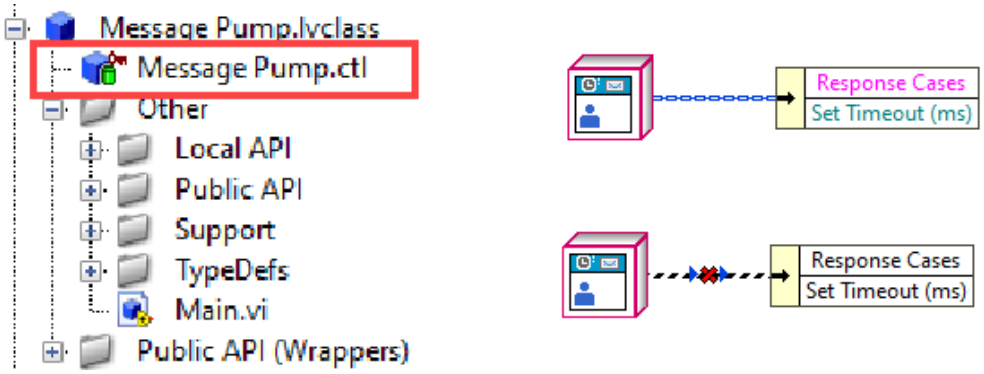

*Figure 82 - (left) Private data cluster of the class called **Message Pump**. (right top) Private data access within a class VI. (right bottom) Failed access of private data from an external VI, resulting in a broken wire.*

Furthermore, the use of a Worker's **Public API** in the Workers framework reinforces encapsulation. Data within a Worker, including its state, can only be requested to change via **Public Request** messages. The private data is not exposed by reference, so other Workers cannot directly modify it without sending the Worker a Public Request. This design ensures that all of a Worker's data and state are protected from direct modification by external code, enhancing the stability and reliability of each Worker.

In summary, encapsulation in LabVIEW classes hides the internal workings of a Worker, making the data within the object more secure and stable. It shifts the focus to **what** the Worker does rather than **how** it does it, promoting modularity, maintainability, and robustness in your applications.

Inheritance

Inheritance is a fundamental principle of object-oriented programming that allows new classes to acquire the properties and methods of existing classes. This promotes code reusability and provides a hierarchical structure to code through the clear separation of class abilities, leading to clearer and more organized software applications.

In the context of the Workers framework, inheritance plays a very important role. While every new Worker initially contains only one VI—its **Main VI**—a Worker actually contains over 150 VIs (see Figure 83). These additional VIs are contained in ***Worker.lvclass*** which is the common ancestor class of all Workers. This class exists in the **vi.lib** folder in LabVIEW, and every Worker inherits from it, allowing each Worker to utilize these VIs as well as its own.

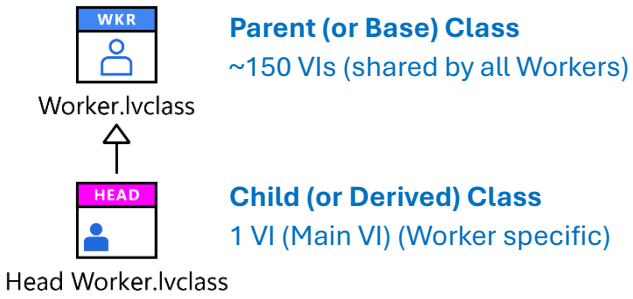

Figure 83 – A Worker's inheritance hierarchy. A Worker is made up of itself and all the classes that it inherits from.

The VIs from *Worker.lvclass* are recognizable as the **blue and white header VIs** used throughout a Worker's QMH on its Main VI (Figure 84). By leveraging inheritance, these common VIs are shared amongst all Workers, which significantly reduces code duplication and keeps the overall codebase of each Worker small and context specific.

For example, if each Worker required its own set of 150 VIs, then 10 Workers would result in a total of 1,500 VIs. However, by utilizing inheritance, these 10 Workers share the 150 VIs from *Worker.lvclass* and only need to contain their own specific VIs, resulting in a total of just 160 VIs (150 shared VIs plus 10 Worker-specific VIs). The larger the application, the greater the benefits in terms of reduced code size and maintainability.

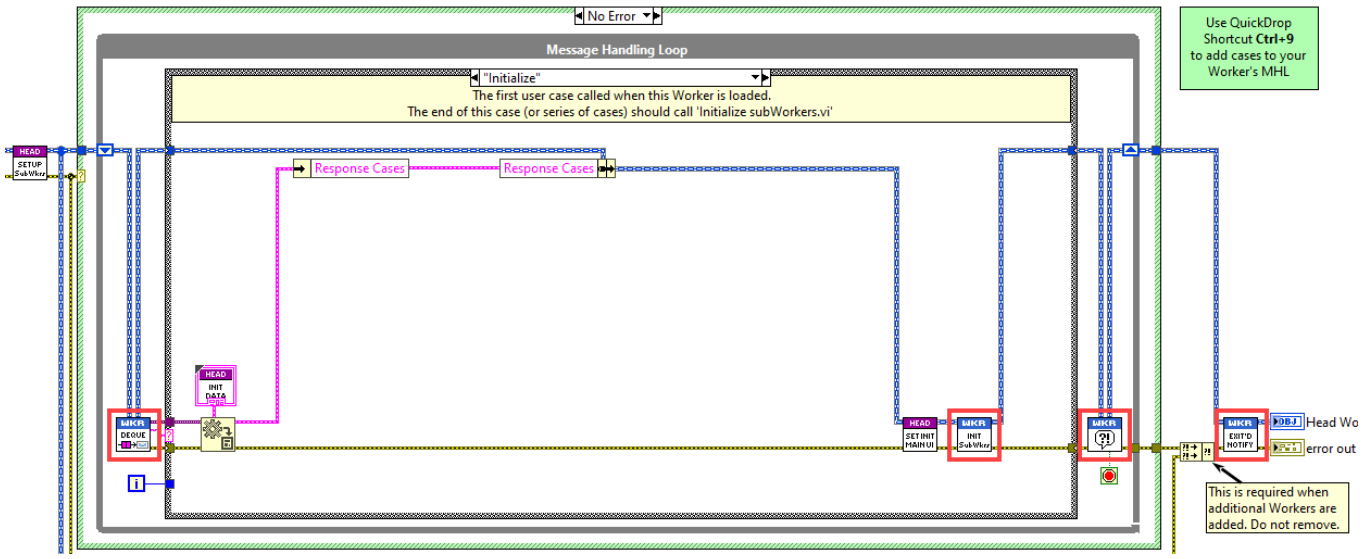

Figure 84 – Framework API VIs used by a Worker's QMH that are members of Worker.lvclass

Dynamic Dispatch (Overriding VIs)

Dynamic Dispatch is LabVIEW's implementation of the object-oriented programming concept called **polymorphism**. Polymorphism allows objects of different classes to be treated as instances of a common parent class, enabling them to override methods to perform class-specific behaviors. In LabVIEW, dynamic dispatch permits a child class to execute its own implementation of a VI that shares the same name as one in the parent class, as long as the child class defines a VI with that name.

When you have established an inheritance hierarchy between two classes in LabVIEW, you can add VIs to the child class to **override** VIs that exist in the parent class. This means that at runtime, LabVIEW will automatically use the child class's VI instead of the parent's when the VI is called on an object of the child class. This powerful feature allows you to extend or modify the behavior of inherited methods without changing the parent class's code.

There are approximately 40 Worker Palette VIs that can be overridden with your own extended implementation of these VIs. Below is an example of how VI overriding looks on the block diagram of a Worker's Main VI (Figure 85).

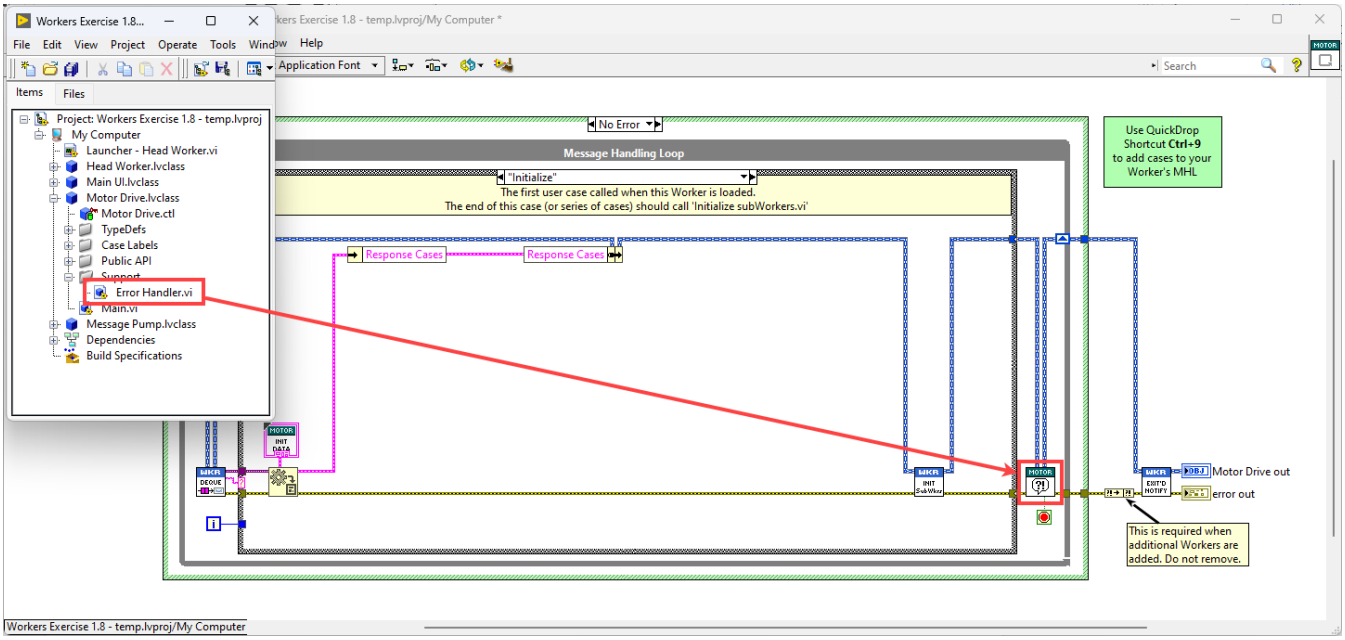

*Figure 85 – Overriding the framework API VI **Error Handler.vi** with a Worker's own implementation of the VI*

In Figure 86, the ***Worker.lvclass : Error Handler.vi*** (highlighted in the red square) is overridden by ***Motor Drive.lvclass : Error Handler.vi*** through dynamic dispatch. Since the Motor Drive Worker has its own *Error Handler.vi*, LabVIEW replaces the default *Error Handler.vi* from *Worker.lvclass* with the one defined in *Motor Drive.lvclass*. As a result, the *Error Handler.vi* on the block diagram now displays the icon header of the **Motor Drive** Worker instead of the generic *Worker.lvclass* header, indicating that it has been overridden. In the override VI, you can implement custom error handling specific to the **Motor Drive** Worker.

Later in the course exercises, you will learn how to override a Worker's Palette VI with your own implementation. This feature is also essential when creating API abstractions, a topic that will be discussed in the next section.

Abstraction

Finally, we arrive at **abstraction**, a concept that involves exposing only the essential features of a class while hiding its complex implementation details behind simple interfaces. Abstraction in LabVIEW is made possible through the combined use of inheritance and dynamic dispatch.

As discussed previously, inheritance allows child classes to inherit features from a base class, and dynamic dispatch enables VIs to be automatically selected at runtime based on the actual class type. In practice, abstraction allows you to create **abstract VIs** in a base class without providing any implementation. These abstract VIs act as placeholders—or contracts—that must be fulfilled by the child classes that override them, providing specific implementations.

This approach is particularly useful for creating APIs. For example, an API can be defined in a base class as a set of abstract VIs that represent the interface, with the actual implementation provided in the child classes. This effectively creates an abstraction layer, such as a Hardware Abstraction Layer (HAL) for hardware devices, where different hardware implementations can share the same set of API VIs while providing their own specific implementations for the API.

Figure 86 illustrates a Worker call-chain diagram showing how a hardware abstraction layer is created in a Worker base class and plugged-in to a Workers application through its Public API. Workers that inherit from the base class can be loaded at runtime to provide different implementations while using the same set of Public API VIs defined in the base class.

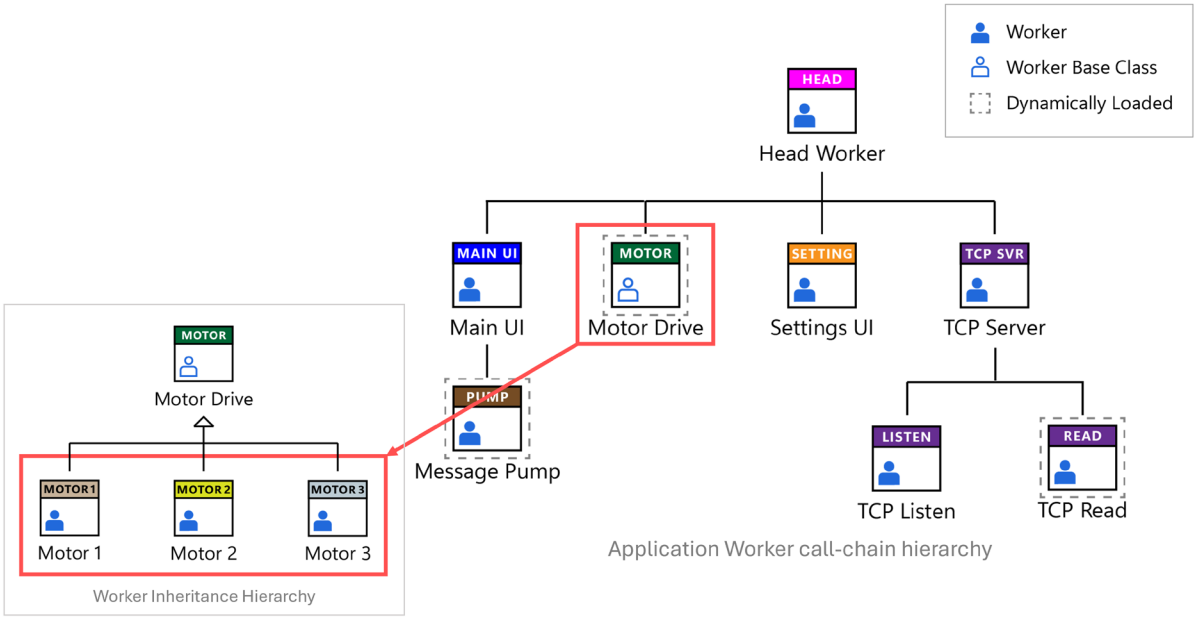

Figure 86 - An example of a hardware abstraction layer for a motor drive in a Workers application

Later in the course, you will create an API abstraction in the form of a Hardware Abstraction Layer (HAL), demonstrating how to implement LVOOP abstraction using the framework's **Public API Builder** tool. This hands-on experience will help solidify your understanding of abstraction and its practical applications within the Workers framework.

Worker Base Classes

How can we leverage inheritance in the framework? As discussed, a Worker not only has access to the VIs and typedefs within its own class but also to those contained within the classes above it in its inheritance hierarchy. By default, every Worker inherits from *Worker.lvclass*. Figure 87 shows the class inheritance hierarchy of all the Workers in a Workers application, where each Worker inherits from *Worker.lvclass*.

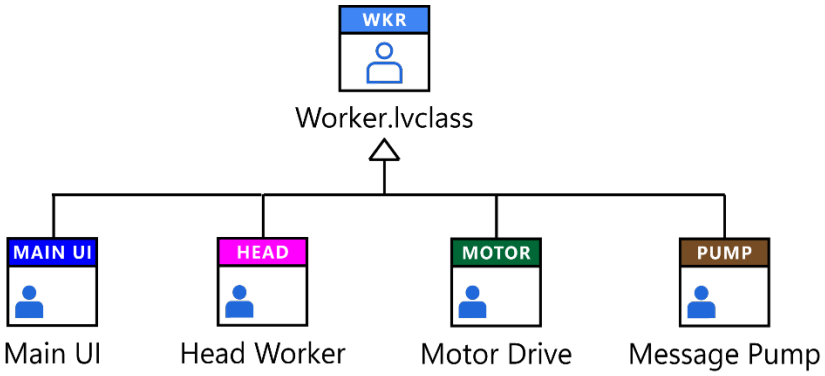

Figure 87 – an example of an application's Worker inheritance hierarchy

Each of the four Workers in Figure 87 contains the VIs and typedefs within their own classes and also inherits the VIs and typedefs from *Worker.lvclass*. The VIs and typedefs within *Worker.lvclass* can also be used by all Workers that inherit from it. This illustrates the power of inheritance in object-oriented programming: the ability of a class to inherit functionality from a parent (aka base) class. Each Worker extends the common functionality of *Worker.lvclass* with its own specific functionality.

Now suppose we want to add our own custom common VIs and typedefs to every Worker in an application or to a subset of Workers that require the same set of abilities. By adding a common **Worker base class** into our Worker inheritance hierarchy, we can achieve this. We can insert custom common code into a Worker base class and place this base class into the inheritance hierarchy ***between*** *Worker.lvclass* and each Worker that needs to use the base class's common functionality.

Figure 88 illustrates a Worker base class called **Worker Base Class** inserted into the application's Worker inheritance hierarchy. Any common code that we want to share can be added to **Worker Base Class** and then used by all the Workers that inherit from it.

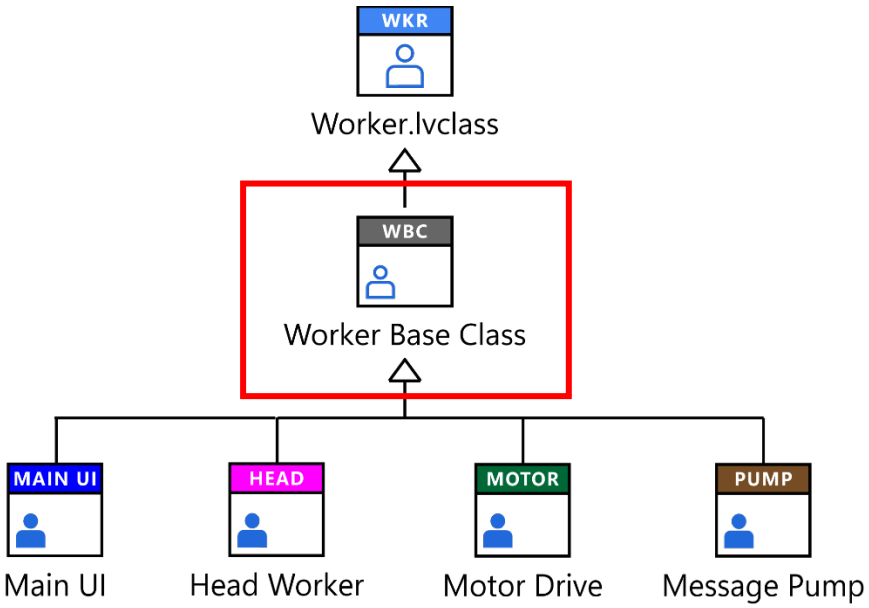

Figure 88 - Worker base class inserted into application's inheritance hierarchy

Parts of a Worker Base class

You can create a new Worker base class using the **Create Worker Base Class** tool available in the Workers Tools menu. When you create a new Worker base class, you will notice that the class contains the following default items (Figure 89):

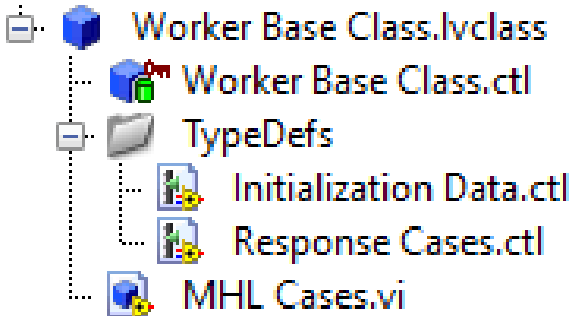

Figure 89 – Default items of a new Worker base class

Worker base class.lvclass

Like a Worker, a Worker base class is a LabVIEW class that inherits from the common ancestor class of all Workers, *Worker.lvclass*, although sometimes not directly if multiple Worker base classes inherit from each other. i.e. one base class inherits directly from another base class.

Initialization Data.ctl

This typedef serves the same purpose that it does for a Worker, however, it is used instead for the base class. *Initialization Data.ctl* is a typedef that is used to pass data into the base class's private data cluster when a Worker is initialized.

Response Cases.ctl

Similar to its role in a Worker, this typedef holds multiple Message Handling Loop (MHL) case names belonging to a Worker's Caller. It is used to register Public API Response messages so that asynchronous messages can be sent from the Worker to its Caller.

MHL Cases.vi

This VI contains the MHL cases of the base class. By default, this VI exists in every Worker in the <Default> MHL case of a Worker's Main VI. It allows you to add common MHL cases to a Worker without directly adding these cases to the MHL case structure on a Worker's Main VI.

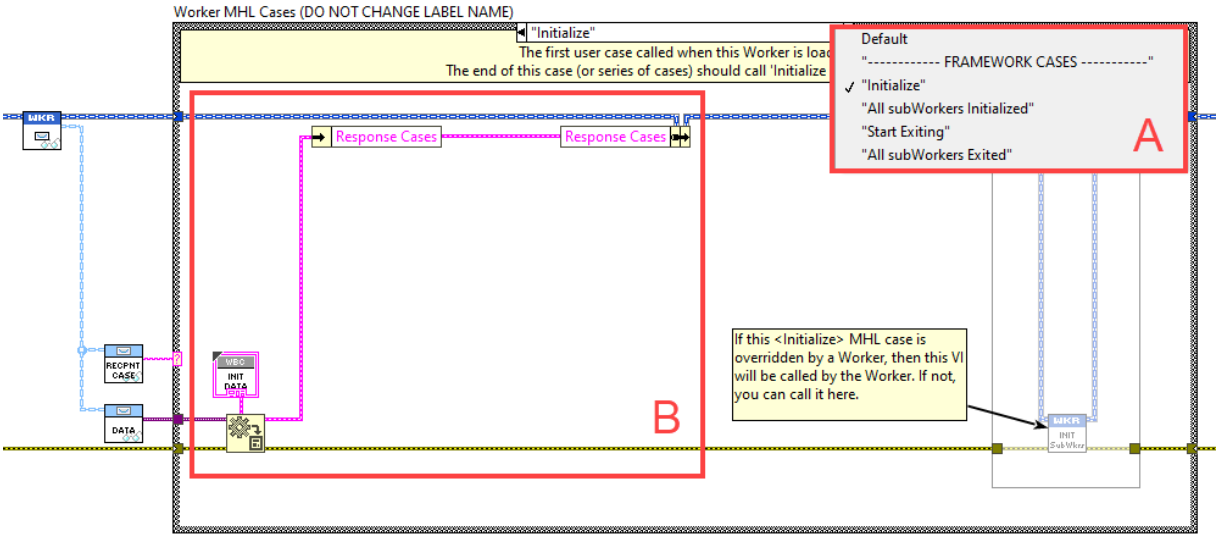

Figure 90 - the block diagram of a new MHL Cases.vi

Figure 90 shows the block diagram of a new *MHL Cases.vi*.

A. As with a Worker's MHL, a Worker base class contains the same four default framework cases:

1. <Initialize>
2. <All subWorkers Initialized>
3. <Start Exiting>
4. <All subWorkers Exited>

These cases ensure that if you remove one of the four default framework cases from a Worker's MHL, the case will be handled in the Worker base class instead. You have the option to run any of these cases in a Worker, in a Worker base class, or both by including the base class's *MHL Cases.vi* in the corresponding MHL case of a Worker's Main VI.

Additional MHL cases can be added to a base class's *MHL Cases.vi* in the same way you would add new MHL cases to a Worker's Main VI.

B. You can pass the base class's *Initialization Data.ctl* into the base class just like you do with a Worker. You can then write this data to the base class's private data cluster.

Good to know

Notice in Figure 89 that a Worker base class doesn't have a Main VI. This is because the purpose of the base class is to provide a collection of methods (VIs) and datatypes (typedefs) that can be used by a Worker. The base class can also provide additional MHL cases for a Worker but cannot replace the actual QMH of the Worker. A Worker can only have one QMH, which is contained within the Worker's Main VI. The base class supports the Worker by offering shared functionality without acting as an independent process.

Exercise 2.1 – Creating a Worker Base Class

In this exercise, you will learn how to create a Worker base class and insert it into your application’s inheritance hierarchy, allowing Workers to inherit from it.

Overriding Framework API VIs

Overriding the Workers framework API VIs provides a simple way to extend or modify the default functionality of the framework by implementing your own versions of these VIs. There are approximately 40 dynamic dispatch API VIs in the Workers framework that you can override with custom implementations. These override VIs can exist in either a Worker base class or a Worker that you create.

Most of the Workers framework API VIs are included in the Workers palette and some exist behind the scenes. Figure 91 shows how you can access and override a Workers framework API VI in a Worker that inherits from *Worker.lvclass* (as all Workers and Worker base classes do).

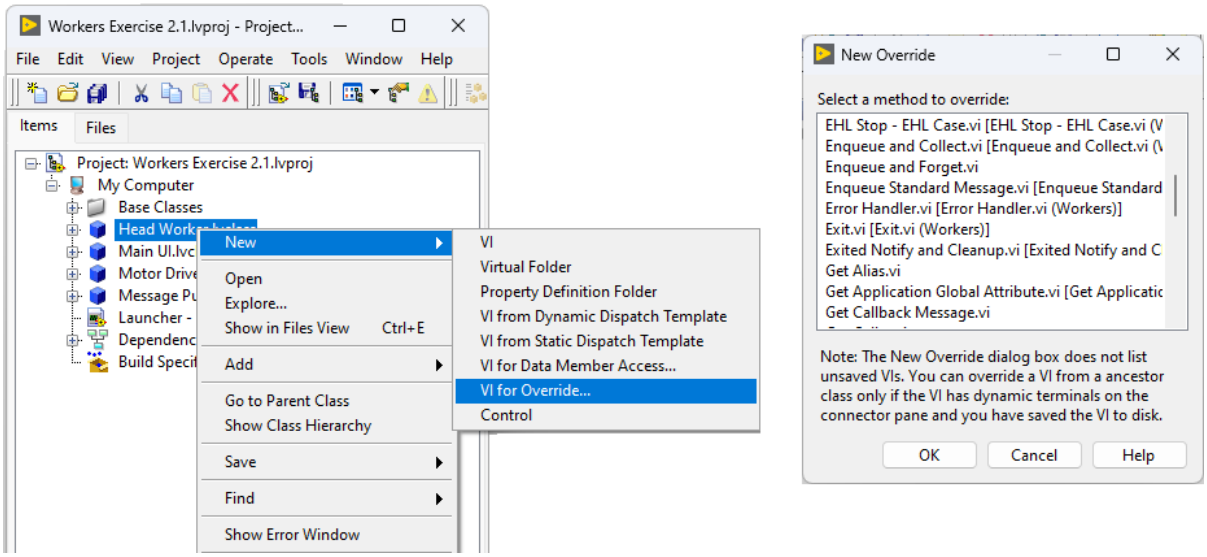

Figure 91 - Accessing the overridable Workers framework API VIs

Examples of Overridable Framework API VIs

Here are a few examples of Workers framework API VIs that you can override with your own implementations and reasons why you might wish to do so (see Figure 92):

A. Dequeue Message.vi

- **Purpose**: The VI waits until a message exists in the Worker's message queue, and dequeues the messages in the enqueued order based on the priority of the message. This VI then feeds the messages into the case structure of a Worker's Message Handling Loop (MHL) for processing.
- **Why Override**: If you want to apply pre-filtering on messages received by the Worker's priority message queue, this is the VI to override. For example, you might implement state filtering so that if a Worker is not in the correct state for a particular operation, the message is not passed into the MHL's case structure for processing.

B. Error Handler.vi

- **Purpose**: This VI receives errors passed into it that occur in a Worker's MHL and EHL. The VI forwards these errors to the Workers Debug Server and the framework's Error Logger (if included in the application's Launcher VI). It does not decide what action to take when an error occurs.
- **Why Override**: By overriding this VI in a Worker or Worker base class, you can implement custom error handling tailored to your application's needs. For example, you might choose to pass all MHL and EHL errors up to a Worker's Caller for processing.

C. Exited Notify and Cleanup.vi

- **Purpose**: This VI is the last VI that should be called in every Worker's Main VI. It ensures that all framework related resources are properly released when the Worker exits, by cleaning up all references that were created for the Worker when the Worker was initialized by its Caller.
- **Why Override**: If you have additional cleanup tasks that need to be performed before the Worker's Main VI exits—such as releasing custom references—you can override this VI and add your cleanup code here.

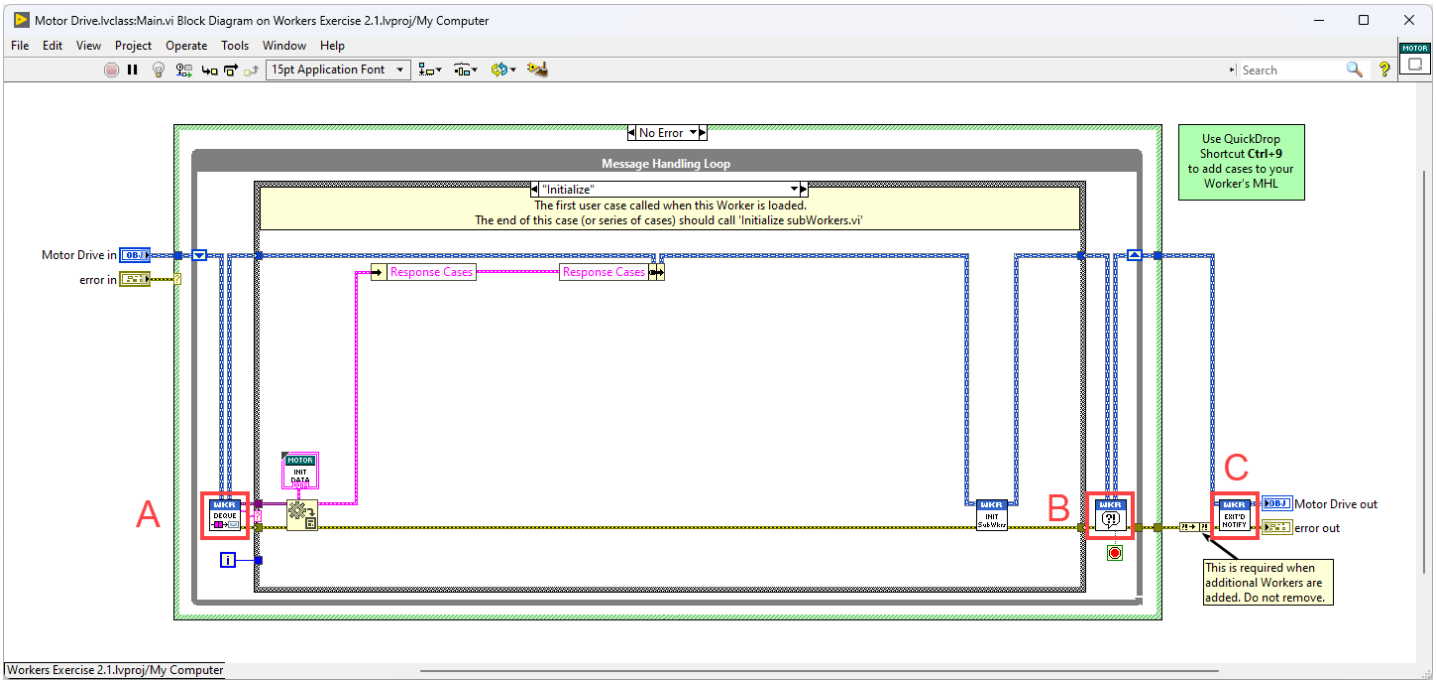

Figure 92 - Three Workers framework API VIs that you can override

Exercise 2.2 – Overriding the framework's Error Handler VI

In this exercise, you will learn how to override the framework's Error Handler VI with your own customized implementation of the Error Handler VI in a Worker base class.

Public API Requests in a Worker base class

Using **Worker base classes** in your applications provides numerous advantages. Functionality added to a Worker base class can be utilized by all Workers that inherit from that base class, promoting code reuse and consistency across your application. For example, in a previous exercise, you were able to replace every single *Error Handler.vi* in every Worker throughout the entire application by simply overriding the framework's *Error Handler.vi* in a Worker base class.

When you create a **Public API Request** directly in a Worker, you are developing a Public Request VI that sends a message to a specific Message Handling Loop (MHL) case of that Worker. In this Worker's MHL case, you add code that implements the functionality for the specific Public Request. However, the code within a Worker's MHL case is ***unique*** to that Worker and cannot be shared with other Workers, as it does not exist in a Worker base class.

In the Workers framework, you can also create Public API Requests within a Worker base class and add the corresponding code into the base class's *MHL Cases.vi*. These Public Requests defined in the Worker base class can be sent to any Worker that inherits from the base class, allowing you to create a set of common Public Request implementations that provide common functionality to multiple Workers.

For example, you might have a Public Request called <Show Front Panel> defined in the Worker base class's *MHL Cases.vi* (Figure 93). This Public Request can be used by all Workers that inherit from the Worker base class to display their front panels when needed.

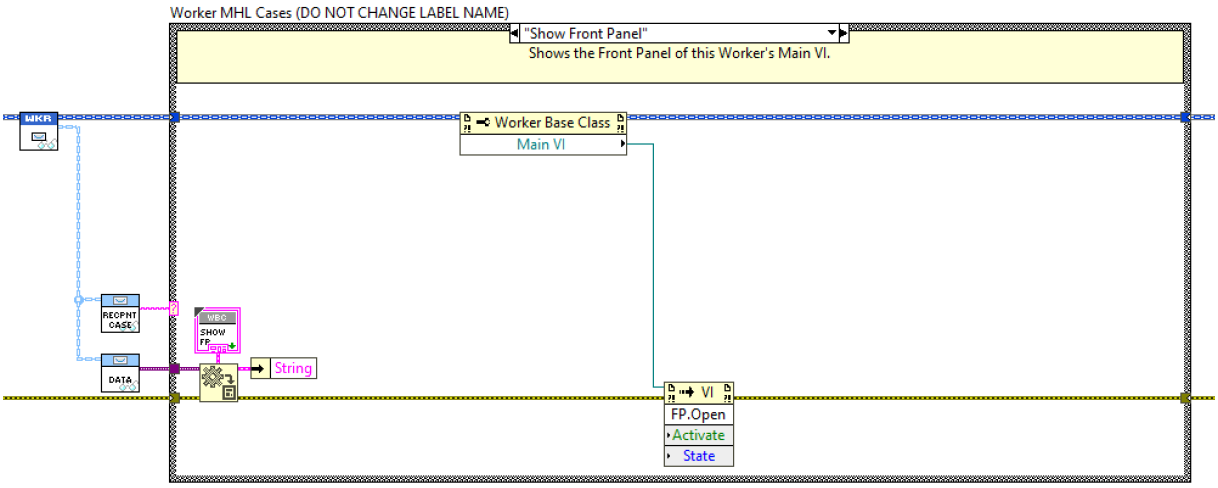

*Figure 93 - Example of a Public Request MHL case called <Show Front Panel> in a Worker base class's **MHL Cases.vi**. This Public Request can be used by all Workers that inherit from the Worker base class.*

Exercise 2.3 – Creating a Public Request in a Worker base class

In this exercise, you will learn how to create a Public API Request in a Worker base class that can provide shared functionalities for all Workers that inherit from the base class.

API Abstractions and HALs

API abstraction involves creating a simplified, high-level interface for interacting with complex systems. This **abstraction layer** allows developers to utilize a system's functionality without needing to understand or manage its complex internal workings. The core idea is to hide the complexity of underlying operations and present a set of well-defined, user-friendly methods or functions—a public API—that developers can easily integrate into their applications.

Advantages of Using API Abstractions

API abstractions offer numerous benefits that make them essential in modern software development:

Simplification of Complex Systems

API abstractions provide a high-level interface that enables developers to perform complex operations using simple commands. This simplification frees developers from needing to understand the inner workings of the system, allowing them to focus on solving business problems rather than managing technical complexities. By simplifying interactions with complex systems, API abstractions help reduce development time and lower the likelihood of errors.

Improved Maintainability

API abstractions decouple the underlying system's implementation details from its interface, allowing changes to be made without impacting the code that depends on the API. This separation of concerns ensures that developers can update or refactor parts of the system without causing widespread issues across the application. Consistent interfaces across updates keep the application stable, making it easier to maintain over time and reducing the risk of bugs or incompatibilities.

Enhanced Interoperability and Integration

In today's interconnected technological landscape, interoperability is critical. By providing a uniform interface, API abstractions make it easier to integrate diverse systems—including third-party services, legacy systems, and new technologies—into an existing application. This interoperability minimizes the need for extensive code rewrites and facilitates smoother integration of different platforms and tools, making applications more adaptable and scalable.

Improved Testing and Debugging

API abstractions allow for isolated testing of individual components, simplifying the process of identifying and fixing issues. By focusing on testing the API interface rather than delving into the complexities of the underlying system, developers can more easily trace and resolve problems. This modular approach makes debugging more efficient and the development process more reliable overall.

Hardware Abstraction Layers (HALs)

Hardware Abstraction Layers (HALs) are a practical and essential use case of API abstraction, particularly in systems where software needs to interact with various types of hardware. A HAL provides a uniform interface between the software and hardware, abstracting the differences among various hardware implementations and allowing the software to operate independently from the specific details of the underlying hardware.

A HAL sits between an application's core logic and hardware devices. Its primary function is to abstract hardware specifics—such as device registers, protocols, or control commands—into a set of generic, high-level functions. These functions provide the necessary operations to interact with hardware, such as reading sensor data, controlling actuators, turning devices on or off or managing communication interfaces.

For example, consider a software system that needs to communicate with different models of temperature sensors, each with its own communication protocol or data format. A HAL would provide a consistent API that abstracts these differences, allowing the software to retrieve temperature data through a universal Public API, regardless of the sensor model being used.

Implementing HALs with Workers for LabVIEW

In Workers 5.0, Worker HALs can be natively integrated into the framework's modular hierarchy architecture. Each Worker interacts with hardware components through standardized Public APIs existing in Worker base classes, allowing the same Public API to be used by different hardware implementations. This integration enhances flexibility and strengthens the robustness of the system by reducing the effort required to adapt to new hardware devices.

For example, in a system designed to interface with various types of data acquisition devices, a HAL provides a consistent Public API for configuring devices, starting and stopping data acquisition, and retrieving measurement data. The underlying complexity of dealing with different device drivers, communication protocols, and data formats is hidden behind the HAL, allowing developers to build flexible and scalable applications without being bogged down by hardware differences.

By decoupling the software API from hardware details, HALs promote hardware independence, improved portability, enhanced maintainability, and easier testing. These qualities make HALs a crucial component in developing scalable and maintainable software solutions, particularly in environments where hardware configurations need to remain flexible.

Conclusion

API abstractions and hardware abstraction layers play a vital role in modern software development by simplifying complex systems, improving maintainability, enhancing security, and promoting scalability. Within the Workers framework, these concepts allow developers to build flexible and maintainable applications that can adapt to changing hardware configurations and evolving technological demands.

As software systems continue to grow in complexity, the importance of API abstractions and HALs will only increase, making them essential tools in a developer's toolkit. By leveraging these concepts, you can create robust applications that are easier to develop, test, and maintain—ultimately leading to more successful and sustainable software projects.

Creating an API Abstraction using Workers

Creating an **API abstraction** with Workers for LabVIEW allows you to develop scalable and maintainable applications by decoupling the application's core logic from specific modules. This section of the course will guide you through the process of implementing a **Hardware Abstraction Layer (HAL)** using multiple Workers and a single Worker base class.

Part 1: The Application Architecture

Before creating a HAL, it's important to understand where it will fit within your application's Worker call-chain hierarchy. Consider the example application illustrated in Figure 94.

In this application, there are seven Workers and one Worker base class called **Motor Drive**. The **Motor Drive** Worker base class provides a standard set of Public API VIs that are plugged into **Head Worker**, in the same way you would connect a Worker to a Caller using the Worker's Public API VIs. However, unlike a Worker, the **Motor Drive** Worker base class does not implement any functionality within its *MHL Cases.vi*. Instead, it serves as an **abstraction layer** with each Public API VI containing only message enqueuing code. This means that while the **Motor Drive** Worker base class defines the set Public API VIs for a hardware device, it does not specify which Worker will receive and process the messages.

The advantage of this approach is that developers can create an application and specify a Public API ***without*** directly linking it to a specific Worker. This separation allows for flexibility; different hardware implementations can be integrated later without altering the core logic of the application.

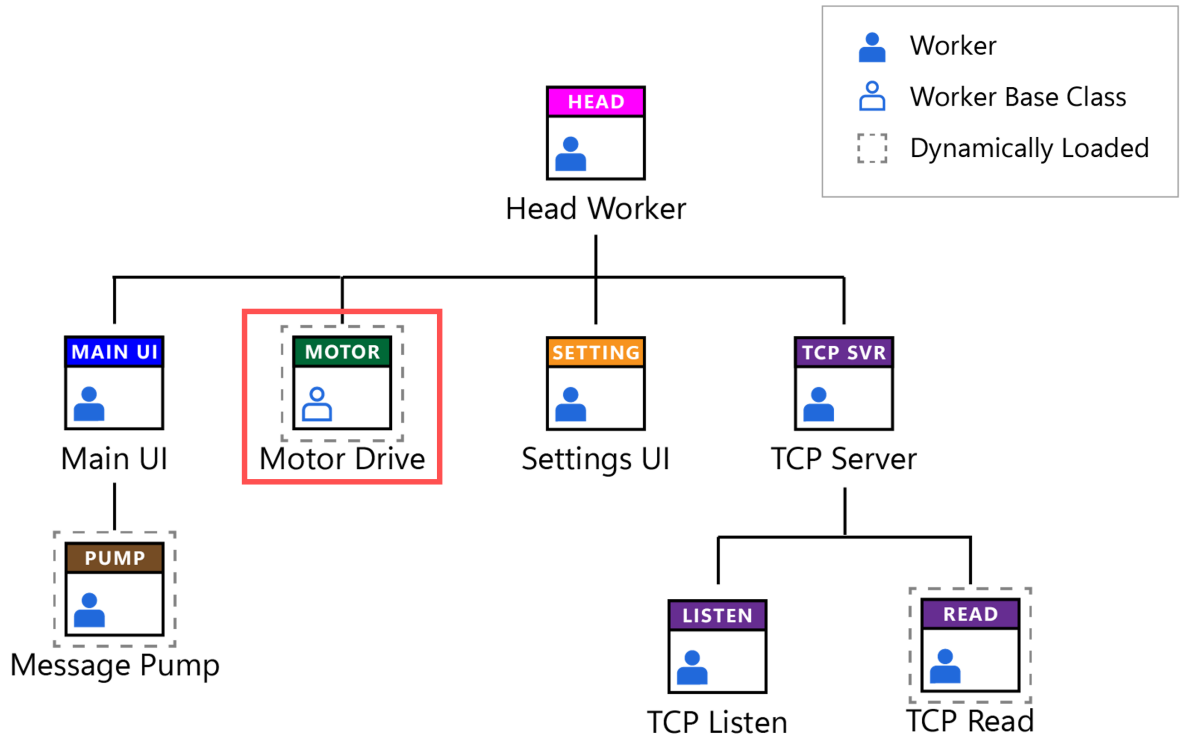

*Figure 94 – Application's Worker call-chain hierarchy with a HAL implemented via the **Motor Drive** Worker base class*

Part 2: The API Abstraction Implementation

Implementing the application's Worker call-chain hierarchy involves creating specific Workers that inherit from the **Motor Drive** Worker base class. As shown in Figure 95, three Workers—**Motor 1**, **Motor 2**, and **Motor 3**—inherit from the **Motor Drive** base class. Each of these Workers contains a Main VI, and within each Main VI are the Message Handling Loop (MHL) cases that receive and process the messages defined by the Public API VIs in the **Motor Drive** base class.

By inheriting from the **Motor Drive** base class, these Workers **commit** to implementing the functionalities promised by the base class's Public API VIs. This setup allows you to send request messages to the MHL cases of these Workers using the single set of Public API VIs defined in the base class. Essentially, the **Motor Drive** base class acts as a Hardware Abstraction Layer (HAL) for any Worker that inherits from it, providing a consistent interface while allowing for different underlying implementations.

The takeaway here is that we need two parts to create a HAL using Workers:

1. **A Set of Public API VIs in a Worker Base Class**: These VIs define the interface and are responsible for sending messages to Workers.
2. **Workers That Inherit from the Worker Base Class**: These Workers implement the functionality for the Public API in their own MHL cases, providing the specific hardware interactions for the Public API.

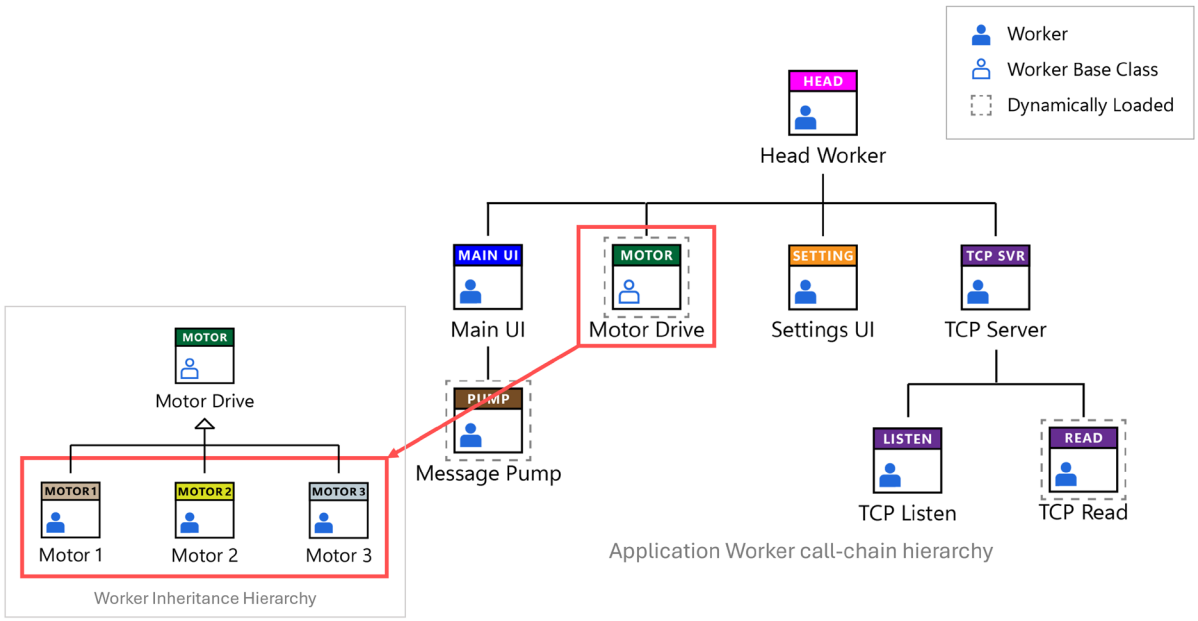

*Figure 95 – The **Motor Drive** Worker base class used as a Hardware Abstraction Layer for different motor Workers*

Part 3 : Using the API Abstraction

Once the API abstraction is established, you can use the **Motor Drive** Public API VIs to interact with any of the Workers that inherit from the base class. These Public API VIs can be directly integrated into a Workers application or called externally from generic LabVIEW code or applications capable of invoking LabVIEW VIs, such as NI TestStand.

As illustrated in Figure 96, the code allows the developer to first select which implementation (i.e., which Worker) of the **Motor Drive** to load—**Motor 1**, **Motor 2**, or **Motor 3**. The next step involves using an Asynchronous Launcher VI created for the Worker base class, to dynamically load the selected Worker. Because all three Workers share the same class wire as the **Motor Drive** Worker base class, you can seamlessly use the **Motor Drive** Public API VIs to send messages to the selected Worker.

This approach demonstrates that you can **utilize a single set of Public API VIs for multiple Workers** without the need to create separate Public API VIs for each Worker or manually add code to determine which set to execute.

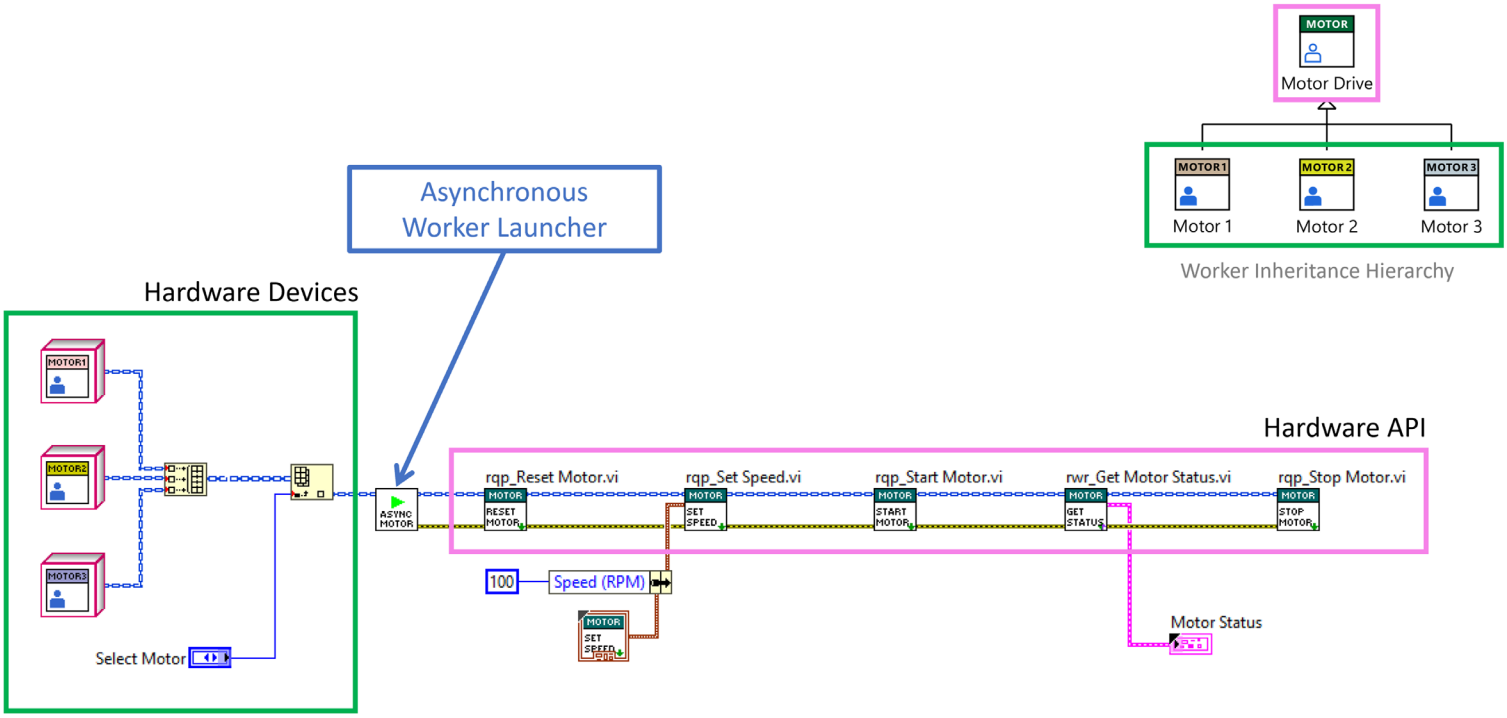

*Figure 96 - Using the **Motor Drive** HAL (Public API VIs) to send messages to different motor Workers*

Part 4: Scalability and Maintainability using HALs

Creating API abstractions using Workers for LabVIEW helps you to build applications that are both scalable and maintainable. Adding new hardware device implementations, such as additional motors, does not require changes to the application's core logic. For example, you could create new Workers called **Motor 4**, **Motor 5**, and **Motor 6** that inherit from the **Motor Drive** Worker base class. These Workers can be dynamically loaded at runtime, and your application's Worker call-chain hierarchy remains unchanged. If your application has been compiled into a packed project library, you do not need to recompile it to utilize the functionality of these additional Workers.

This method allows for future expansion and adaptation without changing the application's existing functionality. Developers can focus on creating new hardware implementations in additional Workers while using the established base class Public API and core application logic.

Exercise 2.4 – Creating a Hardware Abstraction Layer

This is the last exercise of the course. You will start from a blank project and create an API abstraction for three different digital multimeters (DMMs) using a Worker base class and three Workers.

You will also learn how to create an Asynchronous Launcher VI and use the returned Worker's handle with the HAL Public API VIs to communicate with a running Worker.

Congratulations! 🎉

Well done on completing the **Workers for LabVIEW Training Course**! We hope you found the course both informative and engaging. You can refer back to this course manual at any time you wish to gain a deeper understanding of a particular topic.

What's Next?

You might be wondering where to go from here and what additional resources are available to help you continue your journey with the Workers framework.

Join the Workers Community Forum (https://community.workersforlabview.io/)

We encourage you to engage with other developers on the **Workers Community Forum**. This is a great place to:

- **Ask Questions**: If you encounter challenges or have questions as you start developing your own Workers projects, the Workers Community Q&A forum is there to help.
- **Share Knowledge**: Contribute your insights and solutions to assist others.
- **Stay Updated**: Keep up to date with the latest developments, software updates, and best practices within the Workers framework.

Explore Example Projects

There are numerous example projects that have been created to demonstrate how to apply the concepts you've learned to solve real-world software engineering problems. In addition to the example projects that are supplied with the Workers 5.0 SDK, there are additional example projects available for download from the **Workers Community GitHub repositories**. Some of the available Workers example projects include the following:

Available in Workers 5.0 from the Workers tools menu:

Fundamentals Sample Project: Heavily commented, this example project is a great starting project for new developers, demonstrating how to use Worker APIs to send messages between Workers.

Continuous Measurement and Logging: Created to be a side-by-side comparison (using Workers) of the Continuous Measurement and Logging sample project that ships with LabVIEW.

TCP Server and Client: Demonstrates how to use the TCP Server and Client Worker libraries (available from the Worker User Library tool) to stream data over TCP between multiple Workers applications.

DMM HAL Demo Project: Demonstrates how you can create an API abstraction for a DMM, by creating an API abstraction in a Worker base class, and multiple implementations in Workers that inherit from the Worker base class.

Available on the Workers Community GitHub Repo (https://github.com/w4lv-community)

Embed subWorker into subpanel: Demonstrates how to dynamically load a Worker at runtime and embed its front panel into a subpanel of the Worker's Caller.

Recursive Public API Requests: Demonstrates how to create recursive Public API Requests in a Worker base class, allowing you to send a message throughout an application's entire Worker call-chain.

cRIO TCP Server-Client Example: Demonstrates how you can use the Workers TCP Server and Client libraries to stream data between a host PC and an NI real-time embedded target.

Interface DMM HAL example: Demonstrates how to use a LabVIEW interface to create an API abstraction for a Worker's Public API.

Workers 5.0 Debug Server application (flagship example) : The source code of the Workers Debug Server application is available as an open-source project developed using Workers 5.0. Exploring this project will give you a deeper understanding of how to build complex applications with the framework. (https://github.com/w4lv-community/Workers-Debug-Server)

Final Thoughts

Congratulations once again on your achievement! You now have the tools and knowledge to create well architectured LabVIEW applications that can make a significant impact in your field.

We look forward to seeing the innovative solutions you'll develop. Remember, the community is here to support you every step of the way.

Good luck in all your future projects! 😊